Построчный разбор пакета ml_freq

gtm_detector/ml_freq — 27 модулей, 215 классов, функций и методов. Собрано 02.11.2026 командой `python docs/ml_freq_walkthrough/build.py`.

Содержание

Как устроен пакет

__init__.py

__main__.py

config.py

- DataConfig
- DecoderConfig
- TransformerConfig
- TrainingConfig
- ExperimentConfig

data/__init__.py

data/models.py

- Interval
- Interval.covers
- WellSeries
- WellSeries.well_id
- WellSeries.dates
- WellSeries.__len__
- WellSeries.target
- WellSeries.training_mask

data/readers.py

- normalize_id
- parse_date
- MeasurementsReader
- MeasurementsReader.__init__
- MeasurementsReader.read
- MeasurementsReader.read_parameter
- MeasurementsReader._read_states
- LabelsReader
- LabelsReader.__init__
- LabelsReader.read
- WellRepository
- WellRepository.__init__
- WellRepository.load
- WellRepository._attach
- WellRepository._fingerprint

data/statistics.py

- MergedStatisticsReader
- MergedStatisticsReader.__init__
- MergedStatisticsReader.read
- MergedStatisticsReader._read_sheet
- HalfYearStatisticsReader

HalfYearStatisticsReader.__init__
HalfYearStatisticsReader.files
HalfYearStatisticsReader.read
_workbook_stream
_chain
_sheet_table
_join_periods
_header_date
_position
_numbers
_number
LabelRecord
LabelRecord.key
LabelRecord.censored
MonitoringLabelsReader
MonitoringLabelsReader.__init__
MonitoringLabelsReader.read
MonitoringLabelsReader.rejection
_text

data/repair.py

FrequencyCheck
FrequencyCheck.confirms
FrequencyCheck._working
Evidence
Evidence.share
Segment
Segment.days
Segment.decision
LabelPoint
BlockShiftRepair
BlockShiftRepair.__init__
BlockShiftRepair.__repr__
BlockShiftRepair.segments
BlockShiftRepair.assess
BlockShiftRepair.apply
BlockShiftRepair._label_evidence
BlockShiftRepair._reference_match
BlockShiftRepair._decide

data/dataset.py

DataAudit
MergedWellRepository
MergedWellRepository.__init__
MergedWellRepository.load
MergedWellRepository.build
MergedWellRepository._build
MergedWellRepository._repair
MergedWellRepository._prefilter
MergedWellRepository._verify
MergedWellRepository._wells
MergedWellRepository._reference
MergedWellRepository._telemetry_share
MergedWellRepository._dropped_days
MergedWellRepository._labels_table
MergedWellRepository._fingerprint

features/__init__.py

features/builder.py

- FeatureBuilder
- FeatureBuilder.sequence_size
- FeatureBuilder.tabular_names
- FeatureBuilder.sequence
- FeatureBuilder.tabular
- FeatureBuilder._channels
- FeatureBuilder._relative_to_past
- FeatureBuilder._difference_to_past
- FeatureBuilder._since_last_step
- _ratio

models/__init__.py

- model_registry
- load_model

models/base.py

- EventSegmenter
- EventSegmenter.__init__
- EventSegmenter.fit
- EventSegmenter.predict_proba
- EventSegmenter.save
- EventSegmenter.load
- EventSegmenter.file_suffix

models/classic.py

- SklearnSegmenter
- SklearnSegmenter.__init__
- SklearnSegmenter.fit
- SklearnSegmenter.predict_proba
- SklearnSegmenter.save
- SklearnSegmenter.load
- classic_models

models/transformer.py

- PositionalEncoding
- PositionalEncoding.__init__
- PositionalEncoding.forward
- TimeSeriesTransformer
- TimeSeriesTransformer.__init__
- TimeSeriesTransformer.forward
- TransformerSegmenter
- TransformerSegmenter.__init__
- TransformerSegmenter.file_suffix
- TransformerSegmenter.n_parameters
- TransformerSegmenter.fit
- TransformerSegmenter.predict_proba
- TransformerSegmenter.save
- TransformerSegmenter.load
- TransformerSegmenter._sample_crops
- TransformerSegmenter._collate

evaluation/__init__.py

evaluation/decoding.py

- SegmentDecoder
- SegmentDecoder.__init__
- SegmentDecoder.decode
- SegmentDecoder._runs

SegmentDecoder._merge

evaluation/metrics.py

_share
_mae
Score
Score.recall
Score.precision
Score.f1
Score.__add__
Score.as_dict
EventMatcher
EventMatcher.__init__
EventMatcher.match
EventMatcher.score

evaluation/experiment.py

ModelResult
ModelResult.as_row
CrossValidator
CrossValidator.__init__
CrossValidator.folds
CrossValidator.run
CrossValidator._evaluate
CrossValidator._rows

evaluation/report.py

ComparisonReport
ComparisonReport.__init__
ComparisonReport.table
ComparisonReport.to_text
ComparisonReport.save
ComparisonReport._plot

audit.py

DataAuditReport
DataAuditReport.__init__
DataAuditReport.segments_table
DataAuditReport.non_numeric_by_month
DataAuditReport.to_text
DataAuditReport.rejected_table
DataAuditReport.save
DataAuditReport.plot
TrainingDataReport
TrainingDataReport.__init__
TrainingDataReport.labels_table
TrainingDataReport.wide
TrainingDataReport.target_table
TrainingDataReport.save
_evidence
_indent
build_parser
add_arguments
run
main

cli.py

_run_dir
_registry

load_wells
run_compare
run_train
run_predict
_find_weights
build_parser
main

tests/conftest.py

day
make_well
wells

tests/test_data_and_features.py

test_target_and_training_mask
test_features_shapes_and_values
test_parsing_helpers

tests/test_models_and_evaluation.py

test_decoder_merges_gaps_and_marks_open_end
test_matcher_one_to_one_by_start
test_transformer_forward_shape
test_transformer_fit_predict_save_load
test_sklearn_models_learn_the_effect
test_cross_validation_runs_for_all_models

tests/test_repair.py

test_frequency_check
_shifted_block
test_repair_finds_and_undoes_shift
test_repair_drops_segment_contradicting_labels
test_seam_takes_neighbour_decision
test_censored_label

tests/test_statistics.py

_ole_file
test_workbook_stream_follows_chain_longer_than_declared
_Sheet
_Sheet.__init__
_Sheet.row_values
_Sheet.row_types
_sheet
test_sheet_table_keeps_only_numbers
test_join_periods_makes_continuous_calendar
test_reader_requires_files

Как устроен пакет

Задача

На нефтяных скважинах с насосом ЭЦН технологи периодически **поднимают частоту** двигателя (например, с 48 до 50 Гц), чтобы получить больше жидкости. Это геолого-техническое мероприятие (ГТМ) «увеличение частоты F». Технолог записывает в итоговом файле дату начала мониторинга и дату окончания мониторинга: период, пока эффект от подъёма заметен в суточных замерах (дебит жидкости и нефти, обводнённость, давления).

Пакет `gtm_detector/ml_freq` решает обратную задачу: **по суточным замерам скважины найти мероприятие и поставить даты начала и окончания эффекта так, как их ставил бы технолог.** Модель для каждого дня отвечает «этот день внутри эффекта или нет», а затем из вероятностей по дням собираются интервалы.

Особенности, из-за которых код устроен именно так:

- **Данных немного.** В обучение попадает 424 мероприятия на 369 скважинах (раньше, на склеенном файле, было 135 на 122), поэтому модели небольшие, а проверка всегда идёт по скважинам, а не по дням.
- **Разметка неполная.** Технолог размечает не все реальные подъёмы частоты, поэтому «вне разметки» не значит «эффекта нет». Модели учатся только на днях вокруг размеченных мероприятий, а точность (precision) занижена намеренно.
- **Источник данных — исходные полугодовые выгрузки.** В прежнем склеенном файле часть выгрузок была сдвинута на 180 дней (это исправляет `BlockShiftRepair`, но только для склеенного файла); в 14 исходных `.xls` 2020–2026 сдвига нет.
- **Есть опорный уровень.** Детектор на правилах (ветка `feature/increase-f-gtm`) находит 89 % мероприятий и точно ставит дату начала в 78 %. Модели сравниваются с ним.

Путь данных

```
Excel
  resources/statistics/<год>/*.xls            - замеры по дням, 14 полугодовых файлов 2020-2026, листы Qж,
Обв, Qн, Рлин, Fэцн, Рзаб
  resources/labeled_data.xlsx                 - разметка технологов («Лист1»)
        |
        v   HalfYearStatisticsReader / MonitoringLabelsReader (data/statistics.py)
  таблицы «скважина x дата» + записи разметки
        |
        v   BlockShiftRepair                                  (data/repair.py)
  только для склеенного .xlsx: блоки, сдвинутые на 180 дней, возвращены на место или исключены
        |
        v   MergedWellRepository                              (data/dataset.py)
  список WellSeries: таблица frame (дни x ряды) + labels (интервалы)
        |
        v   FeatureBuilder                                    (features/builder.py)
  15 каналов (Transformer) или 106 признаков (scikit-learn) на каждый день
        |
        v   модели: transformer, logreg, random_forest, extra_trees, gboost   (models/)
  вероятность «день в эффекте» для каждого дня
        |
        v   SegmentDecoder                                    (evaluation/decoding.py)
  интервалы: начало, окончание (или «эффект продолжается»)
        |
        v   EventMatcher, Score, CrossValidator               (evaluation/)
  сопоставление с разметкой (допуск +-7 дней), метрики, перекрёстная проверка по скважинам
        |
```

```
        v   ComparisonReport, audit                          (evaluation/report.py, audit.py)
  отчёты в results/ml_freq/
```

Пути к данным берутся из `config.py`, вручную их не набирают (в прежних именах файлов встречалась латинская «c» вместо кириллической).

Модули пакета

Модуль	Что делает
`config.py`	Все настройки в неизменяемых dataclass: пути, декодер, Transformer, обучение, эксперимент
`data/models.py`	`Interval` (мероприятие) и `WellSeries` (скважина, `target()`, `training_mask()`)
`data/readers.py`	Чтение Excel, разбор номеров и дат с опечатками (`normalize_id`, `parse_date`), тестовый набор замеров
`data/statistics.py`	Чтение полугодовых файлов `.xls` (`HalfYearStatisticsReader`), склеенной статистики и файла разметки
`data/repair.py`	`FrequencyCheck` и `BlockShiftRepair`: проверка подъёма частоты; поиск и исправление сдвига на 180 дней (для склеенного файла)
`data/dataset.py`	`MergedWellRepository`: сборка скважин, проверка разметки по частоте, кэш, `DataAudit`
`features/builder.py`	`FeatureBuilder`: `sequence()` и `tabular()`
`models/base.py`	Интерфейс `EventSegmenter`: `fit`, `predict_proba`, `save`, `load`
`models/transformer.py`	Transformer-энкодер для временных рядов и его обёртка `TransformerSegmenter`
`models/classic.py`	Модели scikit-learn: `logreg`, `random_forest`, `extra_trees`, `gboost`
`models/__init__.py`	Реестр моделей `model_registry()` и `load_model()`
`evaluation/decoding.py`	`SegmentDecoder`: вероятности по дням в интервалы
`evaluation/metrics.py`	`EventMatcher` и `Score`: сопоставление и метрики
`evaluation/experiment.py`	`CrossValidator`: перекрёстная проверка по скважинам
`evaluation/report.py`	`ComparisonReport`: таблицы и графики сравнения моделей
`audit.py`	Аудит данных 2020–2026: что отброшено и почему, выгрузка обучающих данных
`cli.py`, `__main__.py`	Командная строка `python -m gtm_detector.ml_freq`
`tests/`	Тесты на синтетических скважинах

Главные команды

Запускать из корня проекта, в виртуальном окружении `.venv`:

```
# сравнить модели перекрёстной проверкой по скважинам (gboost и extra_trees - около 2 минут)
.venv/Scripts/python -m gtm_detector.ml_freq compare --models gboost extra_trees
# все модели, около 15 минут на процессоре (Transformer на 424 мероприятиях — около часа)
.venv/Scripts/python -m gtm_detector.ml_freq compare

# обучить модель на всей разметке и сохранить веса
.venv/Scripts/python -m gtm_detector.ml_freq train --model transformer

# найти мероприятия обученной моделью
.venv/Scripts/python -m gtm_detector.ml_freq predict --model gboost

# аудит данных 2020-2026
.venv/Scripts/python -m gtm_detector.ml_freq audit
```

```
# тесты
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests -q
```

Отчёты команд попадают в `results/ml_freq/`.

Как читать этот документ

Документ — «построчный разбор»: для каждого модуля пакета сначала краткое пояснение, зачем он нужен, затем каждый класс, функция и метод с номерами строк, как в редакторе. У объекта сначала идёт пояснение (что делает, что принимает и возвращает, почему так) и пример с реальными данными проекта — номерами скважин, датами, значениями из `config.py` и `tests/conftest.py`; ниже сам код.

Лучше читать в порядке потока данных: `config.py` → `data/` → `features/` → `models/` → `evaluation/` → `cli.py`; тесты в конце показывают на маленьких синтетических скважинах, что каждая часть должна делать.

Как обновляется документ

- Код в PDF попадает из файлов проекта автоматически, пояснения хранятся в `docs/ml_freq_walkthrough/explanations/<путь модуля>.md`, этот раздел — в `explanations/_intro.md`.
- Каждое пояснение помечено отпечатком кода объекта. Если код изменился, а пояснение нет, `build.py --check` сообщит, что оно устарело.
- PDF собирает `build.py`: `.venv/Scripts/python -X utf8 docs/ml_freq_walkthrough/build.py` (около минуты, печать через Microsoft Edge без окна). Проверка пояснений: ключ `--check`, пометка проверенного — `--stamp`.
- Обновляет пояснения и PDF агент `docs-maintainer` (`.claude/agents/docs-maintainer.md`). Stop-хук `.claude/hooks/docs_sync.py` после каждого ответа Claude Code, в котором менялись файлы проекта, просит вызвать этого агента.

__init__.py

Корень пакета `gtm_detector.ml_freq`. Файл содержит только строку документации: ничего не импортирует и не запускает, поэтому `import gtm_detector.ml_freq` мгновенный (не тянет PyTorch и scikit-learn).

Пакет решает одну задачу: для каждого дня скважины оценить вероятность, что идёт эффект мероприятия «увеличение частоты F», а затем общим декодером получить даты начала и окончания. Цепочка такая: Excel → `data` (скважины) → `features` (признаки) → `models` (Transformer, scikit-learn) → `evaluation` (интервалы и метрики) → `cli`.

Пример

```
import gtm_detector.ml_freq as pkg
print(pkg.__doc__.splitlines()[0])   # ML-детекция мероприятий «увеличение частоты F».
```

Запуск самого детектора идёт не отсюда, а через `python -m gtm_detector.ml_freq compare` (см. `__main__.py`).

__main__.py

Точка входа `python -m gtm_detector.ml_freq <команда>`. Модуль делает единственное: вызывает `cli.main()` и передаёт её код возврата в `sys.exit`, чтобы оболочка видела успех (0) или ошибку.

Вся логика команд (`compare`, `train`, `predict`, `audit`) лежит в `cli.py`; здесь её нет, чтобы модуль можно было импортировать в тестах без побочных эффектов.

Пример

```
.venv/Scripts/python -m gtm_detector.ml_freq audit                              # аудит данных 2020–2026
.venv/Scripts/python -m gtm_detector.ml_freq compare --models gboost extra_trees # около 2 минут
```

Обе команды запускают `main()` из `cli.py`; результаты попадают в `results/ml_freq/`.

Код модуля вне классов и функций

```python
import sys

from .cli import main

sys.exit(main())
```

config.py

Все настройки проекта в одном месте: неизменяемые (`frozen=True`) dataclass-классы со значениями по умолчанию. Чисел, вписанных прямо в код, в пакете нет — любой порог ищите здесь или в константах классов `data/repair.py` и `data/dataset.py`.

Модуль задаёт три корневых пути: `PROJECT_ROOT` (папка проекта, два уровня выше `config.py`), `PACKAGE_DIR` (`gtm_detector/ml_freq`) и `RESOURCES` (`resources/`, исходные Excel — только чтение). Константа `CONTEXT_DAYS = 45` — сколько дней до начала и после окончания размеченного мероприятия используется для обучения (см. `WellSeries.training_mask`).

Пример

```
from gtm_detector.ml_freq.config import ExperimentConfig, CONTEXT_DAYS
cfg = ExperimentConfig()
cfg.folds, cfg.decoder.threshold, CONTEXT_DAYS   # (5, 0.5, 45)
cfg.folds = 3   # FrozenInstanceError: настройки менять нельзя, только создавать новые
```

Чтобы изменить значение, создают новый объект: `ExperimentConfig(folds=3)`.

Код модуля вне классов и функций

```
  3 from __future__ import annotations
  4
  5 from dataclasses import dataclass, field
  6 from pathlib import Path
  7
  8 PROJECT_ROOT = Path(__file__).resolve().parents[2]
  9 PACKAGE_DIR = Path(__file__).resolve().parent
 10 RESOURCES = PROJECT_ROOT / "resources"
 11
    …
 44 # Дней до начала и после окончания размеченного мероприятия, на которых учатся модели.
 45 # Остальные дни не используются: в разметке есть не все реальные подъёмы частоты.
 46 CONTEXT_DAYS = 45
```

класс-данные (dataclass) DataConfig

`class DataConfig`

```
 13 @dataclass(frozen=True)
 14 class DataConfig:
 15     """Где лежат данные и куда писать результаты.
 16
 17     ``dataset="merged"`` — статистика 2020–2026 и разметка технологов (по умолчанию),
 18     ``dataset="test"`` — первые тестовые файлы из ``resources/test_datas``, если они есть.
 19     """
 20
 21     dataset: str = "merged"
 22     # Папка <год>/<файл>.xls: исходные выгрузки по полугодиям. Можно указать и один
 23     # склеенный файл .xlsx: тогда сдвинутые блоки исправит BlockShiftRepair.
 24     statistics: Path = RESOURCES / "statistics"
 25     labels: Path = RESOURCES / "labeled_data.xlsx"
 26     # Эталонная выгрузка для проверки сдвигов склеенного файла; для полугодий не нужна.
 27     reference: Path | None = None
 28     measurements: Path = RESOURCES / "test_datas" / "input_dataset.xlsx"
 29     results: Path = RESOURCES / "test_datas" / "output_dataset.xlsm"
 30     output_dir: Path = PROJECT_ROOT / "results" / "ml_freq"
 31     weights_dir: Path = PACKAGE_DIR / "weights"
```

Где лежат входные файлы и куда писать результаты. Поля — пути `pathlib.Path`:

Поле	Значение по умолчанию
`dataset`	`"merged"` (статистика 2020–2026, по умолчанию) или `"test"` (первые тестовые файлы 2026 г.; папки `resources/test_datas` в проекте больше нет)
`statistics`	`resources/statistics` — папка `<год>/*.xls` с 14 полугодовыми выгрузками (20–26, 01.01.2020–02.11.2026). Можно указать и один склеенный `.xlsx`: тогда включается исправление сдвигов
`labels`	`resources/labeled_data.xlsx` — разметка технологов, лист «Лист1», 786 строк
`reference`	`None`: эталонная выгрузка нужна только для проверки сдвигов склеенного файла, у полугодовых файлов сдвига нет
`measurements`, `results`	`input_dataset.xlsx` и `output_dataset.xlsm` для режима `"test"`
`output_dir`	`results/ml_freq`
`weights_dir`	`gtm_detector/ml_freq/weights` (файлы `.pt`)

Пример

```
cfg = DataConfig()
cfg.statistics.is_dir()          # True: resources/statistics с папками 20 … 26
cfg.reference                    # None
DataConfig(dataset="test")       # режим первых тестовых файлов (если они есть)
```

класс-данные (dataclass) DecoderConfig

`class DecoderConfig`

```
34  @dataclass(frozen=True)
35  class DecoderConfig:
36      """Как превратить вероятности по дням в мероприятия."""
37
38      threshold: float = 0.5  # день «в эффекте», если сглаженная вероятность выше
39      smooth_days: int = 5  # окно скользящего среднего вероятностей
40      max_gap: int = 3  # разрывы короче — склеиваются в один период
41      min_length: int = 2  # более короткие периоды отбрасываются
```

Параметры `SegmentDecoder`, превращающего вероятности по дням в интервалы; одни и те же для всех моделей, иначе сравнение нечестное.

- `threshold = 0.5` — день «в эффекте», если сглаженная вероятность выше;
- `smooth_days = 5` — окно скользящего среднего вероятностей (дни);
- `max_gap = 3` — разрывы короче 3 дней склеиваются в один период;
- `min_length = 2` — периоды короче 2 дней отбрасываются.

Пример

Если эффект «мигнул» на 2 дня (вероятность упала ниже 0,5 между двумя высокими участками), при `max_gap=3` это один период, а не два. Порог 0,5 и окно 5 дней применяются ко всем моделям одинаково.

класс-данные (dataclass) TransformerConfig

`class TransformerConfig`

```
49  @dataclass(frozen=True)
50  class TransformerConfig:
51      """Архитектура Transformer-энкодера для временных рядов."""
52
53      # Небольшая модель (~20 тыс. параметров): на ~100 скважинах крупнее — хуже
```

```
    # (d_model=64, 3 слоя: точное начало 16% против 24% на перекрёстной проверке).
    d_model: int = 32
    n_heads: int = 4
    n_layers: int = 2
    dim_feedforward: int = 64
    dropout: float = 0.2
    conv_kernel: int = 5  # свёртка перед энкодером ловит локальные ступеньки
    max_length: int = 4096
```

Архитектура Transformer-энкодера: `d_model=32`, `n_heads=4`, `n_layers=2`, `dim_feedforward=64`, `dropout=0.2`, свёртка `conv_kernel=5` перед энкодером (ловит локальные ступеньки частоты), `max_length=4096` дней в позиционном кодировании. Модель маленькая (~20 тыс. параметров) намеренно: размеченных мероприятий всего 135, и большая модель переобучается.

Пример

Вариант `d_model=64`, 3 слоя на перекрёстной проверке дал точное начало 16 % против 24 % у `d_model=32` (комментарий в `config.py`). Поэтому значения по умолчанию такие.

класс-данные (dataclass) TrainingConfig

`class TrainingConfig`

```
@dataclass(frozen=True)
class TrainingConfig:
    """Обучение Transformer."""

    epochs: int = 80
    batch_size: int = 16
    learning_rate: float = 1e-3
    weight_decay: float = 1e-4
    crop_length: int = 128  # обучение на случайных окнах ряда — аугментация
    crops_per_well: int = 4
    seed: int = 0
```

Обучение Transformer: `epochs=80`, `batch_size=16`, `learning_rate=1e-3`, `weight_decay=1e-4`. Обучение идёт на случайных окнах ряда `crop_length=128` дней, по `crops_per_well=4` окна на скважину за эпоху — это аугментация при малом числе данных. `seed=0` передаётся в `torch.manual_seed` и генераторы NumPy (воспроизводимость).

Пример

Быстрая проверка из CLAUDE.md: `compare --models transformer --epochs 20` подменяет только `epochs=80` на 20, остальное остаётся. Для сравнения зёрен создают `TrainingConfig(seed=1)`, `seed=2` и т. д.

класс-данные (dataclass) ExperimentConfig

`class ExperimentConfig`

```
@dataclass(frozen=True)
class ExperimentConfig:
    """Перекрёстная проверка по скважинам."""

    folds: int = 5
    seed: int = 0
    start_tolerance: int = 7  # дней: допуск при сопоставлении с разметкой
    decoder: DecoderConfig = field(default_factory=DecoderConfig)
```

Настройки перекрёстной проверки по скважинам: `folds=5`, `seed=0` (перемешивание скважин по фолдам), `start_tolerance=7` (допуск ±7 дней при сопоставлении найденного мероприятия с разметкой в

`EventMatcher`) и вложенный `decoder` (`DecoderConfig`, создаётся через `default_factory`, чтобы объект не был общим изменяемым значением).

Пример

```
ExperimentConfig().decoder.smooth_days   # 5
ExperimentConfig(folds=3, seed=1)        # другое разбиение скважин на 3 фолда
```

data/__init__.py

Пакет `data` — первый шаг цепочки «Excel → скважины → признаки → модель → интервалы». Он ничего не вычисляет сам, а собирает публичный интерфейс из шести модулей и перечисляет его в `__all__`, поэтому остальной код пишет `from ..data import WellSeries`.

Что экспортируется	Из модуля	Назначение
`SERIES_COLUMNS`, `Interval`, `WellSeries`	`models.py`	модель скважины и мероприятия
`MeasurementsReader`, `LabelsReader`, `WellRepository`, `normalize_id`, `parse_date`	`readers.py`	чтение тестовых файлов 2026 г.
`HalfYearStatisticsReader`, `MergedStatisticsReader`, `MonitoringLabelsReader`, `LabelRecord`	`statistics.py`	чтение полугодовых файлов `.xls`, прежнего склеенного файла и разметки 2020–2026
`BlockShiftRepair`, `FrequencyCheck`, `Segment`	`repair.py`	проверка подъёма частоты; исправление блоков, сдвинутых на 180 дней (только для склеенного файла)
`MergedWellRepository`, `DataAudit`	`dataset.py`	итоговая загрузка и отчёт об отброшенном

Пример

```python
from gtm_detector.ml_freq.data import MergedWellRepository
from gtm_detector.ml_freq.config import DataConfig
c = DataConfig()
wells, audit = MergedWellRepository(c.statistics, c.labels, c.reference).build()   # reference = None
len(wells)   # 611 скважин, из них 369 с разметкой (424 мероприятия)
```

Код модуля вне классов и функций

```python
from .dataset import DataAudit, MergedWellRepository
from .models import SERIES_COLUMNS, Interval, WellSeries
from .readers import LabelsReader, MeasurementsReader, WellRepository, normalize_id, parse_date
from .repair import BlockShiftRepair, FrequencyCheck, Segment
from .statistics import HalfYearStatisticsReader, LabelRecord, MergedStatisticsReader,
MonitoringLabelsReader

__all__ = [
    "SERIES_COLUMNS",
    "BlockShiftRepair",
    "DataAudit",
    "FrequencyCheck",
    "HalfYearStatisticsReader",
    "Interval",
    "LabelRecord",
    "LabelsReader",
    "MeasurementsReader",
    "MergedStatisticsReader",
    "MergedWellRepository",
    "MonitoringLabelsReader",
    "Segment",
    "WellRepository",
    "WellSeries",
    "normalize_id",
    "parse_date",
]
```

data/models.py

Две структуры данных, на которых держится весь пакет: `Interval` (одно мероприятие) и `WellSeries` (скважина с суточными рядами и списком мероприятий). Читатели Excel (`readers.py`, `dataset.py`) их создают, `features.FeatureBuilder` превращает `WellSeries` в признаки, `evaluation` сравнивает предсказанные и размеченные `Interval`.

`SERIES_COLUMNS` — порядок шести измеряемых рядов: `frequency` (Гц), `debit_liquid` (м³/сут), `debit_oil`, `watercut` (обводнённость), `pressure_zab` (забойное давление), `pressure_line` (линейное давление). Ряды берутся из листов Excel «Fэцн», «Qж», «Qн», «Обв», «Рзаб», «Рлин».

Пример

```python
from gtm_detector.ml_freq.data import SERIES_COLUMNS
SERIES_COLUMNS[0]   # 'frequency'
```

Код модуля вне классов и функций

```python
from __future__ import annotations

from dataclasses import dataclass, field
from datetime import date

import numpy as np
import pandas as pd

SERIES_COLUMNS = (
    "frequency",
    "debit_liquid",
    "debit_oil",
    "watercut",
    "pressure_zab",
    "pressure_line",
)
```

класс-данные (dataclass) Interval

`class Interval`

```python
@dataclass(frozen=True)
class Interval:
    """Мероприятие: период эффекта. ``end=None`` — эффект продолжается до конца данных."""

    start: date
    end: date | None
    date_done: date | None = None  # «Дата выполнения» (только в разметке)
    # Мониторинг остановлен, а эффект продолжался («ПРОДОЛЖАЕТСЯ ЭФФЕКТ»):
    # ``end`` — не конец эффекта, а последний день наблюдения.
    censored: bool = False

```

Одно мероприятие: период эффекта от `start` до `end` включительно (даты, не время). Неизменяемый объект.

- `end=None` — эффект длится до конца данных;
- `date_done` — «Дата выполнения» из листа «Результат__*» (только в разметке 2026 г.);
- `censored=True` — технолог написал «ПРОДОЛЖАЕТСЯ ЭФФЕКТ»: мониторинг остановили, эффект не закончился, поэтому `end` — не конец эффекта, а последний день наблюдения. После такого `end` нельзя считать «эффекта нет» (см. `training_mask`).

Пример

Скважина 10335, куст 988, строка 307 итогового файла, и скважина 8814, куст 365:

```
Interval(date(2023, 8, 14), date(2023, 9, 20))                # причина: «прекращение эффекта по воде»
Interval(date(2021, 7, 11), date(2021, 8, 12), censored=True)  # «ПРОДОЛЖАЕТСЯ ЭФФЕКТ»
```

Во втором случае технолог просто перестал наблюдать 12.08.2021; эффект мог идти и дальше.

метод Interval.covers

```
def covers(self, day: date, last_day: date) -> bool
```

```
    def covers(self, day: date, last_day: date) -> bool:
        return self.start <= day <= (self.end or last_day)
```

Попадает ли день `day` в период эффекта. Принимает `day` и `last_day` (последний день данных скважины), возвращает `bool`. Если `end=None`, правой границей служит `last_day`.

Пример

```
iv = Interval(date(2023, 8, 14), None)
iv.covers(date(2024, 1, 1), last_day=date(2024, 5, 1))   # True: эффект «до конца данных»
iv.covers(date(2023, 8, 13), last_day=date(2024, 5, 1))  # False: раньше начала
```

класс-данные (dataclass) WellSeries

```
class WellSeries
```

```
@dataclass
class WellSeries:
    """Скважина: суточные ряды на непрерывном календаре + разметка."""

    number: str
    cluster: str
    frame: pd.DataFrame  # индекс — даты, колонки — SERIES_COLUMNS и state
    labels: list[Interval] = field(default_factory=list)

...

...

...

...
```

Скважина: `number` (номер, например `'10335'`), `cluster` (куст, `'988'`), `frame` — таблица `pandas`, индекс — подряд идущие календарные даты, столбцы — `SERIES_COLUMNS` и `state` (1 — скважина работает), `labels` — список `Interval`. Обычный (не frozen) dataclass: читатели дописывают разметку в `labels` после создания.

Календарь непрерывный, пропущенные дни остаются `NaN` — признаки сами учитывают пропуски, а не выбрасывают строки.

Пример

Скважина `10335/988` из статистики 2020–2026: `frame` с 01.01.2020 по 18.03.2026 (2269 дней), 7 столбцов, `labels` из одного `Interval` (14.08.2023 – 20.09.2023). Всего `MergedWellRepository.build()` даёт 611 скважин, из них 369 с разметкой.

свойство WellSeries.well_id

```
def well_id(self) -> str
```

```
45      @property
46      def well_id(self) -> str:
47          return f"{self.number}/{self.cluster}"
```

Строка `"номер/куст"` — удобный идентификатор для логов и отчётов, где одного номера мало (на разных кустах номера могут повторяться).

Пример

`WellSeries(number='10335', cluster='988', ...).well_id == '10335/988'`.

свойство WellSeries.dates

```
def dates(self) -> list[date]
```

```
49      @property
50      def dates(self) -> list[date]:
51          return [day.date() for day in self.frame.index]
```

Список обычных `datetime.date` по индексу `frame` (без времени), в том же порядке, что и строки таблицы. Нужен там, где даты сравнивают с `Interval.start/end`, которые тоже `date`.

Пример

Для `10335/988`: `dates[0] == date(2023, 6, 3)`, `dates[-1] == date(2024, 5, 1)`.

метод WellSeries.__len__

```
def __len__(self) -> int
```

```
53      def __len__(self) -> int:
54          return len(self.frame)
```

Число дней в `frame`. Позволяет писать `len(well)`.

Пример

`len(well) == 334` для `10335/988` (с 03.06.2023 по 01.05.2024 включительно).

метод WellSeries.target

```
def target(self) -> np.ndarray
```

```
56      def target(self) -> np.ndarray:
57          """1 — день внутри размеченного периода эффекта, 0 — вне."""
58          days = self.frame.index
59          target = np.zeros(len(days), dtype=np.float32)
60          for label in self.labels:
61              end = pd.Timestamp(label.end) if label.end else days[-1]
62              target[(days >= pd.Timestamp(label.start)) & (days <= end)] = 1.0
63          return target
```

Целевой вектор для обучения: массив `float32` длиной `len(self)`, 1.0 в днях внутри любого размеченного периода, 0.0 вне. Если у мероприятия `end=None`, период идёт до последнего дня `frame`.

Важно: 0 здесь означает «в разметке эффекта нет», а не «эффекта точно нет» — разметка неполная. Поэтому `target` используют только вместе с `training_mask`.

Пример

Для `10335/988` период 14.08–20.09.2023 даёт `target().sum() == 38.0`; для `8814/365` (11.07–12.08.2021) — 33.0.

метод WellSeries.training_mask

```
def training_mask(self, context_days: int | None) -> np.ndarray
```

```
65      def training_mask(self, context_days: int | None) -> np.ndarray:
66          """Дни, на которых можно учиться.
67
68          В разметке есть не все реальные подъёмы частоты, поэтому «вне разметки» не
69          значит «эффекта нет». Надёжны только окрестности размеченных мероприятий:
70          ``context_days`` дней до начала и после окончания. ``None`` — все дни.
71          После окончания ``censored``-мероприятия эффект неизвестен — эти дни не берутся.
72          """
73          days = self.frame.index
74          if context_days is None:
75              return np.ones(len(days), dtype=bool)
76          mask = np.zeros(len(days), dtype=bool)
77          margin = pd.Timedelta(days=context_days)
78          for label in self.labels:
79              end = pd.Timestamp(label.end) if label.end else days[-1]
80              after = end if label.censored else end + margin
81              mask[(days >= pd.Timestamp(label.start) - margin) & (days <= after)] = True
82          return mask
```

Булев массив длиной `len(self)`: дни, на которых модель вправе учиться (попадают в функцию потерь). Параметр `context_days` — окно в днях (в проекте `CONTEXT_DAYS = 45`); `None` — все дни.

Для каждого мероприятия берутся дни от `start − context_days` до `end + context_days`. Причина: технологи размечают не все подъёмы частоты, поэтому далёкие от разметки дни нельзя считать отрицательными примерами. Исключение — `censored`: после `end` эффект неизвестен, поэтому окно заканчивается ровно на `end`, а дни после не берутся.

Пример

```
well_10335.training_mask(45).sum()   # 128: с 30.06.2023 по 04.11.2023
well_8814.training_mask(45).sum()    # 78: censored, правая граница = 12.08.2021 без запаса
well_10335.training_mask(None).sum() # 334, то есть вся скважина
```

У 10335: 45 дней до + 38 дней эффекта + 45 дней после = 128. У 8814: 45 до + 33 дня эффекта = 78, потому что после `censored`-окончания дни не берутся.

data/readers.py

Читатели исходных файлов **первого тестового** набора 2026 г. (`resources/test_datas/`, этой папки в проекте больше нет — читатели оставлены для режима `--dataset test` и тестов): замеры `input_dataset.xlsx` (лист на параметр, строка — скважина, столбец — дата `ДД.ММ.ГГГГ`) и разметка `output_dataset.xlsm` (лист «Результат_*»). Основной набор 2020–2026 читает `statistics.py`, но он берёт отсюда `normalize_id`, `parse_date`, а `dataset.py` — `MeasurementsReader.read_parameter` (для эталонной выгрузки склеенного файла, по которой проверяются сдвиги).

`WellRepository` собирает замеры и разметку в список `WellSeries` и кэширует результат в `%TEMP%/gtm_ml_freq_cache`.

Пример

```
from gtm_detector.ml_freq.data import WellRepository
from gtm_detector.ml_freq.config import DataConfig
c = DataConfig()
wells = WellRepository(c.measurements, c.results).load()   # 516 скважин; первый запуск ~30 с
```

Код модуля вне классов и функций

```
 8  from __future__ import annotations
 9
10  import hashlib
11  import logging
12  import pickle
13  import re
14  import tempfile
15  from collections.abc import Iterable
16  from datetime import date
17  from pathlib import Path
18  from typing import Any
19
20  import pandas as pd
21
22  from .models import Interval, WellSeries
23
24  logger = logging.getLogger(__name__)
25
26  _LATIN_TO_CYRILLIC = str.maketrans("ABEKMHOPCTXabekmhopctx", "АВЕКМНОРСТХавекмнорстх")
27  _DATE_IN_TEXT = re.compile(r"(\d{1,2})\.(\d{1,2})\.(\d{2,4})")
28  _DATE_HEADER = re.compile(r"\d{2}\.\d{2}\.\d{4}")
```

функция normalize_id

```
def normalize_id(value: Any) -> str
```

```
31  def normalize_id(value: Any) -> str:
32      """Номер скважины/куста к единому виду: ``'561E '`` -> ``'561E'``, ``7782.0`` -> ``'7782'``."""
33      if value is None or (isinstance(value, float) and pd.isna(value)):
34          return ""
35      if isinstance(value, float) and value.is_integer():
36          value = int(value)
37      return str(value).strip().upper().translate(_LATIN_TO_CYRILLIC)
```

Приводит номер скважины или куста к одному виду. Принимает что угодно из ячейки Excel (строка, число, `NaN`, `None`), возвращает строку: обрезает пробелы, переводит в верхний регистр и заменяет **латинские буквы, похожие на русские**, на кириллицу (A→А, E→Е, K→К, M→М, H→Н, O→О, P→Р, C→С, T→Т, X→Х). Целые числа в `float` (`7782.0`) превращает в `'7782'`; пустое значение даёт `""`.

Нужно потому, что номера набирают вперемешку латиницей и кириллицей, а скважина — ключ для объединения замеров и разметки.

Пример

```
normalize_id('561E ')    # '561E'  (латинская E стала русской Е)
normalize_id(7782.0)     # '7782'
normalize_id(None)       # ''
normalize_id('2015Б')    # '2015Б'
```

функция parse_date

```
def parse_date(value: Any) -> date | None
```

```
40  def parse_date(value: Any) -> date | None:
41      """Дата из ячейки, в том числе с опечатками: ``'21.04.2026г'``, ``'08.02.206'``."""
42      if value is None or (isinstance(value, float) and pd.isna(value)):
43          return None
44      if isinstance(value, (pd.Timestamp, date)):
45          timestamp = pd.Timestamp(value)
46      else:
47          found = _DATE_IN_TEXT.search(str(value))
48          if not found:
49              return None
50          day, month, year = map(int, found.groups())
51          year = {206: 2026, 26: 2026}.get(year, year)
52          try:
53              timestamp = pd.Timestamp(year=year, month=month, day=day)
54          except ValueError:
55              return None
56      if pd.isna(timestamp) or timestamp.year < 2000:
57          return None
58      return timestamp.date()
```

Достаёт дату из ячейки разметки, терпя опечатки технологов. Принимает `Timestamp` / `date` /текст/ `NaN`, возвращает `date` или `None`. В тексте ищется шаблон `д.м.гг(гг)`, поэтому `'21.04.2026г'` работает. Год `206` (потерянная цифра) исправляется на 2026, `26` — тоже на 2026. Даты раньше 2000 года и несуществующие (31.02) дают `None`, чтобы мусор из Excel не попадал в разметку.

Пример

```
parse_date('21.04.2026г')              # date(2026, 4, 21)
parse_date('08.02.206')                # date(2026, 2, 8)
parse_date('abc')                      # None
parse_date(pd.Timestamp('2023-08-14'))  # date(2023, 8, 14)
```

класс MeasurementsReader

```
class MeasurementsReader
```

```
61  class MeasurementsReader:
62      """Файл замеров -> список :class:`WellSeries` (без разметки)."""
63
64      SHEETS = {
65          "debit_liquid": "Qж",
66          "watercut": "Обв",
67          "debit_oil": "Qн",
68          "pressure_line": "Рлин",
69          "frequency": "Fэцн ТМ",
70          "pressure_zab": "Рзаб",
71      }
72      STATE_SHEET = "Прим"
73      WORKING_STATES = frozenset({"раб.", "нак"})
```

```
74
    ...
77
    ...
96
    ...
111
```

Читает файл замеров (`input_dataset.xlsx`) в список `WellSeries` **без разметки**. Константа `SHEETS` связывает параметр и лист Excel: `frequency` — «Fэцн ТМ», `debit_liquid` — «Qж», `debit_oil` — «Qн», `watercut` — «Обв», `pressure_zab` — «Рзаб», `pressure_line` — «Рлин». `STATE_SHEET = "Прим"` — лист с состоянием скважины, `WORKING_STATES` — значения «работает» (`раб.`, `нак`).

Пример

В тестовом файле 516 скважин с 01.01.2026 по 31.08.2026; у первой (`124/2079`) насос работал примерно в 98 % дней.

метод MeasurementsReader.__init__

```
def __init__(self, path: Path) -> None
```

```
75      def __init__(self, path: Path) -> None:
76          self.path = Path(path)
```

Запоминает путь к файлу (`Path`). Файл не открывается до вызова `read`.

Пример

`MeasurementsReader(DataConfig().measurements)`.

метод MeasurementsReader.read

```
def read(self) -> list[WellSeries]
```

```
78      def read(self) -> list[WellSeries]:
79          tables = {name: self.read_parameter(sheet) for name, sheet in self.SHEETS.items()}
80          states = self._read_states()
81          frequency = tables["frequency"]
82          calendar = pd.date_range(frequency.columns.min(), frequency.columns.max(), freq="D")
83
84          wells = []
85          for key in frequency.index:
86              frame = pd.DataFrame(index=calendar)
87              for name, table in tables.items():
88                  frame[name] = table.loc[key].reindex(calendar) if key in table.index else float("nan")
89              if key in states.index:
90                  frame["state"] =
    states.loc[key].reindex(calendar).isin(self.WORKING_STATES).to_numpy()
91              else:
92                  frame["state"] = (frame["frequency"] > 0).to_numpy()
93              wells.append(WellSeries(number=key[0], cluster=key[1], frame=frame.astype(float)))
94          logger.info("Замеры: %d скважин, %s — %s", len(wells), calendar[0].date(),
    calendar[-1].date())
95          return wells
```

Основной метод: читает все шесть листов параметров и лист «Прим», строит единый **непрерывный календарь** от минимальной до максимальной даты листа частоты и для каждой скважины из листа частоты создаёт `WellSeries`. Если параметра у скважины нет — столбец из `NaN`. Столбец `state`: из листа «Прим» (`раб.`/`нак` → 1), а если скважины там нет — «частота > 0». Возвращает `list[WellSeries]`, все столбцы приведены к `float`.

Скважины берутся по листу частоты: без телеметрии частоты задача бессмысленна.

Пример

Первая скважина тестового файла — `124/2079`, календарь 01.01.2026 – 31.08.2026, `frame['state'].mean()` ≈ 0,98.

метод MeasurementsReader.read_parameter

```
def read_parameter(self, sheet: str) -> pd.DataFrame
```

```
 97      def read_parameter(self, sheet: str) -> pd.DataFrame:
 98          table = pd.read_excel(self.path, sheet_name=sheet, header=1).iloc[1:]
 99          dates = {}
100          for column in table.columns:
101              if isinstance(column, (pd.Timestamp, date)):
102                  dates[column] = pd.Timestamp(column)
103              elif isinstance(column, str) and _DATE_HEADER.fullmatch(column.strip()):
104                  dates[column] = pd.to_datetime(column.strip(), format="%d.%m.%Y")
105          values = table[list(dates)].apply(pd.to_numeric, errors="coerce")
106          values.columns = pd.DatetimeIndex(list(dates.values()))
107          values.index = pd.MultiIndex.from_arrays(
108              [table["№ скв."].map(normalize_id), table["№ куста"].map(normalize_id)]
109          )
110          return values[~values.index.duplicated()]
```

Читает **один** лист параметра и возвращает таблицу «скважина × дата»: индекс `MultiIndex(номер, куст)` после `normalize_id`, столбцы — `Timestamp`. Шапка листа — вторая строка (`header=1`), строка-подзаголовок под ней (`iloc[1:]`) отбрасывается. Столбцами считаются и настоящие даты, и строки `ДД.ММ.ГГГГ`; значения приводятся `to_numeric(errors='coerce')`, повторы скважин удаляются. Метод публичный, потому что `MergedWellRepository._reference` читает им лист «Qж» исходной выгрузки.

Пример

```
MeasurementsReader(c.reference).read_parameter("Qж")   # таблица «скважины × даты 2026 г.»
```

метод MeasurementsReader._read_states

```
def _read_states(self) -> pd.DataFrame
```

```
112      def _read_states(self) -> pd.DataFrame:
113          sheet = pd.read_excel(self.path, sheet_name=self.STATE_SHEET, header=None)
114          is_state = sheet[4] == "Состояние"
115          info = sheet.shift(-1)[is_state][1].astype(str)  # строка «Прим»: «..., куст 2079, ...»
116          clusters = info.str.extract(r"куст\s+([^,]+)", expand=False).map(normalize_id)
117          values = sheet[is_state].iloc[:, 7:].apply(lambda column: column.astype(str).str.strip())
118          values.columns = pd.to_datetime(sheet.iloc[1, 7:].values)
119          values.index = pd.MultiIndex.from_arrays([sheet[is_state][3].map(normalize_id), clusters])
120          return values[~values.index.duplicated()]
```

Читает лист «Прим» (без шапки) и возвращает таблицу состояний «скважина × дата» из **строк**, где в пятом столбце написано «Состояние». Отдельного столбца куста здесь нет: куст вынут регулярным выражением `куст\s+([^,]+)` из строки «Прим» под строкой состояния («..., куст 2079, ...»). Значения очищаются от пробелов (`'раб.'`, `'нак'` и т. п.); повторы скважин отбрасываются.

Пример

Для `124/2079` строка состояния содержит «раб.» почти во всех днях, куст `2079` берётся из соседней строки примечания.

класс LabelsReader

```
class LabelsReader
```

```
123 class LabelsReader:
124     """Лист «Результат__*»: мероприятия «Увеличение F», успешные, с обеими датами мониторинга."""
125 
126     # Номер столбца (с нуля) -> имя. Шапка листа — 4 строки.
127     COLUMNS = {3: "cluster", 4: "number", 26: "event", 35: "date_done", 39: "success", 41: "start",
    42: "end"}
128 
    ...
132 
```

Читает разметку из листа «Результат__*» файла `output_dataset.xlsm`. `COLUMNS` — номера столбцов (с нуля) → имена: 3 — куст, 4 — номер скважины, 26 — тип мероприятия, 35 — дата выполнения, 39 — успешность, 41/42 — даты начала/окончания мониторинга. Шапка занимает 4 строки (`skiprows=4`).

Пример

В тестовой разметке 105 скважин и 117 мероприятий «Увеличение F» (успешных, с обеими датами).

метод LabelsReader.__init__

```
def __init__(self, path: Path, sheet: str | None = None) -> None
```

```
129     def __init__(self, path: Path, sheet: str | None = None) -> None:
130         self.path = Path(path)
131         self.sheet = sheet
```

Принимает путь к файлу и, при желании, имя листа; без имени берётся первый лист, название которого начинается с «Результат».

Пример

`LabelsReader(DataConfig().results)` — лист находится сам.

метод LabelsReader.read

```
def read(self) -> dict[tuple[str, str], list[Interval]]
```

```
133     def read(self) -> dict[tuple[str, str], list[Interval]]:
134         sheet = self.sheet or next(
135             name for name in pd.ExcelFile(self.path).sheet_names if name.startswith("Результат")
136         )
137         table = pd.read_excel(self.path, sheet_name=sheet, header=None, skiprows=4)
138         table = table[list(self.COLUMNS)].rename(columns=self.COLUMNS)
139 
140         labels: dict[tuple[str, str], list[Interval]] = {}
141         for row in table.itertuples(index=False):
142             if str(row.event).strip().lower() != "увеличение f" or str(row.success).strip().lower() !=
    "да":
143                 continue
144             start, end = parse_date(row.start), parse_date(row.end)
145             if start is None or end is None or end < start:
146                 continue
147             key = (normalize_id(row.number), normalize_id(row.cluster))
148             labels.setdefault(key, []).append(Interval(start, end, parse_date(row.date_done)))
149         logger.info("Разметка: %d мероприятий", sum(map(len, labels.values())))
150         return labels
```

Возвращает словарь `{(номер, куст): [Interval, ...]}`. В разметку попадают только строки с типом «Увеличение F», успешные («да»), у которых обе даты распознаны `parse_date` и окончание не раньше начала. В `Interval` записывается и `date_done`. Число принятых мероприятий пишется в лог.

Пример

```
LabelsReader(c.results).read()   # 105 ключей (скважин), 117 мероприятий
```

класс WellRepository

`class WellRepository`

```
153 class WellRepository:
154     """Скважины с привязанной разметкой; результат чтения Excel кэшируется на диске."""
155 
    ...
160 
    ...
175 
    ...
193 
```

Фасад над читателями для **тестового** набора: замеры + разметка → готовые скважины. Кэш — файл `wells_<отпечаток>.pkl` в `%TEMP%/gtm_ml_freq_cache`.

Пример

`WellRepository(c.measurements, c.results).load()` вернёт 516 скважин; разметка привязана к тем, где ключ `(номер, куст)` совпал.

метод WellRepository.__init__

`def __init__(self, measurements: Path, results: Path | None, cache_dir: Path | None = None) -> None`

```
156     def __init__(self, measurements: Path, results: Path | None, cache_dir: Path | None = None) ->
    None:
157         self.measurements = Path(measurements)
158         self.results = Path(results) if results else None
159         self.cache_dir = cache_dir or Path(tempfile.gettempdir()) / "gtm_ml_freq_cache"
```

Принимает путь к замерам, путь к разметке (может быть `None` — тогда скважины без разметки, например для предсказания) и необязательную папку кэша. По умолчанию кэш лежит во временной папке системы.

Пример

`WellRepository(c.measurements, None)` — только замеры.

метод WellRepository.load

`def load(self) -> list[WellSeries]`

```
161     def load(self) -> list[WellSeries]:
162         cache = self.cache_dir / f"wells_{self._fingerprint()}.pkl"
163         if cache.exists():
164             logger.info("Данные из кэша: %s", cache)
165             with cache.open("rb") as file:
166                 return pickle.load(file)
167 
168         wells = MeasurementsReader(self.measurements).read()
169         if self.results:
170             self._attach(wells, LabelsReader(self.results).read())
```

```
171         self.cache_dir.mkdir(parents=True, exist_ok=True)
172         with cache.open("wb") as file:
173             pickle.dump(wells, file)
174         return wells
```

Возвращает `list[WellSeries]`. Сначала ищет файл кэша по отпечатку; если есть — читает `pickle`. Иначе читает Excel (около 30 секунд), привязывает разметку и записывает кэш. Поэтому первое чтение долгое, а дальнейшие — за секунды.

Пример

Первый вызов ~30 с, повторный — доли секунды. Ключ кэша зависит только от файлов данных, а не от кода: после правки `readers.py` кэш может быть устаревшим — папку кэша надо удалить.

статический метод WellRepository._attach

```
def _attach(wells: Iterable[WellSeries], labels: dict[tuple[str, str], list[Interval]]) -> None
```

```
176     @staticmethod
177     def _attach(wells: Iterable[WellSeries], labels: dict[tuple[str, str], list[Interval]]) -> None:
178         by_key = {(well.number, well.cluster): well for well in wells}
179         by_number: dict[str, list[WellSeries]] = {}
180         for well in by_key.values():
181             by_number.setdefault(well.number, []).append(well)
182         lost = 0
183         for (number, cluster), intervals in labels.items():
184             well = by_key.get((number, cluster))
185             if well is None and len(by_number.get(number, [])) == 1:
186                 well = by_number[number][0]  # куст записан иначе, но номер уникален
187             if well is None:
188                 lost += len(intervals)
189                 continue
190             well.labels.extend(intervals)
191         if lost:
192             logger.warning("Не найдено скважин для %d мероприятий разметки", lost)
```

Статический метод: дописывает интервалы из словаря разметки в `labels` скважин (на месте, ничего не возвращает). Ключ — `(номер, куст)`. Если пары нет, но скважина с таким номером **одна**, берётся она («куст записан иначе, но номер уникален»). Остальные мероприятия считаются потерянными, их число пишется предупреждением в лог.

Пример

Разметка `('124', '2079')`, а в замерах куст записан иначе: пара не найдена, но номер 124 один — разметка привяжется. Если скважин с таким номером две, мероприятие уйдёт в «не найдено».

метод WellRepository._fingerprint

```
def _fingerprint(self) -> str
```

```
194     def _fingerprint(self) -> str:
195         parts = []
196         for path in (self.measurements, self.results):
197             if path is not None:
198                 stat = path.stat()
199                 parts.append(f"{path.resolve()}:{stat.st_size}:{stat.st_mtime_ns}")
200         return hashlib.sha1("|".join(parts).encode()).hexdigest()[:12]
```

Короткий хэш (12 символов SHA-1) от абсолютных путей, размеров и времён изменения файлов замеров и разметки. Любая правка Excel меняет хэш, и кэш пересчитывается.

Пример

Имя вида `wells_<12 hex-символов>.pkl` в `%TEMP%/gtm_ml_freq_cache`.

data/statistics.py

Читатели статистики и разметки 2020–2026. Основной источник — **14 исходных полугодовых выгрузок** `resources/statistics/<год>/*.xls` (лист на параметр, строка — скважина, столбцы — даты полугодия): `HalfYearStatisticsReader`. Прежний источник — один склеенный `.xlsx` с теми же листами (`MergedStatisticsReader`, даты 01.01.2020–31.12.2026): его склеил макрос, и часть выгрузок в нём записана со сдвигом на 180 дней. Разметка технологов — `resources/labeled_data.xlsx` (лист «Лист1», строка — мероприятие, 786 строк): `MonitoringLabelsReader`.

Модуль данные **не исправляет**, а читает «как есть» (только числа): сдвиг склеенного файла чинит `repair.py` (и только для него — в полугодовых файлах сдвига нет). Результат работы модуля — таблицы «скважина × дата» и список `LabelRecord`, которые `dataset.py` превращает в `WellSeries`. `.xls` читается через `xlrd==2.0.2`, поток книги достаёт собственная `_workbook_stream`.

Пример

```
from gtm_detector.ml_freq.data import HalfYearStatisticsReader, MonitoringLabelsReader
tables = HalfYearStatisticsReader(c.statistics).read()   # 6 таблиц, частота: 616 скважин × 2498 дат
records = MonitoringLabelsReader(c.labels).read()        # 786 записей
```

В склеенном файле эти данные были сдвинуты: у скважины 10335/988 ступенька частоты 49,5 → 50,5 → 51,5 Гц стояла на 10.02.2024, а технолог ставил начало 14.08.2023 (ровно 180 дней разницы). В исходных полугодовых файлах подъём стоит на своём месте.

Код модуля вне классов и функций

```
12  from __future__ import annotations
13
14  import logging
15  import re
16  import struct
17  from dataclasses import dataclass
18  from datetime import date
19  from pathlib import Path
20
21  import numpy as np
22  import pandas as pd
23  import xlrd
24
25  from .readers import normalize_id, parse_date
26
27  logger = logging.getLogger(__name__)
28
29  _DATE_HEADER = re.compile(r"\d{2}\.\d{2}\.\d{4}")
```

класс MergedStatisticsReader

`class MergedStatisticsReader`

```
32  class MergedStatisticsReader:
33      """Файл статистики -> таблицы «скважина × дата» по параметрам.
34
35      В ячейку годится только число. Пустые столбцы в формате даты (2020 г., конец
36      2026 г.) pandas читает как NaT — это пропуски, а не числа. Заполненные ячейки
37      с текстом или датой тоже пропуски; их число по датам — в :attr:`non_numeric`.
38      """
39
40      # Имя параметра -> начало имени листа (листы: «Qж (из OIS)», «Fэцн ТМ(сут)», …).
41      SHEETS = {
42          "debit_liquid": "Qж",
```

```
43          "watercut": "Обв",
44          "debit_oil": "Qн",
45          "pressure_line": "Рлин",
46          "frequency": "Fэцн",
47          "pressure_zab": "Рзаб",
48      }
49      HEADER_ROW = 1  # строка с названиями столбцов; данные — со строки HEADER_ROW + 2
50      NUMBER, CLUSTER = "№ скв.", "№ куста"
51
    ...
55
    ...
66
```

Читает **склеенный** файл статистики (прежний источник; используется, если `statistics` указывает на файл `.xlsx`) в словарь `{параметр: DataFrame}`. Строки таблицы — `MultiIndex(номер, куст)`, столбцы — `Timestamp` с 01.01.2020 по 31.12.2026 (2557 дат), значения — `float`. В ячейке годится **только число**: текст, даты и `bool` становятся `NaN`. Пустые столбцы в формате даты (2020 г., конец 2026 г.) pandas читает как `NaT` — это тоже пропуски. Счётчик заполненных, но не числовых ячеек по датам хранится в `non_numeric`.

Пример

Листы файла: «Qж (из OIS)», «Обв (из OIS)», «Qн (из OIS)», «Рлин (из OIS)», «Fэцн ТМ(сут)», «Рзаб». Словарь `SHEETS` ищет лист по началу имени, поэтому суффиксы «(из OIS)» и «(сут)» не мешают.

метод MergedStatisticsReader.__init__

```
def __init__(self, path: Path) -> None
```

```
52      def __init__(self, path: Path) -> None:
53          self.path = Path(path)
54          self.non_numeric = pd.Series(dtype=int)
```

Запоминает путь и создаёт пустой `non_numeric` (`Series` из нулей до вызова `read`).

Пример

`MergedStatisticsReader(DataConfig().statistics)`.

метод MergedStatisticsReader.read

```
def read(self) -> dict[str, pd.DataFrame]
```

```
56      def read(self) -> dict[str, pd.DataFrame]:
57          sheets = pd.ExcelFile(self.path).sheet_names
58          tables, garbage = {}, []
59          for name, prefix in self.SHEETS.items():
60              sheet = next(s for s in sheets if normalize_id(s).startswith(normalize_id(prefix)))
61              tables[name], bad = self._read_sheet(sheet)
62              garbage.append(bad)
63          self.non_numeric = pd.concat(garbage, axis=1).sum(axis=1)
64          logger.info("Статистика: %d скважин, %d дат", len(tables["frequency"]),
    tables["frequency"].shape[1])
65          return tables
```

Читает шесть листов (`debit_liquid`, `watercut`, `debit_oil`, `pressure_line`, `frequency`, `pressure_zab`). Лист подбирается по префиксу после `normalize_id`: так латинская «F» в имени листа не помешает найти «Fэцн ТМ(сут)». Складывает вклад каждого листа в `non_numeric` (сумма по листам) и пишет в лог число скважин и дат. Возвращает словарь таблиц.

Таблицы разных параметров могут содержать разные наборы скважин: выравнивает их потом `dataset.py`.

Пример

```
tables = reader.read()
{k: v.shape for k, v in tables.items()}
# frequency: (766, 2557); debit_liquid, watercut, pressure_line, pressure_zab: (768, 2557); debit_oil: (767, 2557)
```

метод MergedStatisticsReader._read_sheet

```
def _read_sheet(self, sheet: str) -> tuple[pd.DataFrame, pd.Series]
```

```
67      def _read_sheet(self, sheet: str) -> tuple[pd.DataFrame, pd.Series]:
68          raw = pd.read_excel(self.path, sheet_name=sheet, header=None)
69          header = raw.iloc[self.HEADER_ROW]
70          body = raw.iloc[self.HEADER_ROW + 2 :]
71          columns = {i: _header_date(value) for i, value in header.items()}
72          columns = {i: day for i, day in columns.items() if day is not None}
73
74          values = pd.DataFrame({day: _numbers(body[i]) for i, day in columns.items()})
75          values.index = pd.MultiIndex.from_arrays(
76              [
77                  body[_position(header, self.NUMBER)].map(normalize_id).to_numpy(),
78                  body[_position(header, self.CLUSTER)].map(normalize_id).to_numpy(),
79              ]
80          )
81          filled = pd.DataFrame({day: body[i].notna().to_numpy() for i, day in columns.items()})
82          bad = (filled.to_numpy() & values.isna().to_numpy()).sum(axis=0)
83
84          keep = (values.index.get_level_values(0) != "") & ~values.index.duplicated()
85          return values[keep], pd.Series(bad, index=values.columns)
```

Читает один лист целиком (`header=None`). Строка `HEADER_ROW = 1` — названия столбцов, данные начинаются со строки 3 (`HEADER_ROW + 2`: одна строка под шапкой — подзаголовок). Столбцы с датой в заголовке (настоящей или строкой `ДД.ММ.ГГГГ`) превращаются в числовые колонки через `_numbers`; столбцы «№ скв.» и «№ куста» находит `_position` и нормализует `normalize_id`. Строки без номера скважины и повторы пропускаются.

Возвращает пару: таблицу значений и `Series` «сколько ячеек по датам было заполнено, но не числом» (нужна для отчёта аудита).

Пример

Если бы в листе «Fэцн ТМ(сут)» в дату 14.08.2023 стоял текст (например «ремонт»), он стал бы `NaN`, а в `non_numeric` за эту дату прибавилась бы 1. В текущем файле таких ячеек нет: `non_numeric.sum() == 0`.

класс HalfYearStatisticsReader

```
class HalfYearStatisticsReader
```

```
88  class HalfYearStatisticsReader:
89      """Папка с полугодовыми файлами ``.xls`` -> таблицы «скважина × дата» по параметрам.
90
91      Результат — как у :class:`MergedStatisticsReader`: столбцы — непрерывный календарь
92      от первой до последней даты файлов, скважины — объединение по всем файлам.
93      """
94
95      FILES = "*/*.xls"  # <год>/<файл>.xls
96
    ...
100
```

```
        ...
    103
```

Читает **основной** источник статистики: папку `resources/statistics` с 14 исходными полугодовыми выгрузками `<год>/stats_by_first_half(…).xls` и `stats_by_second_half(…).xls` (годы 20–26, с 01.01.2020 по 02.11.2026). Результат такой же, как у `MergedStatisticsReader`: словарь `{параметр: DataFrame}`, строки — `MultiIndex(номер, куст)`, столбцы — непрерывный календарь, значения — `float`; поэтому `dataset.py` не видит разницы между источниками. Скважины — объединение по всем файлам (в 2020 г. их было меньше, чем в 2026).

Выгрузки исходные, не склеенные макросом: **сдвига на 180 дней в них нет**. Проверка по разметке без порога телеметрии: подъём частоты подтверждается у 427 мероприятий при сдвиге 0 и лишь у 79 при +180. Поэтому `BlockShiftRepair` для этого источника не запускается.

Пример

```python
from gtm_detector.ml_freq.config import DataConfig
from gtm_detector.ml_freq.data import HalfYearStatisticsReader
reader = HalfYearStatisticsReader(DataConfig().statistics)   # resources/statistics
len(reader.files())                                          # 14
tables = reader.read()
tables["frequency"].shape                                    # (616, 2498): 01.01.2020 – 02.11.2026
```

Первое чтение 14 файлов занимает около трёх минут (результат потом лежит в кэше `dataset.py`).

метод HalfYearStatisticsReader.__init__

```
def __init__(self, directory: Path) -> None
```

```
    97      def __init__(self, directory: Path) -> None:
    98          self.directory = Path(directory)
    99          self.non_numeric = pd.Series(dtype=int)
```

Запоминает папку и создаёт пустой счётчик `non_numeric` (заполняется в `read`). Шаблон файлов `FILES = "*/*.xls"` — «подпапка года / файл».

Пример

`HalfYearStatisticsReader(Path("resources/statistics"))`.

метод HalfYearStatisticsReader.files

```
def files(self) -> list[Path]
```

```
    101     def files(self) -> list[Path]:
    102         return sorted(self.directory.glob(self.FILES))
```

Отсортированный список файлов `*/*.xls` в папке. Сортировка по пути даёт порядок «20, 21, …, 26», внутри года имена `first_half` раньше `second_half`. Этот же список использует `MergedWellRepository._fingerprint` для ключа кэша: замена любого файла сбрасывает кэш.

Пример

```python
reader.files()[0].name    # 'stats_by_first_half(01.01.2020-30.06.2020).xls'
reader.files()[-1].name   # 'stats_by_second_half(01.07.2026-02.11.2026).xls'
```

метод HalfYearStatisticsReader.read

```
def read(self) -> dict[str, pd.DataFrame]
```

```
    def read(self) -> dict[str, pd.DataFrame]:
        files = self.files()
        if not files:
            raise FileNotFoundError(f"Нет файлов статистики {self.FILES} в {self.directory}")
        sheets = MergedStatisticsReader.SHEETS
        parts: dict[str, list[pd.DataFrame]] = {name: [] for name in sheets}
        garbage = []
        for path in files:
            book = xlrd.open_workbook(file_contents=_workbook_stream(path), on_demand=True)
            for name, prefix in sheets.items():
                title = next(
                    s for s in book.sheet_names() if normalize_id(s).startswith(normalize_id(prefix))
                )
                table, bad = _sheet_table(book.sheet_by_name(title))
                parts[name].append(table)
                garbage.append(bad)
            book.release_resources()
        tables = {name: _join_periods(frames) for name, frames in parts.items()}
        self.non_numeric = pd.concat(garbage).groupby(level=0).sum()
        logger.info(
            "Статистика: %d файлов, %d скважин, %d дат",
            len(files),
            len(tables["frequency"]),
            tables["frequency"].shape[1],
        )
        return tables
```

Для каждого из 14 файлов: достаёт поток книги (`_workbook_stream`), открывает его в `xlrd`, по шести параметрам (`MergedStatisticsReader.SHEETS`) находит лист по префиксу имени (как у склеенного файла: «Гэцн ТМ(сут)» ищется по началу, латинская «F» не мешает) и переводит лист в таблицу (`_sheet_table`). Затем по каждому параметру склеивает таблицы периодов (`_join_periods`), суммирует нечисловые ячейки по датам в `non_numeric` и пишет в лог число файлов, скважин и дат. Если файлов нет, бросает `FileNotFoundError` — чтобы пустая папка не дала молча пустое обучение.

Пример

```
tables = reader.read()
{k: v.shape for k, v in tables.items()}   # все шесть: (616, 2498)
reader.non_numeric.sum()                  # 0: в полугодовых файлах нечисловых ячеек нет
```

функция _workbook_stream

```
def _workbook_stream(path: Path) -> bytes
```

```
def _workbook_stream(path: Path) -> bytes:
    """Поток ``Workbook`` из файла OLE2 (старый формат Excel), без разбора контейнера в xlrd.

    Выгрузки сохранены не самим Excel: цепочка секторов потока длиннее, чем размер в
    каталоге файла, и xlrd отказывается их открывать («size exceeds expected»). Лишнее —
    только заполнение последних секторов; саму книгу xlrd читает из потока как обычно.
    """
    data = Path(path).read_bytes()
    sector = 1 << struct.unpack_from("<H", data, 30)[0]

    def ints(sid: int) -> tuple[int, ...]:
        return struct.unpack_from(f"<{sector // 4}i", data, (sid + 1) * sector)

    fat_count, directory = struct.unpack_from("<ii", data, 44)
    extra = struct.unpack_from("<i", data, 68)[0]
    fat_sectors = list(struct.unpack_from("<109i", data, 76))
```

```
    while extra >= 0:  # больше 109 секторов таблицы: продолжение списка в отдельных секторах
        values = ints(extra)
        fat_sectors.extend(values[:-1])
        extra = values[-1]
    fat = [link for sid in fat_sectors[:fat_count] for link in ints(sid)]

    for sid in _chain(fat, directory):
        for offset in range(0, sector, 128):
            entry = data[(sid + 1) * sector + offset :][:128]
            length = struct.unpack_from("<H", entry, 64)[0]
            if entry[: max(length - 2, 0)].decode("utf-16-le") == "Workbook":
                start = struct.unpack_from("<i", entry, 116)[0]
                return b"".join(data[(s + 1) * sector :][:sector] for s in _chain(fat, start))
    raise ValueError(f"В файле {path} нет потока Workbook")
```

Достаёт из файла `.xls` (контейнер OLE2) поток `Workbook` — собственно книгу Excel — в виде `bytes`, **минуя разбор контейнера в `xlrd`**. Принимает путь, возвращает поток; если потока нет, `ValueError`.

Зачем. Прямой `xlrd.open_workbook(path)` на этих файлах падает с «size exceeds expected»: выгрузки сохранены не самим Excel, цепочка секторов потока длиннее размера, записанного в каталоге файла. Лишнее — только заполнение последних секторов, поэтому функция сама идёт по таблице размещения (FAT) до конца цепочки, не доверяя размеру, а `xlrd` получает готовый поток через `file_contents=`. Для больших файлов (больше 109 секторов таблицы) продолжение списка читается из отдельных секторов. Сверка с настоящим Excel: 504 случайные ячейки, расхождений 0.

Пример

```
from pathlib import Path
from gtm_detector.ml_freq.data.statistics import _workbook_stream
import xlrd
path = Path("resources/statistics/26/stats_by_first_half(01.01.2026-30.06.2026).xls")
book = xlrd.open_workbook(file_contents=_workbook_stream(path), on_demand=True)
book.sheet_names()[:2]   # листы параметров: «Qж (из OIS)», «Обв (из OIS)» …
```

Тест `test_workbook_stream_follows_chain_longer_than_declared` воспроизводит этот дефект на минимальном файле.

функция _chain

```
def _chain(fat: list[int], start: int) -> list[int]
```

```
def _chain(fat: list[int], start: int) -> list[int]:
    """Номера секторов потока: отрицательная ссылка — конец цепочки."""
    chain = []
    while start >= 0:
        chain.append(start)
        start = fat[start]
    return chain
```

Номера секторов потока: от стартового сектора идёт по ссылкам таблицы `fat` (`fat[сектор]` — следующий), пока ссылка не отрицательная (-2 — конец цепочки). Принимает таблицу и стартовый номер, возвращает список номеров по порядку.

Пример

Для таблицы `[-3, -2, 3, 4, -2]` цепочка от сектора 2 — `[2, 3, 4]`; от сектора 1 — `[1]`.

функция _sheet_table

```
def _sheet_table(sheet) -> tuple[pd.DataFrame, pd.Series]
```

```python
def _sheet_table(sheet) -> tuple[pd.DataFrame, pd.Series]:
    """Лист xlrd -> таблица «скважина × дата» и число не-чисел по датам."""
    header = pd.Series(sheet.row_values(MergedStatisticsReader.HEADER_ROW))
    number = _position(header, MergedStatisticsReader.NUMBER)
    cluster = _position(header, MergedStatisticsReader.CLUSTER)
    columns = {i: _header_date(value) for i, value in header.items()}
    columns = {i: day for i, day in columns.items() if day is not None}
    positions = list(columns)

    index, rows = [], []
    bad = np.zeros(len(positions), dtype=int)
    for r in range(MergedStatisticsReader.HEADER_ROW + 2, sheet.nrows):
        values, types = sheet.row_values(r), sheet.row_types(r)
        well = normalize_id(values[number])
        if not well:
            continue
        kinds = np.array([types[i] for i in positions])
        numeric = kinds == xlrd.XL_CELL_NUMBER
        bad += ~numeric & ~np.isin(kinds, (xlrd.XL_CELL_EMPTY, xlrd.XL_CELL_BLANK))
        row = np.full(len(positions), np.nan)
        row[numeric] = [values[i] for i, ok in zip(positions, numeric, strict=True) if ok]
        index.append((well, normalize_id(values[cluster])))
        rows.append(row)
    days = list(columns.values())
    table = pd.DataFrame(rows, index=pd.MultiIndex.from_tuples(index), columns=days, dtype=float)
    return table[~table.index.duplicated()], pd.Series(bad, index=days)
```

Лист `xlrd` -> таблица «скважина × дата» плюс `Series` «сколько ячеек по датам заполнено, но не числом». Логика та же, что у `MergedStatisticsReader._read_sheet`: заголовок в строке `HEADER_ROW = 1`, данные с третьей строки, колонки «№ скв.» и «№ куста» ищет `_position`, колонки с датой — `_header_date`. Но тип ячейки берётся у `xlrd` (`row_types`), а не угадывается: числом считается только `XL_CELL_NUMBER`; пустая и «пробельная» ячейка — пропуск; текст, дата, ошибка — пропуск, который считается в `bad`. Строки без номера скважины (итоги) пропускаются, повтор скважины игнорируется (верна первая строка).

Пример

Лист с датами 01.01.2021 и 02.01.2021: строка `["101", "5А", 48.0, "ремонт"]` даёт `48.0` и `NaN`, а в `bad` за 02.01.2021 прибавляется 1; пустая ячейка в `bad` не попадает. Это ровно сценарий `test_sheet_table_keeps_only_numbers`.

функция _join_periods

```
def _join_periods(frames: list[pd.DataFrame]) -> pd.DataFrame
```

```python
def _join_periods(frames: list[pd.DataFrame]) -> pd.DataFrame:
    """Таблицы периодов -> одна на непрерывном календаре; при наложении дат верен поздний файл."""
    table = pd.concat(frames, axis=1, sort=True)
    table = table.loc[:, ~table.columns.duplicated(keep="last")]
    return table.reindex(columns=pd.date_range(table.columns.min(), table.columns.max()))
```

Таблицы периодов -> одна таблица на непрерывном календаре. Склеивает столбцы (`concat(axis=1)`), скважины объединяются, нет скважины в файле — `NaN`. Если даты двух файлов налагаются, остаётся более поздний файл (`keep="last"`, файлы идут по порядку). Затем `reindex` на `date_range` от первой до последней даты: пропущенные дни становятся пустыми столбцами, а не исчезают — иначе окна «30 дней назад» в признаках считали бы пропуск как соседний день.

Пример

Файл 1: 29.06.2021 и 30.06.2021; файл 2: 02.07.2021 (скважина 202 только в нём). Итог — четыре столбца с 29.06 по 02.07, 01.07 пуст у всех, у скважины 202 значения только в последнем. На реальных данных 14

таблиц дают 2498 столбцов (01.01.2020 – 02.11.2026).

функция _header_date

```
def _header_date(value) -> pd.Timestamp | None
```

```
208 def _header_date(value) -> pd.Timestamp | None:
209     if isinstance(value, (pd.Timestamp, date)) and not pd.isna(value):
210         return pd.Timestamp(value)
211     if isinstance(value, str) and _DATE_HEADER.fullmatch(value.strip()):
212         return pd.to_datetime(value.strip(), format="%d.%m.%Y")
213     return None
```

Определяет, дата ли в ячейке заголовка. Принимает значение ячейки; возвращает `Timestamp` для настоящей даты или строки `ДД.ММ.ГГГГ`, иначе `None` (заголовок «№ скв.», «Итого» и т. п. в календарь не входит).

Пример

```
_header_date('14.08.2023')          # Timestamp('2023-08-14')
_header_date(pd.NaT)                # None
_header_date('№ скв.')              # None
```

функция _position

```
def _position(header: pd.Series, title: str) -> int
```

```
216 def _position(header: pd.Series, title: str) -> int:
217     return next(i for i, value in header.items() if str(value).strip() == title)
```

Номер столбца (с нуля) в строке заголовков, где написано ровно `title` (после `strip`). Нужен, чтобы не зависеть от того, что «№ скв.» стоит в таком-то по счёту столбце. Если столбца нет — `StopIteration`.

Пример

`_position(header, "№ скв.")` — индекс столбца с номерами скважин; `_position(header, "№ куста")` — с кустами.

функция _numbers

```
def _numbers(column: pd.Series) -> np.ndarray
```

```
220 def _numbers(column: pd.Series) -> np.ndarray:
221     """Только настоящие числа: даты, NaT, текст и bool -> NaN."""
222     if pd.api.types.is_bool_dtype(column) or pd.api.types.is_datetime64_any_dtype(column):
223         return np.full(len(column), np.nan)
224     if pd.api.types.is_numeric_dtype(column):
225         return column.to_numpy(dtype=float)
226     return column.map(_number).to_numpy(dtype=float)
```

Превращает столбец листа в массив `float` длиной в число скважин. Если pandas определил столбец как `bool` или дату — весь столбец `NaN` (пустой столбец в формате даты читается как `NaT`). Если тип числовой — берётся как есть. Смешанный столбец (число, текст, дата) обрабатывается по ячейкам через `_number`.

Пример

Столбец из `[48.0, 'ремонт', datetime(2023, 8, 14)]` даст `[48.0, nan, nan]`.

функция _number

```
def _number(value) -> float
```

```
229 def _number(value) -> float:
230     if isinstance(value, (bool, np.bool_)) or not isinstance(value, (int, float, np.integer,
    np.floating)):
231         return np.nan
232     return float(value)
```

Одна ячейка: настоящее число (`int`/`float`, в том числе NumPy) → `float`; `bool`, текст, дата → `NaN`. `bool` отдельно отсекается, потому что в Python `True` — это число 1.

Пример

```
_number(49.5)    # 49.5
_number("49,5")  # nan  (текст не пытаемся разбирать)
_number(True)    # nan
```

класс-данные (dataclass) LabelRecord

```
class LabelRecord
```

```
235 @dataclass(frozen=True)
236 class LabelRecord:
237     """Строка итогового файла технологов."""
238 
239     row: int  # номер строки в Excel (с шапкой)
240     number: str
241     cluster: str
242     kind: str
243     status: str
244     success: str
245     reason: str
246     start: date | None
247     end: date | None
248     current: float  # частота до мероприятия, Гц
249     new: float  # частота после мероприятия, Гц
250     year: int | None
251 
    ...
255 
```

Одна строка итогового файла технологов, ещё не проверенная замерами. Неизменяемый dataclass: `row` — номер строки в Excel (с шапкой, поэтому первая строка данных = 2), `number`, `cluster`, `kind` (тип ГТМ), `status`, `success` («да»/«нет»), `reason` (причина прекращения мониторинга), `start`/`end` (даты начала/окончания мониторинга), `current` и `new` — частота до и после мероприятия, **Гц**, `year`.

Пример

Строка 307: скважина 10335, куст 988, «Изменение частоты, Гц», `current=49.5`, `new=49.96`, `start=2023-08-14`, `end=2023-09-20`, причина «прекращение эффекта по воде».

свойство LabelRecord.key

```
def key(self) -> tuple[str, str]
```

```
252     @property
253     def key(self) -> tuple[str, str]:
254         return self.number, self.cluster
```

Пара `(number, cluster)` — ключ скважины для поиска в таблицах статистики.

Пример

`('10335', '988')`.

свойство LabelRecord.censored

```
def censored(self) -> bool
```

```
256     @property
257     def censored(self) -> bool:
258         """Мониторинг остановлен, а эффект продолжался — ``end`` не конец эффекта."""
259         return "продолжается" in self.reason.lower()
```

`True`, если в причине прекращения мониторинга есть слово «продолжается» (регистр не важен). Тогда мониторинг остановили, а эффект шёл, и `end` — не конец эффекта; в `Interval` это превращается в `censored=True`, дни после него не обучают модель.

Пример

Строка 6: скважина 8814, куст 365, причина «ПРОДОЛЖАЕТСЯ ЭФФЕКТ» → `censored == True`. Всего в файле 164 такие строки, среди принятых мероприятий — 81.

класс MonitoringLabelsReader

```
class MonitoringLabelsReader
```

```
262 class MonitoringLabelsReader:
263     """Итоговый файл технологов: мероприятия по изменению частоты.
264
265     Берётся только «Лист1» — сводный список. «Лист2» — частичная копия 2025 г.:
266     все его строки, которых нет в «Лист1» (кроме одной пустой), — тип «Изменение
267     ЦЭС/Частоты на увеличение», который технологи в сводный список не включили.
268     """
269
270     SHEET = "Лист1"
271     EVENT_TYPES = frozenset({"изменение частоты, гц", "увеличение f"})
272     COLUMNS = {
273         "Куст": "cluster",
274         "Скважина": "number",
275         "Тип ГТМ": "kind",
276         "Статус ГТМ": "status",
277         "Успешно/Неуспешно": "success",
278         "Причина прекращения мониторинга": "reason",
279         "дата начала мониторинга": "start",
280         "дата окончания мониторинга": "end",
281         "Текущее значение": "current",
282         "Новое значение": "new",
283         "Год": "year",
284     }
285
    ...
288
    ...
311
```

Читает «Лист1» итогового файла: сводный список всех мероприятий. «Лист2» намеренно не читается: это частичная копия 2025 г., а строки, которых нет в «Лист1», — тип «Изменение ЦЭС/Частоты на увеличение», который технологи в сводный список не включили. `COLUMNS` — отображение заголовков Excel («Куст», «Скважина», «Тип ГТМ», «Статус ГТМ», «Успешно/Неуспешно», «Причина прекращения мониторинга», «дата

начала мониторинга», «дата окончания мониторинга», «Текущее значение», «Новое значение», «Год») на поля `LabelRecord`. `EVENT_TYPES` — подходящие типы: «изменение частоты, гц» и «увеличение f».

Пример

786 строк; после `rejection` (на уровне одной строки) остаётся 782: 3 строки с «новое ≤ текущего» и 1 «не выполнено». Остальные отказы (нет скважины, нет замеров, нет подъёма) определяются позже в `dataset.py`.

метод MonitoringLabelsReader.__init__

```
def __init__(self, path: Path) -> None
```

```
286      def __init__(self, path: Path) -> None:
287          self.path = Path(path)
```

Запоминает путь к файлу.

Пример

`MonitoringLabelsReader(DataConfig().labels)`.

метод MonitoringLabelsReader.read

```
def read(self) -> list[LabelRecord]
```

```
289      def read(self) -> list[LabelRecord]:
290          table = pd.read_excel(self.path, sheet_name=self.SHEET)
291          table = table[list(self.COLUMNS)].rename(columns=self.COLUMNS)
292          records = [
293              LabelRecord(
294                  row=i + 2,
295                  number=normalize_id(row.number),
296                  cluster=normalize_id(row.cluster),
297                  kind=_text(row.kind),
298                  status=_text(row.status),
299                  success=_text(row.success),
300                  reason=_text(row.reason),
301                  start=parse_date(row.start),
302                  end=parse_date(row.end),
303                  current=pd.to_numeric(row.current, errors="coerce"),
304                  new=pd.to_numeric(row.new, errors="coerce"),
305                  year=int(row.year) if pd.notna(row.year) else None,
306              )
307              for i, row in enumerate(table.itertuples(index=False))
308          ]
309          logger.info("Итоговый файл: %d строк", len(records))
310          return records
```

Читает «Лист1», оставляет нужные столбцы, переименовывает и строит `list[LabelRecord]` в порядке строк файла. Даты разбирает `parse_date` (терпит «21.04.2026г»), номера — `normalize_id`, числа частоты — `to_numeric(errors='coerce')` (нечисло → `NaN`). Номер строки Excel = индекс + 2.

Пример

```
records = MonitoringLabelsReader(c.labels).read()
records[0]   # row=2, скважина 16300, куст 1587, 12.10.2020 – 13.07.2021, частота 98.5 → 102.5 Гц
len(records) # 786
```

метод класса MonitoringLabelsReader.rejection

```
def rejection(cls, record: LabelRecord) -> str | None
```

```
312     @classmethod
313     def rejection(cls, record: LabelRecord) -> str | None:
314         """Причина не брать строку, видная без данных замеров; ``None`` — строка годится."""
315         if record.kind.lower() not in cls.EVENT_TYPES:
316             return "другой тип ГТМ"
317         if record.success.lower() != "да":
318             return "неуспешное"
319         if not record.status.lower().startswith("выполнено"):
320             return "не выполнено"
321         if record.start is None or record.end is None:
322             return "нет даты начала или окончания"
323         if record.end < record.start:
324             return "окончание раньше начала"
325         if not (record.new > record.current):
326             return "частота не увеличена (новое ≤ текущего)"
327         return None
```

Классовый метод: причина не брать запись, **видная без замеров**, либо `None`, если запись годится. Порядок проверок:

1. тип ГТМ не из `EVENT_TYPES` → «другой тип ГТМ»;
2. `success` не «да» → «неуспешное»;
3. статус не начинается с «выполнено» → «не выполнено»;
4. нет даты начала или окончания → «нет даты начала или окончания»;
5. окончание раньше начала → «окончание раньше начала»;
6. `new ≤ current` (или `NaN`) → «частота не увеличена (новое ≤ текущего)».

Проверка `not (new > current)` специально записана так, чтобы `NaN` тоже отбрасывался.

Пример

```
MonitoringLabelsReader.rejection(records[0])   # None: запись годится
```

На всём файле: 782 `None`, 3 «частота не увеличена», 1 «не выполнено». Остальные причины итогового отчёта (на полугодовых данных: 313 частота не поднялась, 40 дубликатов, 5 нет телеметрии) добавляет `dataset.py`.

функция _text

```
def _text(value) -> str
```

```
330 def _text(value) -> str:
331     return "" if value is None or (isinstance(value, float) and np.isnan(value)) else
    str(value).strip()
```

Текст ячейки без пробелов по краям; `None` и `NaN` → пустая строка. Нужна, чтобы сравнения вроде `success.lower() != "да"` не падали на пустых ячейках.

Пример

```
_text("  да ")      # 'да'
_text(float("nan")) # ''
```

data/repair.py

Исправление сдвига в **склеенной** статистике (файла `Статистика_с_2021_2026_в_одном_файле.xlsx` в `resources/` больше нет; модуль работает, если передать такой файл в `--statistics`). Файл был собран макросом из отдельных выгрузок; часть блоков записана на **180 дней позже**, чем должна. В исходных полугодовых файлах `resources/statistics/*/*.xls` сдвига нет (427 подтверждённых подъёмов при сдвиге 0 против 79 при +180), поэтому для них `BlockShiftRepair` не запускается; `FrequencyCheck` из этого модуля используется всегда. Если данные не чинить, подъём частоты стоит в таблице не там, где его поставил технолог, и модель учится на неверных днях.

Модуль работает в три шага:

1. `BlockShiftRepair.segments` делит календарь на **сегменты** — периоды с одним набором скважин (след одной выгрузки);
2. `BlockShiftRepair.assess` для каждого сегмента проверяет сдвиги 0 и 180 дней двумя способами: по разметке технологов (`FrequencyCheck` — на дату начала частота должна подняться) и, когда есть, по исходной выгрузке 2026 г. (дословное совпадение значений);
3. `BlockShiftRepair.apply` переносит данные: сдвинутые сегменты возвращает на 180 дней назад, непонятные исключает.

Сегмент, у которого доказательства противоречат обоим сдвигам, исключается — лучше потерять данные, чем учить модель на чужих датах. Исправлять данные в обход этого модуля нельзя.

Пример

Итог на прежнем склеенном файле: 39 сегментов — 25 «без сдвига», 7 «сдвиг −180 дн», 7 «исключён». В исходном файле у скважины 10335/988 ступенька частоты 49,5 → 50,5 → 51,5 Гц стоит на 10.02.2024, после исправления — на 14.08.2023, как в разметке. Исключённые данные составляют около 17 % ячеек.

Код модуля вне классов и функций

```python
from __future__ import annotations

import logging
from collections.abc import Mapping, Sequence
from dataclasses import dataclass, field
from datetime import date

import numpy as np
import pandas as pd

logger = logging.getLogger(__name__)

SHIFT_DAYS = 180
```

класс FrequencyCheck

`class FrequencyCheck`

```python
class FrequencyCheck:
    """Подтверждают ли замеры подъём частоты с ``current`` до ``new`` Гц около дня ``i``.

    Подъём засчитывается, если медиана частоты за неделю после дня выросла
    относительно недели до него хотя бы на ``MIN_RISE`` и при этом

    * дошла до «Нового значения» (с допуском) — или
    * прирост не меньше половины запланированного (``new - current``): телеметрия
      бывает смещена относительно уставки технолога на несколько Гц, прирост — нет.
```

```
    Частоту часто поднимают ступенями за 1–2 недели, поэтому годится любое из двух.
    """

    WINDOW = 7  # дней до и после скачка
    TOLERANCE = 3  # дней: скачок может быть чуть раньше или позже даты технолога
    MIN_RISE = 0.3  # Гц
    LEVEL_TOLERANCE = 1.5  # Гц (или 3 % от «Нового значения», если больше)
    MIN_PLANNED_SHARE = 0.5
    MIN_WORKING = 10.0  # Гц: меньше — насос стоит или нет телеметрии
    MIN_VALUES = 3

...
```

Проверка «замеры подтверждают, что технолог поднял частоту с `current` до `new` Гц около дня `i`». Подъём засчитывается, если медиана частоты за неделю **после** дня выросла относительно недели **до** на `MIN_RISE` (0,3 Гц) и при этом:

- медиана дошла до «Нового значения» с допуском `max(LEVEL_TOLERANCE, 3 % от new)`, **или**
- прирост не меньше половины запланированного (`MIN_PLANNED_SHARE × (new − current)`).

Второе условие нужно, потому что телеметрия бывает смещена относительно уставки на несколько Гц, а прирост — нет; частоту часто поднимают ступенями за 1–2 недели. Константы: `WINDOW = 7` дней, `TOLERANCE = 3` дня (скачок может быть чуть раньше или позже даты технолога), `MIN_WORKING = 10` Гц (ниже — насос стоит), `MIN_VALUES = 3` замера с каждой стороны.

Пример

Разметка 10335/988: 49,5 → 49,96 Гц, начало 14.08.2023. В замерах 8–13.08 — 49,5; 14–16.08 — 50,5; 17–20.08 — 51,5 Гц. Медиана недели до 14.08 — 49,5, после — 51,5, прирост 2,0 Гц ≥ 0,5 × 0,46 → подтверждено.

метод FrequencyCheck.confirms

```
def confirms(self, frequency: np.ndarray, i: int, current: float, new: float) -> bool | None
```

```
    def confirms(self, frequency: np.ndarray, i: int, current: float, new: float) -> bool | None:
        """``True``/``False`` — подтверждён/нет; ``None`` — около дня ``i`` нет замеров."""
        planned = new - current if np.isfinite(new - current) else np.inf
        tolerance = max(self.LEVEL_TOLERANCE, 0.03 * new)
        tested = False
        for t in range(i - self.TOLERANCE, i + self.TOLERANCE + 1):
            before = self._working(frequency[max(t - self.WINDOW, 0) : max(t, 0)])
            after = self._working(frequency[max(t, 0) : max(t + self.WINDOW, 0)])
            if len(before) < self.MIN_VALUES or len(after) < self.MIN_VALUES:
                continue
            tested = True
            level = np.median(after)
            rise = level - np.median(before)
            if rise >= self.MIN_RISE and (
                abs(level - new) <= tolerance or rise >= self.MIN_PLANNED_SHARE * planned
            ):
                return True
        return False if tested else None
```

Проверяет окрестность дня `i` (индекс в массиве `frequency`, Гц, `NaN` там, где замера нет). Перебирает дни `t` от `i − 3` до `i + 3`: берёт рабочие значения (> 10 Гц) за 7 дней до и после `t`; если с любой стороны меньше 3 значений, `t` пропускается. Для оставшихся считает прирост медиан и применяет правило из описания класса.

Возвращает `True` (подтверждён подъём), `False` (замеры были, подъёма нет), `None` (около дня `i` вообще не было годных замеров, проверить нечем). Различие `False`/`None` важно: «нет данных» не считается опровержением, а «данные есть, подъёма нет» — считается.

Пример

```
check = FrequencyCheck()
check.confirms(freq_10335, i_14_08_2023, 49.5, 49.96)   # True: ступенька на 2 Гц
check.confirms(np.full(60, np.nan), 30, 48.0, 50.0)     # None: замеров нет
```

метод FrequencyCheck._working

`def _working(self, values: np.ndarray) -> np.ndarray`

```
    def _working(self, values: np.ndarray) -> np.ndarray:
        return values[values > self.MIN_WORKING]
```

Оставляет в срезе только значения выше `MIN_WORKING` (10 Гц). `NaN` сравнение `> 10` не проходит, поэтому пропуски и остановки насоса (0 Гц) отбрасываются одной строкой.

Пример

`_working(np.array([0.0, np.nan, 49.5, 50.5]))` → `[49.5, 50.5]`.

класс-данные (dataclass) Evidence

`class Evidence`

```
@dataclass
class Evidence:
    """Сколько объектов проверено (``tested``) и сколько подтвердило сдвиг (``confirmed``)."""

    confirmed: int = 0
    tested: int = 0

```

Счётчик доказательств: `tested` — сколько объектов (мероприятий разметки или скважин исходной выгрузки) удалось проверить, `confirmed` — сколько из них подтвердили данный сдвиг.

Пример

Сегмент 30.11.2022 – 02.05.2023, сдвиг 180: `Evidence(confirmed=27, tested=53)`.

свойство Evidence.share

`def share(self) -> float`

```
    @property
    def share(self) -> float:
        return self.confirmed / self.tested if self.tested else 0.0
```

Доля подтвердивших: `confirmed / tested`, а при `tested = 0` — 0.0 (не деление на ноль).

Пример

`Evidence(27, 53).share ≈ 0.51`; `Evidence().share == 0.0`.

класс-данные (dataclass) Segment

```
class Segment
```

```
 87 @dataclass
 88 class Segment:
 89     """Период с одним набором скважин — след одной выгрузки."""
 90
 91     first: int  # позиции в календаре, включительно
 92     last: int
 93     start: date
 94     end: date
 95     wells: int
 96     cells: int  # ячеек с данными
 97     labels: dict[int, Evidence] = field(default_factory=dict)  # сдвиг -> мероприятия разметки
 98     reference: dict[int, Evidence] = field(default_factory=dict)  # сдвиг -> скважины исходной
    выгрузки
 99     shift: int | None = 0  # None — сегмент исключён
100     verdict: str = ""
101     collisions: int = 0  # ячеек Qж, куда сдвиг попал на уже занятые даты (оставлены прежние)
102
    ...
106
```

Период с одним набором скважин. Поля: `first`/`last` — позиции столбцов календаря (включительно), `start`/`end` — те же границы датами, `wells` — сколько скважин имеют данные в сегменте, `cells` — заполненных ячеек. Дальше заполняет `assess`: `labels` и `reference` — словари «сдвиг → `Evidence`», `shift` (0, 180 или `None` — исключён), `verdict` — текст решения, `collisions` — ячейки Qж, где сдвинутые данные попали на уже занятые даты (остались прежние).

Пример

```
2022-11-30 — 2023-05-02: 84 скважины, 12327 ячеек, shift=180,
labels={0: 0/1, 180: 27/53}, verdict «разметка подтверждает: 27 из 53 мероприятий», collisions=241
```

Всего столкновений по всем сегментам: 907 ячеек.

свойство Segment.days

```
def days(self) -> int
```

```
103     @property
104     def days(self) -> int:
105         return self.last - self.first + 1
```

Длина сегмента в днях: `last - first + 1`.

Пример

Сегмент 30.09.2021 – 30.09.2021 даёт `days == 1` (это «стык выгрузок»), 04.03.2021 – 12.07.2021 даёт 131.

свойство Segment.decision

```
def decision(self) -> str
```

```
107     @property
108     def decision(self) -> str:
109         if self.shift is None:
110             return "исключён"
111         return f"сдвиг −{self.shift} дн" if self.shift else "без сдвига"
```

Короткая подпись решения для логов и отчёта: «исключён» (`shift is None`), «без сдвига» (0) или «сдвиг −180 дн».

Пример

Сегмент с 04.09.2026 по 27.12.2026: «сдвиг −180 дн»; сегмент 02.01.2021 – 03.03.2021: «исключён».

класс-данные (dataclass) LabelPoint

`class LabelPoint`

```
114 @dataclass(frozen=True)
115 class LabelPoint:
116     """Мероприятие разметки в координатах таблиц: строка скважины, день начала, частоты до/после."""
117 
118     row: int
119     day: int
120     current: float
121     new: float
```

Мероприятие разметки в координатах таблиц (неизменяемый объект): `row` — номер строки скважины, `day` — номер столбца календаря в день начала, `current`/`new` — частоты до/после, Гц. `FrequencyCheck` работает с массивами, поэтому датам и номерам скважин место заменяют индексы.

Пример

10335/988: `LabelPoint(row=<строка 10335>, day=<позиция 14.08.2023 в календаре от 01.01.2020>, current=49.5, new=49.96)`.

класс BlockShiftRepair

`class BlockShiftRepair`

```
124 class BlockShiftRepair:
125     """Находит сегменты, выбирает для каждого сдвиг и применяет его."""
126 
    ...
148 
    ...
153 
    ...
170 
    ...
202 
    ...
224 
    ...
244 
    ...
263 
```

Главный класс модуля: находит сегменты, выбирает для каждого сдвиг и применяет его. Все пороги — параметры конструктора, чтобы их можно было менять в тестах и они попадали в ключ кэша.

Пример

```
repair = BlockShiftRepair()
segments = repair.segments(presence, dates)
repair.assess(segments, presence, frequency, points, reference, observed)
repaired = repair.apply(tables, segments)
```

метод BlockShiftRepair.__init__

```
def __init__( self, shifts: Sequence[int] = (0, SHIFT_DAYS), min_flip: int = 5, seam_days: int = 3,
min_confirmed: int = 3, min_share: float = 0.25, well_match: float = 0.9, reference_share: float = 0.5,
min_reference_wells: int = 3, check: FrequencyCheck | None = None, ) -> None
```

```python
    def __init__(
        self,
        shifts: Sequence[int] = (0, SHIFT_DAYS),
        min_flip: int = 5,
        seam_days: int = 3,
        min_confirmed: int = 3,
        min_share: float = 0.25,
        well_match: float = 0.9,
        reference_share: float = 0.5,
        min_reference_wells: int = 3,
        check: FrequencyCheck | None = None,
    ) -> None:
        self.shifts = tuple(shifts)
        self.min_flip = min_flip  # столько скважин должно появиться/пропасть, чтобы начался новый сегмент
        self.seam_days = seam_days  # сегменты короче — стыки выгрузок
        self.min_confirmed = min_confirmed  # по разметке: подтверждённых мероприятий не меньше
        self.min_share = min_share  # и их доля среди проверенных не меньше
        self.well_match = well_match  # скважина совпала с выгрузкой: равны не меньше этой доли ячеек
        self.reference_share = reference_share  # сегмент совпал: не меньше этой доли скважин
        self.min_reference_wells = min_reference_wells
        self.check = check or FrequencyCheck()
```

Параметры (значения по умолчанию в скобках):

Параметр	Смысл
`shifts` (0, 180)	какие сдвиги проверять, дни
`min_flip` (5)	столько скважин должно появиться или пропасть за день, чтобы начался новый сегмент
`seam_days` (3)	сегменты короче — стыки выгрузок
`min_confirmed` (3)	по разметке: подтвердивших мероприятий не меньше
`min_share` (0,25)	и их доля среди проверенных не меньше
`well_match` (0,9)	скважина совпала с выгрузкой: равны не меньше этой доли ячеек
`reference_share` (0,5)	сегмент совпал: не меньше этой доли скважин
`min_reference_wells` (3)	меньше скважин для сравнения — вывода не делаем
`check`	объект `FrequencyCheck`

Пример

`BlockShiftRepair(min_flip=3)` — более чувствительное деление на сегменты (в проекте используется значение 5).

метод BlockShiftRepair.__repr__

```
def __repr__(self) -> str: # входит в ключ кэша
```

```python
    def __repr__(self) -> str:  # входит в ключ кэша
        params = {k: v for k, v in vars(self).items() if k != "check"}
        check = {k: v for k, v in vars(type(self.check)).items() if k.isupper()}
        return f"BlockShiftRepair({params}, check={check})"
```

Строка вида `BlockShiftRepair({'shifts': (0, 180), 'min_flip': 5, ...}, check={'WINDOW': 7, ...})` — все параметры и константы `FrequencyCheck`. Входит в ключ кэша `MergedWellRepository`: изменение любого

порога сбрасывает кэш, а не отдаёт старые данные.

Пример

`repr(BlockShiftRepair())` содержит `'min_share': 0.25` и `'MIN_RISE': 0.3`.

метод BlockShiftRepair.segments

```
def segments(self, presence: np.ndarray, dates: pd.DatetimeIndex) -> list[Segment]
```

```
154     def segments(self, presence: np.ndarray, dates: pd.DatetimeIndex) -> list[Segment]:
155         """Делит календарь там, где набор скважин с данными резко меняется."""
156         flips = (presence[:, 1:] != presence[:, :-1]).sum(axis=0)
157         cuts = [0, *(np.flatnonzero(flips >= self.min_flip) + 1).tolist(), presence.shape[1]]
158         return [
159             Segment(
160                 first=a,
161                 last=b - 1,
162                 start=dates[a].date(),
163                 end=dates[b - 1].date(),
164                 wells=int(presence[:, a:b].any(axis=1).sum()),
165                 cells=int(presence[:, a:b].sum()),
166             )
167             for a, b in zip(cuts[:-1], cuts[1:], strict=True)
168             if presence[:, a:b].any()
169         ]
```

Делит календарь на сегменты. Принимает булеву матрицу `presence` (скважины × дни, есть ли хоть какое-то значение) и даты `dates`. Для каждой пары соседних дней считает, у скольких скважин присутствие данных изменилось; там, где таких `min_flip` (5) и больше, проходит граница. Сегменты без единой ячейки данных отбрасываются. Возвращает список `Segment` (сдвиг пока 0).

Идея: выгрузка приходит пачкой скважин, и когда её данные кончаются или начинаются, набор скважин меняется резко.

Пример

На прежнем склеенном файле получалось 39 сегментов. Перед 01.12.2023 число скважин с данными скачет с 73 (29.11) до 122 (30.11) и 115 (01.12) — здесь начинаются новые сегменты; сегмент в один день 30.11.2023 — стык выгрузок.

метод BlockShiftRepair.assess

```
def assess( self, segments: Sequence[Segment], presence: np.ndarray, frequency: np.ndarray, labels:
Sequence[LabelPoint], reference: np.ndarray | None = None, observed: np.ndarray | None = None, ) -> None
```

```
171     def assess(
172         self,
173         segments: Sequence[Segment],
174         presence: np.ndarray,
175         frequency: np.ndarray,
176         labels: Sequence[LabelPoint],
177         reference: np.ndarray | None = None,
178         observed: np.ndarray | None = None,
179     ) -> None:
180         """Заполняет доказательства и решение каждого сегмента.
181
182         ``reference`` — исходная выгрузка в тех же координатах (NaN, где её нет),
183         ``observed`` — тот же параметр склеенного файла.
184         """
185         for segment in segments:
186             wells = presence[:, segment.first : segment.last + 1].any(axis=1)
```

```
            for shift in self.shifts:
                segment.labels[shift] = self._label_evidence(segment, shift, wells, frequency, labels)
                if reference is not None and observed is not None:
                    segment.reference[shift] = self._reference_match(segment, shift, reference,
observed)
            segment.shift, segment.verdict = self._decide(segment)

        # Стык выгрузок (1–2 дня) проверить нечем — он принадлежит соседнему сегменту.
        for i in reversed(range(len(segments))):
            segment = segments[i]
            if segment.days >= self.seam_days or not segment.verdict.startswith("проверить нечем"):
                continue
            neighbour = segments[i + 1] if i + 1 < len(segments) else segments[i - 1] if i else None
            if neighbour is not None:
                segment.shift = neighbour.shift
                segment.verdict = f"стык выгрузок: как у сегмента {neighbour.start} — {neighbour.end}"
```

Заполняет в каждом сегменте доказательства и решение (на месте, ничего не возвращает). Для каждого сдвига из `shifts`: `_label_evidence` — по разметке, `_reference_match` — по исходной выгрузке (только если переданы `reference` и `observed`); затем `_decide` выбирает `shift` и `verdict`. После этого стыки выгрузок (сегменты короче `seam_days` с вердиктом «проверить нечем») получают решение соседнего сегмента: следующего, а у последнего — предыдущего.

Параметры: `presence` — матрица присутствия данных, `frequency` — таблица частоты (Гц), `labels` — `LabelPoint`, `reference` — Qж исходной выгрузки в тех же координатах, `observed` — Qж склеенного файла.

Пример

Сегмент 2026-06-30 – 2026-06-30 (342 скважины, 1 день): по исходной выгрузке сдвиг 180 подтверждают 332 из 339 скважин, сдвиг 0 — 20 из 339 → «сдвиг −180 дн». Сегмент 30.11.2023 — один день, проверить нечем → берёт решение соседа 01.12.2023 – 01.03.2024 («сдвиг −180 дн»).

метод BlockShiftRepair.apply

```
def apply( self, tables: Mapping[str, pd.DataFrame], segments: Sequence[Segment] ) -> dict[str,
pd.DataFrame]
```

```
    def apply(
        self, tables: Mapping[str, pd.DataFrame], segments: Sequence[Segment]
    ) -> dict[str, pd.DataFrame]:
        """Таблицы после исправления. Сначала несдвинутые сегменты, затем сдвинутые —
        при наложении остаются несдвинутые данные (их место подтверждено)."""
        order = sorted((s for s in segments if s.shift is not None), key=lambda s: s.shift)
        repaired = {}
        for name, table in tables.items():
            source = table.to_numpy(dtype=float)
            target = np.full_like(source, np.nan)
            for segment in order:
                a, b = segment.first, segment.last + 1
                clip = max(0, segment.shift - a)  # сдвиг за начало календаря
                block = source[:, a + clip : b]
                place = target[:, a + clip - segment.shift : b - segment.shift]
                free = np.isnan(place)
                if name == "debit_liquid":
                    segment.collisions = int((~free & ~np.isnan(block)).sum())
                place[free] = block[free]
            repaired[name] = pd.DataFrame(target, index=table.index, columns=table.columns)
        return repaired
```

Строит исправленные таблицы. Принимает словарь `tables` (параметр → таблица «скважина × дата») и сегменты, возвращает новый словарь той же формы. Исключённые сегменты (`shift is None`) не попадают в результат. Сегменты обрабатываются в порядке возрастания сдвига: сначала несдвинутые, потом

сдвинутые; перенос в занятую ячейку не перезаписывает значение (место несдвинутых данных подтверждено). Для параметра `debit_liquid` число таких столкновений пишется в `segment.collisions`. Если сдвиг уходит за начало календаря, начало блока обрезается (`clip`).

Пример

Сегмент 04.09.2026 – 27.12.2026 с `shift=180` переезжает на 08.03.2026 – 30.06.2026 (минус 180 дней). 228 ячеек Qж этого блока попали на уже занятые даты и отброшены.

метод BlockShiftRepair._label_evidence

`def _label_evidence( self, segment: Segment, shift: int, wells: np.ndarray, frequency: np.ndarray, labels: Sequence[LabelPoint], ) -> Evidence`

```
225     def _label_evidence(
226         self,
227         segment: Segment,
228         shift: int,
229         wells: np.ndarray,
230         frequency: np.ndarray,
231         labels: Sequence[LabelPoint],
232     ) -> Evidence:
233         evidence = Evidence()
234         for label in labels:
235             day = label.day + shift  # где подъём должен быть в склеенном файле
236             if not wells[label.row] or not segment.first <= day <= segment.last:
237                 continue
238             series = frequency[label.row, segment.first : segment.last + 1]
239             result = self.check.confirms(series, day - segment.first, label.current, label.new)
240             if result is not None:
241                 evidence.tested += 1
242                 evidence.confirmed += int(result)
243         return evidence
```

Доказательства по разметке для одного сегмента и одного сдвига. Для каждого мероприятия: ожидаемый день подъёма в склеенном файле = `label.day + shift`; мероприятие учитывается, только если скважина есть в сегменте и этот день лежит внутри сегмента. `FrequencyCheck.confirms` на срезе частоты сегмента: `True` → `tested+1, confirmed+1`, `False` → `tested+1`, `None` → мероприятие не считается.

Пример

10335/988, начало 14.08.2023: при сдвиге 0 день подъёма (14.08.2023) вне сегмента 01.12.2023 – 01.03.2024 и не считается; при сдвиге 180 это 10.02.2024, в сегмент попадает, ступенька находится → `confirmed+1`.

метод BlockShiftRepair._reference_match

`def _reference_match( self, segment: Segment, shift: int, reference: np.ndarray, observed: np.ndarray ) -> Evidence`

```
245     def _reference_match(
246         self, segment: Segment, shift: int, reference: np.ndarray, observed: np.ndarray
247     ) -> Evidence:
248         """Сколько скважин сегмента дословно совпадает с исходной выгрузкой при данном сдвиге."""
249         a, b = segment.first, segment.last + 1
250         evidence = Evidence()
251         if a - shift < 0:
252             return evidence
253         mine = observed[:, a:b]
254         theirs = reference[:, a - shift : b - shift]
255         comparable = ~np.isnan(mine) & ~np.isnan(theirs) & (theirs != 0)
256         for row in np.flatnonzero(comparable.any(axis=1)):
257             cells = comparable[row]
```

```
            evidence.tested += 1
            evidence.confirmed += int(
                np.isclose(mine[row, cells], theirs[row, cells]).mean() >= self.well_match
            )
        return evidence
```

Сколько скважин сегмента дословно совпадает с исходной выгрузкой 2026 г. при данном сдвиге. Сравнивается Qж: окно исходной выгрузки `reference[:, a-shift : b-shift]` с окном склейки. Берутся ячейки, где есть оба значения и в исходной выгрузке не 0; скважина «совпала», если не менее `well_match` (90 %) ячеек равны (`np.isclose`). Если окно уходит за начало календаря — пустое `Evidence`.

Пример

Сегмент 04.09.2026 – 27.12.2026: при сдвиге 180 совпали 343 из 343 скважин; при сдвиге 0 сравнивать нечем (исходная выгрузка кончается 31.08.2026, а сегмент начинается 04.09.2026).

метод BlockShiftRepair._decide

```
def _decide(self, segment: Segment) -> tuple[int | None, str]
```

```
    def _decide(self, segment: Segment) -> tuple[int | None, str]:
        reference = {s: e for s, e in segment.reference.items() if e.tested >=
self.min_reference_wells}
        matching = [s for s, e in reference.items() if e.share >= self.reference_share]
        if matching:
            shift = max(matching, key=lambda s: reference[s].share)
            e = reference[shift]
            return shift, f"совпадает с исходной выгрузкой: {e.confirmed} из {e.tested} скважин"

        confirmed = [
            s
            for s, e in segment.labels.items()
            if e.confirmed >= self.min_confirmed and e.share >= self.min_share
        ]
        if confirmed:
            shift = max(confirmed, key=lambda s: (segment.labels[s].share,
segment.labels[s].confirmed))
            e = segment.labels[shift]
            return shift, f"разметка подтверждает: {e.confirmed} из {e.tested} мероприятий"

        if reference:
            parts = ", ".join(f"{s} дн — {e.confirmed}/{e.tested}" for s, e in
sorted(reference.items()))
            return None, f"не совпадает с исходной выгрузкой ни при одном сдвиге (скважин: {parts})"
        tested = sum(e.tested for e in segment.labels.values())
        if tested < self.min_confirmed:
            return 0, f"проверить нечем (мероприятий: {tested}) — оставлен на месте"
        parts = ", ".join(f"{s} дн — {e.confirmed}/{e.tested}" for s, e in
sorted(segment.labels.items()))
        return None, f"разметка не подтверждает ни один сдвиг ({parts})"
```

Выбирает `(shift, verdict)` для сегмента. Порядок:

1. **Исходная выгрузка**: берутся сдвиги, где проверено не меньше `min_reference_wells` (3) скважин; если доля совпавших ≥ 0,5 — побеждает сдвиг с максимальной долей.
2. **Разметка**: сдвиги, где подтверждено ≥ 3 мероприятий и доля ≥ 0,25; побеждает с лучшей долей.
3. Если исходная выгрузка была, но не совпала ни при одном сдвиге — сегмент исключается (`None`).
4. Если проверено меньше `min_confirmed` мероприятий — «проверить нечем», сегмент остаётся на месте (сдвиг 0).
5. Иначе («разметка не подтверждает ни один сдвиг») — исключается.

Пример

- 13.07.2021 – 29.09.2021: сдвиг 0 — 4 из 14 (0,29 ≥ 0,25), сдвиг 180 — 2 из 4 (доля 0,5, но подтверждено 2 < 3) → «без сдвига».
- 01.06.2023 – 30.10.2023: сдвиг 0 — 2 из 3, сдвиг 180 — 1 из 1; подтверждено меньше трёх → «исключён».
- 02.05.2024 – 30.05.2024 (проверено 0 мероприятий) → «проверить нечем (мероприятий: 0) — оставлен на месте».

data/dataset.py

Сборка итогового набора скважин из файлов статистики 2020–2026 и отчёт о том, что и почему не попало в обучение. Это последний шаг пакета `data`: его результат (`list[WellSeries]` с проверенной разметкой) идёт в `features` и `evaluation`.

Порядок работы `MergedWellRepository._build`:

1. читаются числовые ячейки статистики — из папки полугодовых `.xls` (`HalfYearStatisticsReader`) или из склеенного файла (`MergedStatisticsReader`) — и 786 строк разметки (`MonitoringLabelsReader`);
2. отсев разметки по причинам, видным без замеров (`_prefilter`);
3. **только для склеенного файла** (`statistics.is_file()`): поиск и исправление сдвинутых на 180 дней блоков (`BlockShiftRepair`, `_repair`); в исходных полугодовых файлах сдвига нет, шаг пропускается;
4. скважины без телеметрии частоты не используются: подъём F на них не увидеть;
5. каждое оставшееся мероприятие проверяется по замерам (`_verify`); берутся те, у которых частота около даты начала действительно поднялась (`FrequencyCheck`).

Пример

```python
from gtm_detector.ml_freq.config import DataConfig
from gtm_detector.ml_freq.data import MergedWellRepository
c = DataConfig()
wells, audit = MergedWellRepository(c.statistics, c.labels, c.reference).build()
len(wells)                              # 611 скважин в обучении/проверке
sum(1 for w in wells if w.labels)       # 369 скважин с разметкой
sum(len(w.labels) for w in wells)       # 424 мероприятия
audit.labels["решение"].value_counts()  # причины отказа по каждой из 786 строк
```

Это та же таблица, что в `results/ml_freq/data_audit_halfyear/data_audit.xlsx`; CLI-команда `audit` пишет её в файлы.

Код модуля вне классов и функций

```python
from __future__ import annotations

import hashlib
import logging
import pickle
import tempfile
from collections.abc import Sequence
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

from .models import SERIES_COLUMNS, Interval, WellSeries
from .readers import MeasurementsReader
from .repair import BlockShiftRepair, FrequencyCheck, LabelPoint, Segment
from .statistics import (
    HalfYearStatisticsReader,
    LabelRecord,
    MergedStatisticsReader,
    MonitoringLabelsReader,
)

logger = logging.getLogger(__name__)

ACCEPTED = "принято"
```

класс-данные (dataclass) DataAudit

`class DataAudit`

```python
@dataclass
class DataAudit:
    """Что было в файлах и что из этого взято для обучения."""

    segments: list[Segment]
    labels: pd.DataFrame  # строка итогового файла + «решение»
    wells: pd.DataFrame  # скважина + «статус»
    non_numeric: pd.Series  # ячеек «не число» по датам (все листы)
    wells_per_day_before: pd.Series  # скважин с данными по дням, до исправления
    wells_per_day_after: pd.Series
    reference: Path | None
```

Отчёт о сборке (обычный dataclass; пишется в кэш вместе со скважинами). Поля:

Поле	Что содержит
`segments`	список `Segment` с решением по каждому сегменту (39 сегментов склеенного файла: 25 без сдвига, 7 со сдвигом −180, 7 исключено; для полугодовых файлов список пуст)
`labels`	таблица всех 786 строк итогового файла + столбец «решение»
`wells`	таблица по скважинам: дни с замерами, доля дней с частотой > 10 Гц, число принятых мероприятий, статус
`non_numeric`	число заполненных, но нечисловых ячеек по датам
`wells_per_day_before` / `_after`	скважин с данными по дням до и после исправления
`reference`	путь к эталонной выгрузке (склеенный файл) или `None`

Пример

Распределение столбца «решение» на полугодовых файлах: принято 424, частота в замерах не поднялась 313, дубликат 40, нет телеметрии частоты у скважины 5, «частота не увеличена» 3, не выполнено 1 (сумма 786). Статусы скважин: 242 «используется, без разметки», 369 «используется, с разметкой», 5 «не используется: нет телеметрии частоты» (всего 616). Для прежнего склеенного файла картина была иной: принято 135, скважины нет в статистике 333, нет замеров около даты 153, частота не поднялась 89.

класс MergedWellRepository

`class MergedWellRepository`

```python
class MergedWellRepository:
    """Скважины с проверенной разметкой; результат кэшируется.

    ``statistics`` — папка с полугодовыми файлами или один склеенный файл.
    """

    VERSION = 2  # менять при изменении логики — сбрасывает кэш
    MIN_TELEMETRY_SHARE = 0.1  # доля дней с замерами, где частота > 10 Гц
    DATA_WINDOW = 14  # дней вокруг даты начала, где должны быть замеры

    ...

    ...

    ...

    ...

    ...
```

```
176
    ...
204
    ...
229
    ...
278
    ...
285
    ...
291
    ...
300
    ...
322
```

Скважины с проверенной разметкой; результат кэшируется (`merged_<хэш>.pkl` в `%TEMP%/gtm_ml_freq_cache`). `statistics` — папка с полугодовыми файлами или один склеенный файл. Константы: `VERSION = 2` (менять при изменении логики — сбрасывает кэш), `MIN_TELEMETRY_SHARE = 0,1` (доля дней с замерами, где частота > 10 Гц; ниже — скважина не используется) и `DATA_WINDOW = 14` дней вокруг даты начала, где должны быть замеры.

Пример

Первое построение читает Excel около 30 секунд; потом `build()` берёт готовый результат из кэша за секунды.

метод MergedWellRepository.__init__

```
def __init__( self, statistics: Path, labels: Path | None, reference: Path | None = None, cache_dir: Path | None = None, repair: BlockShiftRepair | None = None, ) -> None
```

```
67      def __init__(
68          self,
69          statistics: Path,
70          labels: Path | None,
71          reference: Path | None = None,
72          cache_dir: Path | None = None,
73          repair: BlockShiftRepair | None = None,
74      ) -> None:
75          self.statistics = Path(statistics)
76          self.labels = Path(labels) if labels else None
77          self.reference = Path(reference) if reference and Path(reference).exists() else None
78          self.cache_dir = cache_dir or Path(tempfile.gettempdir()) / "gtm_ml_freq_cache"
79          self.repair = repair or BlockShiftRepair()
80          self.check = self.repair.check
```

Принимает пути: `statistics`, `labels` (может быть `None` — тогда разметки нет), `reference` (исходная выгрузка; если файла нет, считается `None` и сверки по ней не будет), `cache_dir` (по умолчанию `%TEMP%/gtm_ml_freq_cache`), `repair` (по умолчанию `BlockShiftRepair()`). Запоминает также `self.check = repair.check` — тот же `FrequencyCheck`, что использует починка, чтобы проверка подъёма везде была одной.

Пример

```
MergedWellRepository(c.statistics, c.labels, c.reference)   # reference = None для полугодовых файлов
MergedWellRepository(c.statistics, None)   # только скважины, без разметки
```

метод MergedWellRepository.load

```
def load(self) -> list[WellSeries]
```

```
    def load(self) -> list[WellSeries]:
        return self.build()[0]
```

Короткий путь: только список скважин, `build()[0]`. Его вызывает обучение, а отчёт `DataAudit` нужен только команде `audit`.

Пример

`MergedWellRepository(c.statistics, c.labels, c.reference).load()` → 611 `WellSeries`.

метод MergedWellRepository.build

`def build(self) -> tuple[list[WellSeries], DataAudit]`

```
    def build(self) -> tuple[list[WellSeries], DataAudit]:
        cache = self.cache_dir / f"merged_{self._fingerprint()}.pkl"
        if cache.exists():
            logger.info("Данные из кэша: %s", cache)
            with cache.open("rb") as file:
                return pickle.load(file)
        result = self._build()
        self.cache_dir.mkdir(parents=True, exist_ok=True)
        with cache.open("wb") as file:
            pickle.dump(result, file)
        return result
```

Возвращает пару `(скважины, DataAudit)`. Если файл кэша с отпечатком `_fingerprint()` есть — читает `pickle`; иначе вызывает `_build` и записывает кэш. Отпечаток включает версию, `repr` починки и параметры файлов, поэтому смена порогов или данных сама сбрасывает кэш; правки кода в `readers.py`/`statistics.py` кэш не видит (кроме смены `VERSION`) — тогда папку нужно удалить.

Пример

`wells, audit = repo.build()`; повторный вызов — «Данные из кэша: .../merged_<хэш>.pkl» в логе.

метод MergedWellRepository._build

`def _build(self) -> tuple[list[WellSeries], DataAudit]`

```
    def _build(self) -> tuple[list[WellSeries], DataAudit]:
        glued = self.statistics.is_file()
        reader = (
            MergedStatisticsReader(self.statistics)
            if glued
            else HalfYearStatisticsReader(self.statistics)
        )
        tables = reader.read()
        index = tables["frequency"].index
        for table in tables.values():
            index = index.union(table.index)
        dates = tables["frequency"].columns
        tables = {name: table.reindex(index=index, columns=dates) for name, table in tables.items()}
        presence = np.any([table.notna().to_numpy() for table in tables.values()], axis=0)

        records = MonitoringLabelsReader(self.labels).read() if self.labels else []
        decisions, rows = self._prefilter(records, index)
        segments: list[Segment] = []
        repaired = tables
        if glued:
            points = [
                LabelPoint(rows[i], dates.get_loc(pd.Timestamp(r.start)), r.current, r.new)
                for i, r in enumerate(records)
```

```
                if decisions[i] is None and pd.Timestamp(r.start) in dates
            ]
            segments, repaired = self._repair(tables, presence, points, index, dates)

        frequency = repaired["frequency"].to_numpy()
        has_data = np.any([table.notna().to_numpy() for table in repaired.values()], axis=0)
        telemetry = self._telemetry_share(frequency, has_data)
        dropped_day = self._dropped_days(segments, presence)

        for i, record in enumerate(records):
            if decisions[i] is None:
                decisions[i] = self._verify(
                    record, rows[i], dates, frequency, has_data, telemetry, dropped_day
                )

        wells, well_rows = self._wells(repaired, index, has_data, telemetry, records, rows, decisions)
        audit = DataAudit(
            segments=segments,
            labels=self._labels_table(records, decisions),
            wells=pd.DataFrame(well_rows),
            non_numeric=reader.non_numeric,
            wells_per_day_before=pd.Series(presence.sum(axis=0), index=dates),
            wells_per_day_after=pd.Series(has_data.sum(axis=0), index=dates),
            reference=self.reference,
        )
        logger.info(
            "Скважин: %d, с разметкой: %d, мероприятий: %d",
            len(wells),
            sum(1 for w in wells if w.labels),
            sum(len(w.labels) for w in wells),
        )
        return wells, audit
```

Вся сборка без кэша.

- Выбирает читателя: `statistics.is_file()` — склеенный `MergedStatisticsReader`, иначе `HalfYearStatisticsReader` (папка).
- Читает таблицы параметров, объединяет индексы скважин всех листов и приводит таблицы к общему индексу и календарю частоты. `presence` — матрица «есть ли хоть какое-то значение».
- Читает разметку, `_prefilter` даёт причины и строку скважины; для записей без отказа строятся `LabelPoint`.
- Для склеенного файла: строит `LabelPoint` и вызывает `_repair` (сегментирование, оценка, применение; вердикты сегментов — в лог). Для полугодовых файлов сегментов нет, таблицы идут дальше как есть.
- По исправленным данным считает `has_data`, долю телеметрии и «дни исключённых сегментов», проверяет оставшиеся мероприятия через `_verify`, собирает скважины `_wells` и `DataAudit`.

Пример

Лог на полугодовых файлах: «Статистика: 14 файлов, 616 скважин, 2498 дат» и итог «Скважин: 611, с разметкой: 369, мероприятий: 424». Для склеенного файла в лог ещё шли строки вида «Сегмент 2026-09-04 — 2026-12-27: сдвиг −180 дн (совпадает с исходной выгрузкой: 343 из 343 скважин)».

метод MergedWellRepository._repair

```
def _repair( self, tables: dict[str, pd.DataFrame], presence: np.ndarray, points: Sequence[LabelPoint], index: pd.MultiIndex, dates: pd.DatetimeIndex, ) -> tuple[list[Segment], dict[str, pd.DataFrame]]
```

```
    def _repair(
        self,
        tables: dict[str, pd.DataFrame],
        presence: np.ndarray,
```

```
        points: Sequence[LabelPoint],
        index: pd.MultiIndex,
        dates: pd.DatetimeIndex,
    ) -> tuple[list[Segment], dict[str, pd.DataFrame]]:
        """Сдвинутые блоки склеенного файла: найти, оценить, вернуть на место или исключить."""
        reference, observed = self._reference(index, dates), tables["debit_liquid"].to_numpy()
        segments = self.repair.segments(presence, dates)
        frequency = tables["frequency"].to_numpy()
        self.repair.assess(segments, presence, frequency, points, reference, observed)
        repaired = self.repair.apply(tables, segments)
        for segment in segments:
            logger.info(
                "Сегмент %s — %s: %s (%s)",
                segment.start,
                segment.end,
                segment.decision,
                segment.verdict,
            )
        return segments, repaired
```

Вынесенный из `_build` блок исправления сдвигов, который запускается **только для склеенного файла**. Принимает таблицы параметров, матрицу присутствия `presence`, список `LabelPoint` (проверяемые мероприятия), общий индекс и календарь. Берёт эталонную выгрузку (`_reference`), делит календарь на сегменты, оценивает их (`assess`) и применяет решения (`apply`: сдвинуть на −180 дней, исключить или оставить), пишет вердикты в лог. Возвращает пару `(segments, repaired)`.

Для полугодовых файлов `_build` эту функцию не вызывает: `segments == []`, таблицы идут дальше без изменений.

Пример

Склеенный файл: `segments, repaired = repo._repair(...)` -> 39 сегментов (25 без сдвига, 7 со сдвигом −180, 7 исключено). Папка `resources/statistics`: функция не вызывается, а `DataAudit.segments` пуст и лист «сегменты» в `data_audit.xlsx` тоже.

статический метод MergedWellRepository._prefilter

```
def _prefilter( records: Sequence[LabelRecord], index: pd.MultiIndex ) -> tuple[list[str | None], list[int | None]]
```

```
    @staticmethod
    def _prefilter(
        records: Sequence[LabelRecord], index: pd.MultiIndex
    ) -> tuple[list[str | None], list[int | None]]:
        """Причины, видные без замеров, и строка скважины в таблицах для каждой записи."""
        position = {key: i for i, key in enumerate(index)}
        by_number: dict[str, list[int]] = {}
        for i, (number, _) in enumerate(index):
            by_number.setdefault(number, []).append(i)

        decisions: list[str | None] = []
        rows: list[int | None] = []
        seen = set()
        for record in records:
            row = position.get(record.key)
            if row is None and len(by_number.get(record.number, [])) == 1:
                row = by_number[record.number][0]  # куст записан иначе, но номер уникален
            rows.append(row)
            reason = MonitoringLabelsReader.rejection(record)
            if reason is None and (record.number, record.start) in seen:
                reason = "дубликат (та же скважина и дата начала)"
            if reason is None and row is None:
                reason = "скважины нет в статистике"
            if reason is None:
```

```
                seen.add((record.number, record.start))
            decisions.append(reason)
        return decisions, rows
```

Статический метод. Для каждой записи разметки находит строку скважины в таблицах (по `(номер, куст)`; если такой пары нет, но номер уникален — по номеру) и причину отказа, видную без замеров. Возвращает два списка: `decisions` (`None` — пока годится, иначе текст) и `rows` (индекс строки или `None`). Порядок проверок: `MonitoringLabelsReader.rejection`; дубликат (одна скважина и та же дата начала — оставляется первая строка); «скважины нет в статистике».

Пример

Из 786 записей здесь отсеивается 40 «дубликат», 3 «частота не увеличена» и 1 «не выполнено»; «скважины нет в статистике» теперь нет ни у одной строки (в полугодовых файлах 616 скважин). В прежнем склеенном файле таких было 333.

метод MergedWellRepository._verify

```
def _verify( self, record: LabelRecord, row: int, dates: pd.DatetimeIndex, frequency: np.ndarray, has_data: np.ndarray, telemetry: np.ndarray, dropped_day: np.ndarray, ) -> str
```

```
    def _verify(
        self,
        record: LabelRecord,
        row: int,
        dates: pd.DatetimeIndex,
        frequency: np.ndarray,
        has_data: np.ndarray,
        telemetry: np.ndarray,
        dropped_day: np.ndarray,
    ) -> str:
        if pd.Timestamp(record.start) not in dates:
            return "дата начала вне календаря статистики"
        day = dates.get_loc(pd.Timestamp(record.start))
        window = slice(max(day - self.DATA_WINDOW, 0), day + self.DATA_WINDOW + 1)
        if not has_data[row, window].any():
            if dropped_day[row, window].any():
                return "замеры около даты начала — в исключённом сегменте"
            return "нет замеров около даты начала"
        if telemetry[row] < self.MIN_TELEMETRY_SHARE:
            return "нет телеметрии частоты у скважины"
        result = self.check.confirms(frequency[row], day, record.current, record.new)
        if result is None:
            return "нет замеров частоты около даты начала"
        return ACCEPTED if result else "частота в замерах не поднялась около даты начала"
```

Проверка одной записи по замерам после починки. Возвращает «решение» — `ACCEPTED` («принято») или причину отказа:

1. дата начала вне календаря → «дата начала вне календаря статистики»;
2. нет ни одного замера в окне ±14 дней от начала: если там были данные исключённого сегмента — «замеры около даты начала — в исключённом сегменте», иначе «нет замеров около даты начала»;
3. доля телеметрии скважины < 0,1 → «нет телеметрии частоты у скважины»;
4. `FrequencyCheck.confirms` вернул `None` → «нет замеров частоты около даты начала», `False` → «частота в замерах не поднялась около даты начала», `True` → принято.

Пример

10335/988, начало 14.08.2023: замеры есть, доля телеметрии высокая, частота 49,5 → 51,5 Гц → «принято». Скважина 10784/179, начало 23.08.2021 (148 → 150,77 Гц по разметке; строка Excel 79): частота в ±3 дня не

выросла ни до «Нового значения», ни на половину прироста — в `rejected_labels.csv` «частота в замерах не поднялась около даты начала».

метод MergedWellRepository._wells

```
def _wells( self, repaired: dict[str, pd.DataFrame], index: pd.MultiIndex, has_data: np.ndarray, telemetry: np.ndarray, records: Sequence[LabelRecord], rows: Sequence[int | None], decisions: Sequence[str], ) -> tuple[list[WellSeries], list[dict]]
```

```python
    def _wells(
        self,
        repaired: dict[str, pd.DataFrame],
        index: pd.MultiIndex,
        has_data: np.ndarray,
        telemetry: np.ndarray,
        records: Sequence[LabelRecord],
        rows: Sequence[int | None],
        decisions: Sequence[str],
    ) -> tuple[list[WellSeries], list[dict]]:
        labels: dict[int, list[Interval]] = {}
        for record, row, decision in zip(records, rows, decisions, strict=True):
            if decision == ACCEPTED:
                labels.setdefault(row, []).append(
                    Interval(record.start, record.end, censored=record.censored)
                )

        dates = repaired["frequency"].columns
        wells, table = [], []
        for row, (number, cluster) in enumerate(index):
            days = np.flatnonzero(has_data[row])
            if not len(days):
                status = "нет данных после исправления"
            elif telemetry[row] < self.MIN_TELEMETRY_SHARE:
                status = "не используется: нет телеметрии частоты"
            else:
                status = "используется, с разметкой" if row in labels else "используется, без разметки"
            table.append(
                {
                    "скважина": number,
                    "куст": cluster,
                    "дней с замерами": len(days),
                    "доля дней с частотой > 10 Гц": round(float(telemetry[row]), 3),
                    "мероприятий принято": len(labels.get(row, [])),
                    "статус": status,
                }
            )
            if not status.startswith("используется"):
                continue
            span = slice(days[0], days[-1] + 1)
            frame = pd.DataFrame(
                {name: repaired[name].iloc[row, span].to_numpy() for name in SERIES_COLUMNS},
                index=dates[span],
            )
            frame["state"] = (frame["frequency"] > 0).astype(float)
            intervals = sorted(labels.get(row, []), key=lambda interval: interval.start)
            wells.append(WellSeries(number=number, cluster=cluster, frame=frame, labels=intervals))
        return wells, table
```

Собирает `WellSeries` и таблицу по скважинам. Сначала группирует принятые мероприятия по строкам скважин в `Interval(start, end, censored=…)`. Для каждой строки таблиц задаёт статус: «нет данных после исправления» (ни одного дня с данными), «не используется: нет телеметрии частоты» (доля < 0,1) или «используется, с разметкой»/«используется, без разметки». Для используемых строится `frame` от первого до

последнего дня с данными, столбцы `SERIES_COLUMNS`, `state = (frequency > 0)`; интервалы сортируются по началу. Возвращает `(wells, строки таблицы)`.

Пример

Скважина 10335/988: `frame` 01.01.2020 – 18.03.2026 (2269 дней), 1 мероприятие 14.08–20.09.2023, статус «используется, с разметкой». Статус по всем 616 строкам: 242 «без разметки» / 369 «с разметкой» / 5 «не используется».

метод MergedWellRepository._reference

```
def _reference(self, index: pd.MultiIndex, dates: pd.DatetimeIndex) -> np.ndarray | None
```

```
279     def _reference(self, index: pd.MultiIndex, dates: pd.DatetimeIndex) -> np.ndarray | None:
280         """Исходная выгрузка (Qж) в координатах склеенного файла — только для проверки сдвигов."""
281         if self.reference is None:
282             return None
283         table = MeasurementsReader(self.reference).read_parameter("Qж")
284         return table.reindex(index=index, columns=dates).to_numpy(dtype=float)
```

Исходная выгрузка (лист «Qж» файла `input_dataset.xlsx`) в координатах склеенной таблицы: индекс скважин и календарь те же, нет данных — `NaN`. Только для сверки сдвигов; в обучение не идёт. Без `reference` возвращает `None`.

Пример

Нужна только для склеенного файла (для папки полугодий `reference = None`, и функция возвращает `None`). Раньше: матрица 768 × 2557, заполненная только на 2026 г.

статический метод MergedWellRepository._telemetry_share

```
def _telemetry_share(frequency: np.ndarray, has_data: np.ndarray) -> np.ndarray
```

```
286     @staticmethod
287     def _telemetry_share(frequency: np.ndarray, has_data: np.ndarray) -> np.ndarray:
288         working = np.nan_to_num(frequency) > FrequencyCheck.MIN_WORKING
289         days = has_data.sum(axis=1)
290         return np.divide((working & has_data).sum(axis=1), days, out=np.zeros(len(days)), where=days >
    0)
```

Для каждой скважины: доля дней с данными, в которые частота выше `FrequencyCheck.MIN_WORKING` (10 Гц). Принимает массив частоты (`NaN` → 0) и матрицу `has_data`; для скважины без данных — 0. Возвращает массив долей от 0 до 1.

Пример

Скважина, у которой данные есть 200 дней и частота > 10 Гц в 20 из них, получает 0,1 — на пороге `MIN_TELEMETRY_SHARE`. Таких «слепых» скважин с долей < 0,1 в полугодовых данных 5 (в склеенном файле было 70).

статический метод MergedWellRepository._dropped_days

```
def _dropped_days(segments: Sequence[Segment], presence: np.ndarray) -> np.ndarray
```

```
292     @staticmethod
293     def _dropped_days(segments: Sequence[Segment], presence: np.ndarray) -> np.ndarray:
294         dropped = np.zeros_like(presence)
295         for segment in segments:
296             if segment.shift is None:
```

```
                span = slice(segment.first, segment.last + 1)
                dropped[:, span] = presence[:, span]
        return dropped
```

Матрица «скважина × день»: `True` там, где данные были в исходном файле, но сегмент исключён. Нужна `_verify`, чтобы объяснить отказ («замеры около даты начала — в исключённом сегменте»), а не писать просто «нет замеров».

Пример

Только для склеенного файла: если сегмент 01.06.2023 – 30.10.2023 исключён (сдвиг не подтвердился ни разметкой, ни эталоном), мероприятие с датой начала в нём получает причину «замеры около даты начала — в исключённом сегменте» (в склеенном файле таких отказов было 14). На полугодовых файлах исключённых сегментов нет, матрица пуста.

статический метод MergedWellRepository._labels_table

```
def _labels_table(records: Sequence[LabelRecord], decisions: Sequence[str]) -> pd.DataFrame
```

```
    @staticmethod
    def _labels_table(records: Sequence[LabelRecord], decisions: Sequence[str]) -> pd.DataFrame:
        return pd.DataFrame(
            [
                {
                    "строка Excel": r.row,
                    "скважина": r.number,
                    "куст": r.cluster,
                    "год": r.year,
                    "тип ГТМ": r.kind,
                    "начало": r.start,
                    "окончание": r.end,
                    "частота до, Гц": r.current,
                    "частота после, Гц": r.new,
                    "причина окончания": r.reason,
                    "эффект продолжался": r.censored,
                    "решение": decision,
                }
                for r, decision in zip(records, decisions, strict=True)
            ]
        )
```

Таблица для `DataAudit.labels`: столбцы «строка Excel», «скважина», «куст», «год», «тип ГТМ», «начало», «окончание», «частота до, Гц», «частота после, Гц», «причина окончания», «эффект продолжался», «решение». Для каждой из 786 записей одна строка.

Пример

Строка Excel 307: скважина 10335, куст 988, 2023, 14.08.2023 – 20.09.2023, 49.5 → 49.96 Гц, «прекращение эффекта по воде», эффект продолжался — False, решение «принято».

метод MergedWellRepository._fingerprint

```
def _fingerprint(self) -> str
```

```
    def _fingerprint(self) -> str:
        parts = [f"v{self.VERSION}", repr(self.repair)]
        sources = (
            HalfYearStatisticsReader(self.statistics).files()
            if self.statistics.is_dir()
            else [self.statistics]
        )
        for path in (*sources, self.labels, self.reference):
```

```
            if path is not None:
                stat = path.stat()
                parts.append(f"{path.resolve()}:{stat.st_size}:{stat.st_mtime_ns}")
        return hashlib.sha1("|".join(parts).encode()).hexdigest()[:12]
```

12-символьный хэш SHA-1 от версии (`v2`), `repr(self.repair)` и путей, размеров и времён изменения всех исходных файлов: для папки — каждого из 14 `.xls` (список даёт `HalfYearStatisticsReader.files()`), для склеенного файла — одного файла; плюс разметка и эталонная выгрузка, если она есть. Меняется при замене любого Excel или правке порогов починки — кэш тогда пересчитывается.

Пример

`merged_<12 hex-символов>.pkl`; смена `min_flip=5` на 3 даёт новое имя файла.

features/__init__.py

Пакет `features` — маленькая «витрина»: он только реэкспортирует класс `FeatureBuilder` из `features/builder.py`, чтобы в остальном коде писать `from ..features import FeatureBuilder`.

В цепочке «Excel → скважины → признаки → модель → интервалы» пакет стоит между данными и моделями. На входе — `WellSeries` (таблица `frame`: дни × `SERIES_COLUMNS`), на выходе — числовые матрицы «день × признак». Все модели (`TransformerSegmenter`, `SklearnSegmenter`) получают признаки **только** отсюда; это правило проекта: добавил канал — обнови `SEQUENCE_CHANNELS`, тесты и README.

```
from gtm_detector.ml_freq.features import FeatureBuilder

fb = FeatureBuilder()
fb.sequence_size        # 15 каналов для Transformer
len(fb.tabular_names)   # 106 признаков для scikit-learn
```

Код модуля вне классов и функций

```
3  from .builder import FeatureBuilder
4
5  __all__ = ["FeatureBuilder"]
```

features/builder.py

Здесь из сырых суточных замеров скважины делаются числа, которые понимают модели. Один класс `FeatureBuilder` строит два представления одних и тех же дневных каналов:

- `sequence()` — матрица «дни × 15 каналов» для Transformer: соседние дни модель видит сама через внимание и свёртку;
- `tabular()` — «дни × 106 признаков» для scikit-learn: у каждого дня своя строка, поэтому контекст (значения через ±1…14 дней и средние за неделю) приходится дописывать в строку явно.

Общая идея всех каналов — **относительные** величины: частота относительно рабочей частоты скважины, дебит относительно медианы за прошлые 30 дней, дебит относительно уровня «до скачка». Так 50 Гц на одной скважине и 35 Гц на другой, 20 м³/сут и 400 м³/сут выглядят одинаково, и модели не нужны абсолютные числа (а их при ~120 обучающих скважинах не хватило бы для обобщения). Все нормировки считаются **внутри одной скважины** по её собственному ряду — утечки между скважинами нет.

Модуль не знает про разметку: `target()` и `training_mask()` живут в `WellSeries`. Выход ограничен диапазоном [-3; 3] (константа `CLIP`), пропуски заменены нулями.

Код модуля вне классов и функций

```python
from __future__ import annotations

import numpy as np
import pandas as pd

from ..data import WellSeries

CLIP = 3.0
```

класс FeatureBuilder

```
class FeatureBuilder
```

```python
class FeatureBuilder:
    """Строит признаки одной скважины. Без состояния — один экземпляр на все скважины."""

    MIN_WORKING_FREQ = 10.0  # Гц: ниже насос считается остановленным
    MIN_STEP = 0.1  # Гц: подъём частоты, после которого считаются признаки «от скачка»
    WINDOW = 30  # дней: «недавний» уровень параметров

    SEQUENCE_CHANNELS = (
        "working",
        "f_rel",
        "f_diff",
        "f_above_min30",
        "f_below_max30",
        "liq_vs_30d",
        "oil_vs_30d",
        "wc_vs_30d",
        "pzab_vs_30d",
        "days_since_step",
        "step_size",
        "liq_vs_step",
        "oil_vs_step",
        "wc_vs_step",
        "liq_missing",
    )
    SHIFTED_CHANNELS = (
        "working",
        "f_diff",
```

```
        "f_above_min30",
        "liq_vs_30d",
        "oil_vs_30d",
        "wc_vs_30d",
        "liq_vs_step",
        "oil_vs_step",
        "wc_vs_step",
    )
    OFFSETS = (-14, -7, -3, -1, 1, 3, 7, 14)
...

...

...

...

...

...

...
```

Класс без состояния: один экземпляр обслуживает все скважины, нечего «обучать» или сохранять. Константы класса — единственные настройки признаков:

Константа	Значение	Смысл
`MIN_WORKING_FREQ`	10 Гц	ниже этой частоты (или при `state` ≤ 0) насос считается остановленным
`MIN_STEP`	0,1 Гц	минимальный подъём частоты, который считается «скачком»
`WINDOW`	30 дней	окно «недавнего» уровня параметров
`SEQUENCE_CHANNELS`	15 имён	порядок столбцов `sequence()`; сохраняется в весах `.pt`
`SHIFTED_CHANNELS`	9 имён	каналы, которые для scikit-learn размножаются сдвигами и средними
`OFFSETS`	−14, −7, −3, −1, +1, +3, +7, +14	сдвиги в днях

Почему порядок `SEQUENCE_CHANNELS` важен: `TransformerSegmenter.save` записывает список каналов в файл весов, а `load` сверяет его с текущим; если канал добавили или переставили, старые веса дадут `ValueError` (иначе сеть молча получила бы данные «не в тех столбцах»).

Что означают 15 каналов (все безразмерные):

Канал	Что это
`working`	1, если насос работает (`state` > 0 и частота > 10 Гц)
`f_rel`	частота относительно медианной рабочей частоты скважины минус 1
`f_diff`	суточное изменение `f_rel`, ×10 (подъём на 1 Гц при 50 Гц → 0,2)
`f_above_min30`, `f_below_max30`	насколько частота выше минимума / ниже максимума за 30 дней, ×10
`liq_vs_30d`, `oil_vs_30d`	дебит жидкости / нефти относительно медианы за прошлые 30 дней
`wc_vs_30d`	обводнённость минус среднее за прошлые 30 дней, /10
`pzab_vs_30d`	забойное давление относительно медианы за 30 дней
`days_since_step`	дней с последнего скачка частоты вверх, делённые на 120 (не больше 1)
`step_size`	размер последнего скачка относительно частоты до него, ×10

`liq_vs_step`, `oil_vs_step`, `wc_vs_step`	дебит / обводнённость относительно уровня за день до скачка
`liq_missing`	1, если в этот день нет замера дебита жидкости

Пример

Синтетическая скважина из `tests/conftest.py` (`make_well()`: 120 дней, частота 48 → 50 Гц в день 40, дебит жидкости 100 → 110 с дня 42 на 30 дней):

```
fb = FeatureBuilder()
well = make_well()
fb.sequence(well).shape        # (120, 15)
fb.tabular(well).shape         # (120, 106)
```

Реальная скважина 8822 (куст 615A, разметка: начало 30.07.2022, окончание 05.09.2022): 1587 дней ряда, из них `training_mask(45)` оставляет 128 дней, а `target()` отмечает 38 дней эффекта.

свойство FeatureBuilder.sequence_size

```
def sequence_size(self) -> int
```

```
57      @property
58      def sequence_size(self) -> int:
59          return len(self.SEQUENCE_CHANNELS)
```

Число каналов `sequence()`, то есть `len(SEQUENCE_CHANNELS)`. Передаётся в `TimeSeriesTransformer` как размер входа свёртки (`n_channels`), чтобы число не дублировалось в коде.

Пример

```
FeatureBuilder().sequence_size   # 15
```

свойство FeatureBuilder.tabular_names

```
def tabular_names(self) -> list[str]
```

```
61      @property
62      def tabular_names(self) -> list[str]:
63          names = list(self.SEQUENCE_CHANNELS)
64          names += [f"{c}@{o:+d}" for c in self.SHIFTED_CHANNELS for o in self.OFFSETS]
65          names += [f"{c}_mean_{side}7" for c in self.SHIFTED_CHANNELS for side in ("past", "next")]
66          names += ["f_diff_max_pm3"]
67          return names
```

Список из 106 имён столбцов `tabular()`. Нужен для отладки и разбора важности признаков.

Состав: 15 основных каналов + 9 каналов × 8 сдвигов (72, имена вида `liq_vs_30d@+7`) + 9 каналов × 2 средних (18, `..._mean_past7` и `..._mean_next7`) + 1 признак `f_diff_max_pm3` = 15 + 72 + 18 + 1 = 106.

> **Внимание, порядок имён не совпадает с порядком столбцов.** Список строится «канал за каналом» (`working@-14`, `working@-7`, …, `working@+14`, затем `f_diff@-14`, …), а `tabular()` кладёт блоки «сдвиг за сдвигом» (сначала все 9 каналов со сдвигом −14, потом все 9 со сдвигом −7 и т. д.); средние тоже идут блоками (9 `past7`, затем 9 `next7`), а в списке они чередуются парами. Совпадают только первые 15 имён и последнее (`f_diff_max_pm3`, столбец 105). Длина списка верна, на обучение это не влияет (модели работают с номерами столбцов), но использовать `tabular_names` для подписей важности признаков нельзя без исправления.

Пример

```
names = FeatureBuilder().tabular_names
names[0], names[14]    # ('working', 'liq_missing')
names[15], names[86]   # ('working@-14', 'wc_vs_step@+14')  - по списку
names[105]             # 'f_diff_max_pm3'
```

Фактически в матрице столбец 15 — `working@-14`, но столбец 16 — `f_diff@-14` (а не `working@-7`), как и подсказывает порядок блоков. Проверка на синтетической скважине: `liq_vs_30d` в день 42 равен 0,105, и в день 35 это значение «через 7 дней» стоит не в столбце с именем `liq_vs_30d@+7` (индекс 33 в списке, там 0,001), а в столбце 72 (`15 + 6·9 + 3`: шестой по порядку сдвиг +7, четвёртый канал).

метод FeatureBuilder.sequence

`def sequence(self, well: WellSeries) -> np.ndarray`

```
69      def sequence(self, well: WellSeries) -> np.ndarray:
70          channels = self._channels(well)
71          return channels[list(self.SEQUENCE_CHANNELS)].to_numpy(dtype=np.float32)
```

Возвращает матрицу `float32` формы `(число_дней, 15)`: столбцы — `SEQUENCE_CHANNELS` в заданном порядке, строки — дни `well.frame.index`. Длину ряда не меняет: Transformer должен выдать вероятность для каждого дня скважины.

Всю работу делает `_channels`, здесь только выбор и порядок столбцов. Значения лежат в [-3; 3], NaN нет.

Пример

Синтетическая скважина, день 40 — день подъёма 48 → 50 Гц:

```
s = fb.sequence(make_well())
s[40, fb.SEQUENCE_CHANNELS.index("f_diff")]        # 0.4: скачок на 4% частоты ×10
s[40, fb.SEQUENCE_CHANNELS.index("step_size")]     # 0.42
s[45, fb.SEQUENCE_CHANNELS.index("liq_vs_step")]   # около 0.10: дебит на 10% выше уровня до скачка
```

Это то, что проверяет `test_features_shapes_and_values`. На реальной скважине 8822 за день до начала по разметке (29.07.2022) `liq_vs_30d` = 0, а в день начала (30.07.2022) жидкость выросла со 157 до 197 м³/сут и `liq_vs_30d` = `liq_vs_step` = 0,25.

метод FeatureBuilder.tabular

`def tabular(self, well: WellSeries) -> np.ndarray`

```
73      def tabular(self, well: WellSeries) -> np.ndarray:
74          channels = self._channels(well)
75          parts = [channels[list(self.SEQUENCE_CHANNELS)]]
76          shifted = channels[list(self.SHIFTED_CHANNELS)]
77          for offset in self.OFFSETS:
78              # offset > 0 — значение через offset дней (детекция офлайн, будущее известно).
79              parts.append(shifted.shift(-offset).fillna(0.0).add_suffix(f"@{offset:+d}"))
80          parts.append(shifted.rolling(7, min_periods=1).mean().add_suffix("_mean_past7"))
81          parts.append(shifted[::-1].rolling(7, min_periods=1).mean()[::-1].add_suffix("_mean_next7"))
82          parts.append(
83              channels["f_diff"]
84              .rolling(7, center=True, min_periods=1)
85              .max()
86              .rename("f_diff_max_pm3")
87              .to_frame()
88          )
89          table = pd.concat(parts, axis=1)
```

```
90          return table.to_numpy(dtype=np.float32)
```

Возвращает матрицу `float32` формы `(число_дней, 106)` для scikit-learn. Собирает таблицу из блоков:

1. 15 основных каналов (как в `sequence`);
2. для 9 каналов `SHIFTED_CHANNELS` — значения через `offset` дней для каждого из 8 сдвигов. Положительный сдвиг означает «значение в будущем»: детекция офлайн, весь ряд известен, и именно будущее (дебит через неделю после скачка) говорит, был ли эффект. Пустые края (первые/последние дни) заполняются нулями;
3. средние за 7 дней назад (`_mean_past7`, включая сегодня) и за 7 дней вперёд (`_mean_next7`) — сглаживают шум суточных замеров;
4. `f_diff_max_pm3` — максимум `f_diff` в центрированном окне 7 дней (±3): «рядом был скачок частоты», чтобы дату начала можно было искать рядом со скачком.

Для Transformer всё это не нужно: свёртка и внимание сами смотрят на соседей. Для деревьев и логистической регрессии каждая строка независима, поэтому контекст кладётся в строку.

Пример

Фактическая раскладка столбцов (индексы в матрице):

Столбцы	Что
0–14	основные каналы
15–86	8 блоков по 9 каналов: сдвиги −14, −7, −3, −1, +1, +3, +7, +14
87–95	`_mean_past7` для 9 каналов
96–104	`_mean_next7` для 9 каналов
105	`f_diff_max_pm3`

```
t = fb.tabular(make_well())      # (120, 106)
t[35, 72]    # 0.105: liq_vs_30d через 7 дней (день 42, дебит 110 вместо 100)
t[40, fb.SEQUENCE_CHANNELS.index("f_diff")]   # 0.4 в день скачка
t[:, 105]    # 0.4 в днях 37…43 (скачок в день 40, окно ±3), иначе 0
```

Для 698 реальных скважин (201 918 дней) получается `(1587, 106)` на скважине 8822. Обучающих строк после `training_mask(45)` по всем скважинам — 14 247, из них 5 213 положительных.

метод FeatureBuilder._channels

```
def _channels(self, well: WellSeries) -> pd.DataFrame
```

```
92      def _channels(self, well: WellSeries) -> pd.DataFrame:
93          frame = well.frame
94          working = (frame["state"] > 0) & (frame["frequency"] > self.MIN_WORKING_FREQ)
95          frequency = frame["frequency"].where(working).ffill().bfill()
96          reference = frequency[working].median() if working.any() else np.nan
97          reference = reference if reference and not np.isnan(reference) else 1.0
98          f_rel = (frequency / reference - 1).fillna(0.0)
99
100         out = pd.DataFrame(index=frame.index)
101         out["working"] = working.astype(float)
102         out["f_rel"] = f_rel
103         out["f_diff"] = f_rel.diff().fillna(0.0) * 10  # 1 Гц при 50 Гц -> 0.2
104         out["f_above_min30"] = (f_rel - f_rel.rolling(self.WINDOW, min_periods=1).min()) * 10
105         out["f_below_max30"] = (f_rel - f_rel.rolling(self.WINDOW, min_periods=1).max()) * 10
106         out["liq_vs_30d"] = self._relative_to_past(frame["debit_liquid"], working)
107         out["oil_vs_30d"] = self._relative_to_past(frame["debit_oil"], working)
108         out["wc_vs_30d"] = self._difference_to_past(frame["watercut"], working) / 10
```

```
109         out["pzab_vs_30d"] = self._relative_to_past(frame["pressure_zab"], working)
110         out = out.join(self._since_last_step(frame, frequency, working))
111         out["liq_missing"] = frame["debit_liquid"].isna().astype(float)
112         return out.fillna(0.0).clip(-CLIP, CLIP)
```

Главный метод: считает DataFrame со всеми 15 каналами по таблице `well.frame` (колонки `state`, `frequency`, `debit_liquid`, `debit_oil`, `watercut`, `pressure_zab`). Шаги:

1. `working` = `state` > 0 и частота > 10 Гц.
2. Частота «лечится»: на остановках значение заменяется ближайшим рабочим (`where(working).ffill().bfill()`), иначе провал до 0 Гц превращался бы в огромный ложный «скачок» при запуске насоса.
3. Опорная частота (`reference`) — медиана по рабочим дням; `f_rel = f / reference − 1`. Если рабочих дней нет, опора = 1 (всё нулевое, но без деления на ноль).
4. `f_diff`, `f_above_min30`, `f_below_max30` — из `f_rel`, масштаб ×10, чтобы типичный подъём на 1 Гц давал ~0,2, а не 0,02.
5. Относительные изменения дебитов, давления и обводнённости — `_relative_to_past`, `_difference_to_past`.
6. Признаки от последнего скачка — `_since_last_step`.
7. Итог: `fillna(0)` и `clip(−3; 3)`. Ограничение нужно, потому что в данных бывают выбросы (дебит вырос в 20 раз из-за ошибки замера), которые разрушили бы обучение.

Нули вместо NaN неоднозначны (0 — и «ничего не изменилось», и «нет данных»), поэтому отдельно есть `working` и `liq_missing`.

Пример

Синтетическая скважина: частота 48 Гц, рабочая медиана `reference` = 50, значит до скачка `f_rel` = 48/50 − 1 = −0,04 (первый день `s[0, 1]` = −0,04), после — 0.

метод FeatureBuilder._relative_to_past

```
def _relative_to_past(self, series: pd.Series, working: pd.Series) -> pd.Series
```

```
114     def _relative_to_past(self, series: pd.Series, working: pd.Series) -> pd.Series:
115         past = series.where(working).shift(1).rolling(self.WINDOW, min_periods=3).median()
116         return (series / past.replace(0, np.nan) - 1).where(working)
```

Относительное изменение показателя к его «недавнему уровню»: `value / median(прошлые 30 дней) − 1`, только для рабочих дней. Параметры: `series` — суточный ряд (дебит жидкости, нефти или забойное давление), `working` — маска работы насоса. Результат — Series с NaN на неработающих днях и там, где мало истории.

Детали, которые важны:

- `shift(1)` — окно **прошлого**, сегодняшний день в базу не входит, иначе эффект «размазывал» бы собственную базу;
- остановленные дни исключены из базы (`series.where(working)`), иначе нули остановки занижали бы медиану;
- нужно минимум 3 значения (`min_periods=3`), медиана — потому что устойчива к выбросам;
- ноль в базе заменяется NaN, чтобы не делить на ноль.

Пример

Скважина 8822: 29.07.2022 дебит жидкости 157 м³/сут, медиана за прошлые 30 дней тоже 157 → 0. В день начала 30.07.2022 — 197 м³/сут → 197/157 − 1 ≈ 0,25 (это `liq_vs_30d` = 0,25 на реальных данных).

метод FeatureBuilder._difference_to_past

`def _difference_to_past(self, series: pd.Series, working: pd.Series) -> pd.Series`

```
118     def _difference_to_past(self, series: pd.Series, working: pd.Series) -> pd.Series:
119         past = series.where(working).shift(1).rolling(self.WINDOW, min_periods=3).mean()
120         return (series - past).where(working)
```

То же, что `_relative_to_past`, но разностью и по **среднему**, а не медиане: `value − mean(прошлые 30 дней)`. Используется для обводнённости (`watercut`, %): она уже сама доля, относительное изменение от 5 % до 6 % выглядело бы как +20 %, а разность в процентных пунктах честнее. Деление на 10 в `_channels` приводит единицы к масштабу остальных каналов (рост обводнённости на 10 п.п. → 1,0).

Пример

Обводнённость 90 % весь месяц и 92 % сегодня: разность 2 п.п. → после деления на 10 канал `wc_vs_30d` = 0,2. В синтетической скважине обводнённость константа 90 → канал везде 0.

метод FeatureBuilder._since_last_step

`def _since_last_step(self, frame: pd.DataFrame, frequency: pd.Series, working: pd.Series) -> pd.DataFrame`

```
122     def _since_last_step(self, frame: pd.DataFrame, frequency: pd.Series, working: pd.Series) ->
    pd.DataFrame:
123         """Признаки относительно последнего скачка частоты вверх (как у детектора на правилах)."""
124         smoothed = frequency.rolling(3, center=True, min_periods=1).median()
125         is_step = (smoothed.diff() >= self.MIN_STEP) & working
126         positions = np.flatnonzero(is_step.to_numpy())
127 
128         n = len(frame)
129         result = {
130             name: np.zeros(n)
131             for name in ("days_since_step", "step_size", "liq_vs_step", "oil_vs_step", "wc_vs_step")
132         }
133         result["days_since_step"][:] = 1.0  # «давно не было скачка»
134         liquid, oil = frame["debit_liquid"].to_numpy(), frame["debit_oil"].to_numpy()
135         watercut, values = frame["watercut"].to_numpy(), smoothed.to_numpy()
136         boundaries = list(positions) + [n]
137         for step, next_step in zip(boundaries[:-1], boundaries[1:], strict=True):
138             before = max(step - 1, 0)
139             span = slice(step, next_step)
140             days = np.arange(next_step - step)
141             result["days_since_step"][span] = np.minimum(days, 120) / 120
142             result["step_size"][span] = (values[step] - values[before]) / max(values[before], 1.0) *
    10
143             result["liq_vs_step"][span] = _ratio(liquid[span], liquid[before])
144             result["oil_vs_step"][span] = _ratio(oil[span], oil[before])
145             result["wc_vs_step"][span] = (watercut[span] - watercut[before]) / 10
146         return pd.DataFrame(result, index=frame.index)
```

Признаки «относительно последнего подъёма частоты». Это то, как рассуждает детектор на правилах: ищем скачок и смотрим, что стало с дебитом после него.

Алгоритм:

1. Частота сглаживается скользящей медианой по 3 дням (убирает одиночные выбросы).
2. Скачок — день, где сглаженная частота выросла минимум на `MIN_STEP` = 0,1 Гц и насос работает.

3. Ряд делится на отрезки «от скачка до следующего скачка». Для каждого отрезка базой служит значение **за день до** скачка (`before`).
4. В отрезке: `days_since_step` = min(дни от скачка, 120) / 120; `step_size` = (f после − f до) / f до × 10; `liq_vs_step`, `oil_vs_step` — относительное изменение дебита к базе (через `_ratio`); `wc_vs_step` — разность обводнённости, /10.
5. До первого скачка `days_since_step` = 1 («давно не было»), остальные нули.

Возвращает DataFrame с пятью колонками на индексе `frame`.

Пример

Синтетическая скважина (скачок 48 → 50 Гц в день 40): `step_size` = 0,42 с дня 40, `days_since_step` = 0 в день 40, 1/120 ≈ 0,01 в день 41, 2/120 ≈ 0,02 в день 42; а `liq_vs_step` = 0,10 с дня 42, когда дебит стал 110 вместо 100. Скважина 8822 в день разметки: `days_since_step` = 0,02 (около 3 дней после последнего скачка).

функция _ratio

```
def _ratio(values: np.ndarray, base: float) -> np.ndarray
```

```
149 def _ratio(values: np.ndarray, base: float) -> np.ndarray:
150     if base is None or np.isnan(base) or base == 0:
151         return np.zeros_like(values)
152     return values / base - 1
```

Относительное изменение массива к базовому значению: `values / base - 1`. Если база не задана, NaN или 0, возвращает нули — когда уровень «до скачка» неизвестен, признак не должен выдумывать эффект.

Пример

```
_ratio(np.array([110.0, 120.0]), 100.0)   # array([0.1, 0.2])
_ratio(np.array([110.0]), 0.0)            # array([0.])  — делить нельзя
_ratio(np.array([110.0]), float("nan"))   # array([0.])
```

models/__init__.py

Пакет `models` собирает все модели в одном месте и скрывает различия между ними: с точки зрения остального кода (`CrossValidator`, `cli`) любая модель — это `EventSegmenter` с методами `fit`, `predict_proba`, `save`, `load`.

В цепочке «Excel → скважины → признаки → модель → интервалы» пакет — четвёртое звено: берёт признаки из `FeatureBuilder`, учится на днях из `training_mask`, возвращает вероятность «день внутри эффекта». Даты начала и окончания получает уже `SegmentDecoder`, одинаково для всех моделей.

Пакет содержит: `base.py` (общий интерфейс), `classic.py` (4 модели scikit-learn), `transformer.py` (своя сеть на PyTorch). Новая модель — это наследник `EventSegmenter` и одна строка в `model_registry`.

Код модуля вне классов и функций

```python
from collections.abc import Callable

from .base import EventSegmenter
from .classic import SklearnSegmenter, classic_models
from .transformer import TimeSeriesTransformer, TransformerSegmenter

...
__all__ = [
    "EventSegmenter",
    "SklearnSegmenter",
    "TimeSeriesTransformer",
    "TransformerSegmenter",
    "classic_models",
    "load_model",
    "model_registry",
]
```

функция model_registry

```
def model_registry(seed: int = 0) -> dict[str, Callable[[], EventSegmenter]]
```

```python
def model_registry(seed: int = 0) -> dict[str, Callable[[], EventSegmenter]]:
    """Все доступные модели: имя -> фабрика необученной модели."""
    registry: dict[str, Callable[[], EventSegmenter]] = {"transformer": TransformerSegmenter}
    registry.update(classic_models(seed))
    return registry
```

Возвращает словарь «имя модели → фабрика необученной модели». Имена — те, что принимает `--models` в командной строке.

Параметр `seed` (по умолчанию 0) попадает в `random_state` моделей scikit-learn. У Transformer зерно хранится в `TrainingConfig.seed`, поэтому фабрика — сам класс `TransformerSegmenter`.

Фабрика, а не готовый объект, нужна потому, что `CrossValidator` на каждом из 5 фолдов обучает **новую** модель с нуля: переиспользование обученного объекта дало бы утечку между фолдами.

Пример

```python
reg = model_registry()
list(reg)           # ['transformer', 'logreg', 'random_forest', 'extra_trees', 'gboost']
model = reg["gboost"]()          # необученный SklearnSegmenter
model.title                      # 'Градиентный бустинг (HistGB)'
```

В командной строке: `python -m gtm_detector.ml_freq compare --models gboost extra_trees`.

функция load_model

```
def load_model(path) -> EventSegmenter
```

```
17  def load_model(path) -> EventSegmenter:
18      """Загрузить сохранённую модель по расширению файла."""
19      return TransformerSegmenter.load(path) if str(path).endswith(".pt") else
    SklearnSegmenter.load(path)
```

Загружает сохранённую модель, определяя тип по расширению файла: `.pt` — `TransformerSegmenter.load`, всё остальное (`.joblib`) — `SklearnSegmenter.load`. Нужна команде `predict`, которой всё равно, какая модель лежит в `weights/`.

Параметр `path` — путь к файлу (`str` или `Path`). Результат — обученная модель, готовая к `predict_proba`.

Пример

```
model = load_model("gtm_detector/ml_freq/weights/gboost.joblib")   # SklearnSegmenter
model = load_model("gtm_detector/ml_freq/weights/transformer.pt")  # TransformerSegmenter
```

Если для Transformer набор каналов `SEQUENCE_CHANNELS` изменился с момента обучения, `load` выбросит `ValueError("Набор признаков изменился — переобучите модель")`.

models/base.py

Общий интерфейс всех моделей. Он фиксирует единый контракт: «получи размеченные скважины — научись; получи скважину — верни по вероятности на каждый день». Поэтому `CrossValidator`, `SegmentDecoder` и командная строка не знают, что внутри — Transformer или случайный лес, а сравнение моделей честное: одни фолды, один декодер, одни метрики.

Классы-наследники: `SklearnSegmenter` (`classic.py`) и `TransformerSegmenter` (`transformer.py`).

Код модуля вне классов и функций

```python
from __future__ import annotations

from abc import ABC, abstractmethod
from collections.abc import Sequence
from pathlib import Path

import numpy as np

from ..config import CONTEXT_DAYS
from ..data import WellSeries
from ..features import FeatureBuilder
```

класс EventSegmenter

```
class EventSegmenter(ABC)
```

```python
class EventSegmenter(ABC):
    """Модель разметки ряда по дням.

    ``fit`` учится на размеченных скважинах, ``predict_proba`` возвращает для
    каждого дня скважины вероятность того, что идёт эффект мероприятия.
    Даты начала и окончания из вероятностей получает
    :class:`~gtm_detector.ml_freq.evaluation.SegmentDecoder` — одинаково для всех моделей.
    """

    name: str = "base"
    title: str = "Базовая модель"

    ...

    ...

    ...

    ...

    ...
```

Абстрактный класс (`ABC`) модели разметки ряда по дням. Четыре абстрактных метода — `fit`, `predict_proba`, `save`, `load` — и общие атрибуты:

- `name`, `title` — машинное имя (`gboost`) и русское название для таблиц и графиков;
- `features` — `FeatureBuilder`, общий для всех моделей;
- `context_days` — радиус маски обучения (по умолчанию `CONTEXT_DAYS` = 45).

Контракт `predict_proba`: массив `float32/float64` длины `len(well.frame)`, значения в [0; 1] — вероятность, что именно в этот день идёт эффект мероприятия.

Пример

Создать новую модель — значит написать наследника и добавить его в `model_registry`:

```
class MyModel(EventSegmenter):
    name, title = "my", "Моя модель"
    def fit(self, wells): ...
    def predict_proba(self, well): ...
    def save(self, path): ...
    @classmethod
    def load(cls, path): ...
```

Если забыть один из методов, `MyModel()` выбросит `TypeError: Can't instantiate abstract class`.

метод EventSegmenter.__init__

```
def __init__( self, features: FeatureBuilder | None = None, context_days: int | None = CONTEXT_DAYS ) -> None
```

```
28      def __init__(
29          self, features: FeatureBuilder | None = None, context_days: int | None = CONTEXT_DAYS
30      ) -> None:
31          self.features = features or FeatureBuilder()
32          self.context_days = context_days  # см. WellSeries.training_mask
```

Создаёт модель с набором признаков и радиусом маски обучения.

Параметры: `features` — готовый `FeatureBuilder` (по умолчанию создаётся новый, он без состояния); `context_days` — сколько дней до начала и после окончания размеченного мероприятия используется в обучении (по умолчанию 45; `None` — все дни скважины).

Почему 45: в разметке есть не все реальные подъёмы частоты, поэтому «день вне разметки» не значит «эффекта нет». Надёжны только окрестности размеченных мероприятий; их и берёт `WellSeries.training_mask`. После мероприятий «ПРОДОЛЖАЕТСЯ ЭФФЕКТ» (`censored`) дни после даты окончания не берутся совсем.

Пример

```
model = SklearnSegmenter("gboost", "Градиентный бустинг", factory)   # context_days = 45
```

Для скважины 8822 (начало 30.07.2022, окончание 05.09.2022) из 1587 дней ряда обучению достанется 128 (от 15.06.2022 до 20.10.2022), а целевыми будут 38 дней эффекта.

метод EventSegmenter.fit

```
def fit(self, wells: Sequence[WellSeries]) -> EventSegmenter: ...
```

```
34      @abstractmethod
35      def fit(self, wells: Sequence[WellSeries]) -> EventSegmenter: ...
```

Абстрактный метод: обучить модель на размеченных скважинах. Принимает последовательность `WellSeries` (только обучающие скважины фолда — разбиение делает `CrossValidator.folds`), возвращает `self`, чтобы можно было писать `model.fit(train).predict_proba(well)`.

Обязанности реализации: брать признаки только из `self.features`, учиться только на днях `well.training_mask(self.context_days)`, цель — `well.target()`.

Пример

```
model = reg["extra_trees"]().fit(train_wells)    # train_wells: ~110 скважин фолда
```

метод EventSegmenter.predict_proba

```
def predict_proba(self, well: WellSeries) -> np.ndarray: ...
```

```
37      @abstractmethod
38      def predict_proba(self, well: WellSeries) -> np.ndarray: ...
```

Абстрактный метод: вероятность «день внутри эффекта» для каждого дня одной скважины. Принимает `WellSeries` (разметка не нужна), возвращает одномерный массив длины `len(well.frame)`. Вероятности потом сглаживает и режет по порогу `SegmentDecoder` (порог 0,5, окно сглаживания 5 дней).

Пример

Для скважины 8822 `predict_proba` возвращает массив из 1587 чисел; окно дней около 30.07.2022 — ближе к 1, остальные — близко к 0.

метод EventSegmenter.save

```
def save(self, path: Path) -> None: ...
```

```
40      @abstractmethod
41      def save(self, path: Path) -> None: ...
```

Абстрактный метод: записать обученную модель в файл `path` (родительские папки создаются). Формат выбирает наследник: `.joblib` у scikit-learn, `.pt` у Transformer (см. `file_suffix`).

Пример

```
model.save(Path("gtm_detector/ml_freq/weights") / f"{model.name}{model.file_suffix}")
```

метод класса EventSegmenter.load

```
def load(cls, path: Path) -> EventSegmenter: ...
```

```
43      @classmethod
44      @abstractmethod
45      def load(cls, path: Path) -> EventSegmenter: ...
```

Абстрактный классовый метод: восстановить обученную модель из файла. Возвращает готовый к `predict_proba` экземпляр без повторного обучения.

Пример

```
model = TransformerSegmenter.load(Path("gtm_detector/ml_freq/weights/transformer.pt"))
```

свойство EventSegmenter.file_suffix

```
def file_suffix(self) -> str
```

```
47      @property
48      def file_suffix(self) -> str:
49          return ".joblib"
```

Расширение файла для сохранения: по умолчанию `.joblib`. Transformer переопределяет его на `.pt`. Благодаря свойству код сохранения пишет `name + file_suffix`, не зная типа модели.

Пример

```
SklearnSegmenter(...).file_suffix    # '.joblib'
TransformerSegmenter().file_suffix   # '.pt'
```

models/classic.py

Четыре готовые модели scikit-learn, обёрнутые под общий интерфейс `EventSegmenter`: логистическая регрессия, случайный лес, Extra Trees и градиентный бустинг (`HistGradientBoostingClassifier`). Каждая решает задачу «строка = день скважины, 106 признаков из `FeatureBuilder.tabular`, ответ 0/1 — день внутри эффекта». Контекст соседних дней уже зашит в признаки (сдвиги ±1…14 дней, средние за неделю), поэтому моделям не нужны ни последовательности, ни внимание.

Классические модели оказались лучшими в проекте (результаты `compare` на данных 2020–2026, 424 мероприятия на 369 скважинах, 5 фолдов по скважинам, зерно 0):

Модель	полнота	точность	начало ±3 дн	ROC-AUC	обучение, с
Случайный лес	0,679	0,079	0,889	0,862	152,3
Extra Trees	0,630	0,073	0,899	0,884	109,8
Градиентный бустинг (HistGB)	0,736	0,075	0,891	0,871	214,4
Transformer	0,465	0,044	0,802	0,751	3132
Логистическая регрессия	0,608	0,036	0,764	0,818	48,8

Точность занижена намеренно: разметка неполная, многие настоящие подъёмы частоты в ней отсутствуют.

Код модуля вне классов и функций

```
3  from __future__ import annotations
4
5  from collections.abc import Callable, Sequence
6  from pathlib import Path
7
8  import joblib
9  import numpy as np
10 from sklearn.base import ClassifierMixin
11 from sklearn.ensemble import ExtraTreesClassifier, HistGradientBoostingClassifier,
   RandomForestClassifier
12 from sklearn.linear_model import LogisticRegression
13 from sklearn.pipeline import make_pipeline
14 from sklearn.preprocessing import StandardScaler
15
16 from ..data import WellSeries
17 from ..features import FeatureBuilder
18 from .base import EventSegmenter
```

класс SklearnSegmenter

`class SklearnSegmenter(EventSegmenter)`

```
21 class SklearnSegmenter(EventSegmenter):
22     """Обёртка над классификатором scikit-learn: строка = день скважины."""
23
   ...
36
   ...
44
   ...
49
   ...
53
```

Обёртка над любым классификатором scikit-learn с методами `predict_proba` (пайплайн, лес, бустинг). Хранит не готовую модель, а **фабрику** `estimator_factory`: на каждом `fit` создаётся новый необученный

классификатор. Это важно для перекрёстной проверки — иначе веса одного фолда протекли бы в следующий.

Поля: `name` / `title` (как в `model_registry`), `estimator` — обученный классификатор или `None`.

Пример

```
reg = classic_models()
model = reg["random_forest"]()      # SklearnSegmenter, estimator is None
model.title                          # 'Случайный лес'
model.fit(train_wells)               # estimator -> RandomForestClassifier(...)
```

метод SklearnSegmenter.__init__

```
def __init__( self, name: str, title: str, estimator_factory: Callable[[], ClassifierMixin], features: FeatureBuilder | None = None, ) -> None
```

```
    def __init__(
        self,
        name: str,
        title: str,
        estimator_factory: Callable[[], ClassifierMixin],
        features: FeatureBuilder | None = None,
    ) -> None:
        super().__init__(features)
        self.name = name
        self.title = title
        self.estimator_factory = estimator_factory
        self.estimator: ClassifierMixin | None = None
```

Принимает: `name` — короткое имя (`gboost`), `title` — русское название для таблиц, `estimator_factory` — функцию без аргументов, возвращающую необученный классификатор, `features` — `FeatureBuilder` (по умолчанию новый). `context_days` берётся по умолчанию (45) из `EventSegmenter`.

Пример

```
SklearnSegmenter("logreg", "Логистическая регрессия", lambda: LogisticRegression())
```

метод SklearnSegmenter.fit

```
def fit(self, wells: Sequence[WellSeries]) -> SklearnSegmenter
```

```
    def fit(self, wells: Sequence[WellSeries]) -> SklearnSegmenter:
        masks = [well.training_mask(self.context_days) for well in wells]
        x = np.vstack([self.features.tabular(well)[mask] for well, mask in zip(wells, masks, strict=True)])
        y = np.concatenate([well.target()[mask] for well, mask in zip(wells, masks, strict=True)]).astype(int)
        self.estimator = self.estimator_factory()
        self.estimator.fit(x, y)
        return self
```

Обучает классификатор на днях из маски обучения.

1. Для каждой скважины берётся `training_mask(45)`: ±45 дней вокруг размеченного мероприятия (у `censored`-мероприятия «ПРОДОЛЖАЕТСЯ ЭФФЕКТ» дни после даты окончания в маску не входят: что там происходило, неизвестно).
2. Матрица признаков `tabular(well)[mask]` — форма `(дней_в_маске, 106)`; цель `target()[mask]` — 0/1.
3. Строки всех скважин складываются (`vstack`), фабрика создаёт классификатор, вызывается `fit(x, y)`.

Баланс классов: положительных дней заметно меньше (по всем размеченным скважинам 5 213 из 14 247 строк маски — 37 %), поэтому в моделях стоит `class_weight` (`balanced`).

Возвращает `self`. Зерно случайности не здесь, а в `random_state` фабрики (`classic_models(seed)`).

Пример

```
model = classic_models()["gboost"]().fit(train_wells)
```

Если взять все скважины сразу, маска даёт 14 247 строк × 106 признаков (из 201 918 дней рядов 698 скважин; остальное — неразмеченные дни); обучение `gboost` на пяти фолдах заняло 112 секунд, `random_forest` — 42.

метод SklearnSegmenter.predict_proba

```
def predict_proba(self, well: WellSeries) -> np.ndarray
```

```
45      def predict_proba(self, well: WellSeries) -> np.ndarray:
46          if self.estimator is None:
47              raise RuntimeError(f"Модель {self.name} не обучена")
48          return self.estimator.predict_proba(self.features.tabular(well))[:, 1]
```

Возвращает вероятность класса «эффект» для каждого дня скважины: `estimator.predict_proba(tabular(well))[:, 1]`, массив длины `len(well.frame)`. Предсказание идёт по **всем** дням (маска нужна только на обучении), чтобы декодер видел весь ряд. Если модель не обучена, выбрасывается `RuntimeError("Модель gboost не обучена")`.

Пример

```
p = model.predict_proba(well)      # well — 1587 дней
p.shape                            # (1587,)
p[well.target() == 1].mean()       # заметно выше, чем вне эффекта
```

метод SklearnSegmenter.save

```
def save(self, path: Path) -> None
```

```
50      def save(self, path: Path) -> None:
51          path.parent.mkdir(parents=True, exist_ok=True)
52          joblib.dump({"name": self.name, "title": self.title, "estimator": self.estimator}, path)
```

Сохраняет через `joblib.dump` словарь `{name, title, estimator}` — модель вместе с названием, чтобы `load` не нуждался в реестре. Расширение `.joblib`, папки создаются.

Пример

```
model.save(Path("gtm_detector/ml_freq/weights/gboost.joblib"))
```

метод класса SklearnSegmenter.load

```
def load(cls, path: Path) -> SklearnSegmenter
```

```
54      @classmethod
55      def load(cls, path: Path) -> SklearnSegmenter:
56          state = joblib.load(path)
57          model = cls(state["name"], state["title"], estimator_factory=lambda: state["estimator"])
58          model.estimator = state["estimator"]
59          return model
```

Классовый метод: читает файл `.joblib` и собирает `SklearnSegmenter` с сохранённым классификатором. Фабрика в загруженной модели возвращает тот же **уже обученный** объект (`lambda: state["estimator"]`), поэтому повторный `fit` на загруженной модели переобучит именно его; для новой проверки создавайте модель через `model_registry`.

Пример

```
model = SklearnSegmenter.load(Path("gtm_detector/ml_freq/weights/gboost.joblib"))
model.name    # 'gboost'
```

функция classic_models

```
def classic_models(seed: int = 0) -> dict[str, Callable[[], SklearnSegmenter]]
```

```
62  def classic_models(seed: int = 0) -> dict[str, Callable[[], SklearnSegmenter]]:
63      """Набор готовых моделей для сравнения: имя -> фабрика необученной модели."""
64
65      def logistic() -> ClassifierMixin:
66          return make_pipeline(
67              StandardScaler(), LogisticRegression(max_iter=2000, class_weight="balanced", C=0.5)
68          )
69
70      def random_forest() -> ClassifierMixin:
71          return RandomForestClassifier(
72              n_estimators=300,
73              min_samples_leaf=5,
74              class_weight="balanced_subsample",
75              n_jobs=-1,
76              random_state=seed,
77          )
78
79      def extra_trees() -> ClassifierMixin:
80          return ExtraTreesClassifier(
81              n_estimators=300,
82              min_samples_leaf=5,
83              class_weight="balanced_subsample",
84              n_jobs=-1,
85              random_state=seed,
86          )
87
88      def gradient_boosting() -> ClassifierMixin:
89          return HistGradientBoostingClassifier(
90              max_iter=300,
91              learning_rate=0.05,
92              max_leaf_nodes=31,
93              l2_regularization=1.0,
94              class_weight="balanced",
95              random_state=seed,
96          )
97
98      specs = {
99          "logreg": ("Логистическая регрессия", logistic),
100         "random_forest": ("Случайный лес", random_forest),
101         "extra_trees": ("Extra Trees", extra_trees),
102         "gboost": ("Градиентный бустинг (HistGB)", gradient_boosting),
103     }
104     return {
105         name: (lambda name=name, title=title, factory=factory: SklearnSegmenter(name, title, factory))
106         for name, (title, factory) in specs.items()
107     }
```

Возвращает словарь «имя → фабрика» четырёх моделей. Гиперпараметры подобраны консервативно под малые данные (в `config.py` они не вынесены, остались в этой функции).

Ключ	Модель	Параметры
`logreg`	`StandardScaler` + `LogisticRegression`	`C=0.5`, `class_weight="balanced"`, `max_iter=2000`
`random_forest`	`RandomForestClassifier`	300 деревьев, `min_samples_leaf=5`, `balanced_subsample`
`extra_trees`	`ExtraTreesClassifier`	то же, но случайные пороги
`gboost`	`HistGradientBoostingClassifier`	300 итераций, `learning_rate=0.05`, 31 лист, `l2_regularization=1.0`, `balanced`

`min_samples_leaf=5` и L2-регуляризация сдерживают переобучение на 120 скважинах. Логистическая регрессия единственная масштабирует признаки (линейной модели важны масштабы; деревьям — нет). Параметр `seed` (по умолчанию 0) идёт в `random_state` всех моделей, поэтому результаты воспроизводимы.

Тонкость: лямбда `lambda name=name, title=title, factory=factory: ...` связывает переменные цикла сразу; без значений по умолчанию все четыре фабрики создавали бы последнюю модель (`gboost`).

Пример

```
classic_models(seed=0).keys()   # dict_keys(['logreg', 'random_forest', 'extra_trees', 'gboost'])
```

Запуск сравнения: `python -m gtm_detector.ml_freq compare --models gboost extra_trees` (около 2 минут).

models/transformer.py

Собственная модель проекта: Transformer-энкодер для суточных рядов скважины на PyTorch. Схема:

```
15 каналов дня → Conv1d (ступеньки за несколько дней) → + позиционное кодирование
              → 2 × TransformerEncoderLayer (внимание по всему ряду)
              → LayerNorm + Linear → логит «день внутри эффекта»
```

Свёртка ловит форму скачка частоты и отклика дебита за несколько дней, внимание связывает скачок с окончанием эффекта через недели. Модель **двунаправленная** (каждый день видит и прошлое, и будущее): детекция офлайн, весь ряд известен.

Размеры модели задаёт `TransformerConfig`: `d_model=32`, 4 головы, 2 слоя, `dim_feedforward=64`, `dropout=0.2`, ядро свёртки 5 — всего **19 617 параметров**. Это намеренно мало: на ~100 обучающих скважинах крупная модель переобучается (при `d_model=64` и 3 слоях точное попадание в день начала 16 % против 24 %). Обучение задаёт `TrainingConfig`: 80 эпох, батч 16, AdamW (`lr=1e-3`, `weight_decay=1e-4`), окна по 128 дней, 4 окна на скважину.

Результат в сравнении `compare` (данные 2020–2026, 424 мероприятия, зерно 0): полнота 0,465, точность 0,044, начало ±3 дня 0,802 (у деревьев 0,89–0,90), окончание ±7 дней 0,392 и ROC-AUC 0,751 — хуже деревьев (Extra Trees: 0,701 и 0,884). Обучение на 5 фолдах — 3132 с на процессоре против 49–214 с у scikit-learn. На прежних данных (135 мероприятий) начало ±3 дня у Transformer было лучшим (0,831), теперь деревья его обогнали. Для Transformer разброс между зёрнами случайности сравним с эффектом, поэтому выводы «лучше/хуже» делаются по нескольким зёрнам.

Код модуля вне классов и функций

```python
from __future__ import annotations

import logging
import math
from collections.abc import Sequence
from dataclasses import asdict
from pathlib import Path

import numpy as np
import torch
from torch import nn

from ..config import TrainingConfig, TransformerConfig
from ..data import WellSeries
from ..features import FeatureBuilder
from .base import EventSegmenter

logger = logging.getLogger(__name__)
```

класс PositionalEncoding

`class PositionalEncoding(nn.Module)`

```python
class PositionalEncoding(nn.Module):
    """Синусоидальное позиционное кодирование (Vaswani et al., 2017)."""

    ...
```

Модуль PyTorch: добавляет к каждому дню синусоидальный «номер позиции» (Vaswani et al., 2017). Без него внимание не отличало бы порядок дней: ряд из тех же значений в перемешанном порядке выглядел бы так

же.

Вход и выход — тензор `(batch, дни, d_model)`; таблица кодирования не обучается и не сохраняется в весах (`persistent=False`) — её можно пересчитать.

Пример

При `d_model=32`, `max_length=4096` в таблице 4096 × 32 чисел. Сумма для окна из 128 дней: `x + encoding[:128]`.

метод PositionalEncoding.__init__

```
def __init__(self, d_model: int, max_length: int) -> None
```

```
37      def __init__(self, d_model: int, max_length: int) -> None:
38          super().__init__()
39          position = torch.arange(max_length).unsqueeze(1)
40          scale = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
41          encoding = torch.zeros(max_length, d_model)
42          encoding[:, 0::2] = torch.sin(position * scale)
43          encoding[:, 1::2] = torch.cos(position * scale)
44          self.register_buffer("encoding", encoding, persistent=False)
```

Строит таблицу `encoding` формы `(max_length, d_model)`: чётные столбцы — `sin(pos · k)`, нечётные — `cos(pos · k)` с частотами `k = exp(−ln(10000) · i / d_model)`. Параметры: `d_model` — размерность скрытого слоя (32), `max_length` — максимальная длина ряда (4096 дней, ≈11 лет; самая длинная скважина данных — 2257 дней).

Пример

```
pe = PositionalEncoding(32, 4096)
pe.encoding.shape      # torch.Size([4096, 32])
```

метод PositionalEncoding.forward

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

```
46      def forward(self, x: torch.Tensor) -> torch.Tensor:
47          return x + self.encoding[: x.size(1)]
```

Принимает `x` формы `(batch, L, d_model)` и возвращает `x + encoding[:L]` — таблица подрезается по длине ряда и складывается с признаками по всем сериям батча (broadcast). Если `L > max_length`, возникнет ошибка формы: ряды длиннее 4096 дней в `predict_proba` не поддерживаются.

Пример

```
x = torch.zeros(2, 100, 32)
pe(x).shape            # torch.Size([2, 100, 32])
```

класс TimeSeriesTransformer

```
class TimeSeriesTransformer(nn.Module)
```

```
50  class TimeSeriesTransformer(nn.Module):
51      """Сеть: (batch, дни, каналы) -> (batch, дни) логитов."""
52
    ...
69
```

Сама нейросеть: `(batch, дни, каналы) → (batch, дни)` логитов. Состав:

1. `embed` — `Conv1d(15 → 32, kernel=5, padding=2)`: превращает 15 каналов дня в 32 скрытых признака, учитывая ±2 соседних дня; `padding=2` сохраняет длину ряда;
2. `position` — `PositionalEncoding`;
3. `encoder` — 2 слоя `TransformerEncoderLayer` с 4 головами, `norm_first=True` (нормализация до внимания — стабильнее на маленьких данных), `dropout=0,2`;
4. `head` — `LayerNorm` + `Linear(32 → 1)`: по одному числу на день.

Выход — **логиты**, а не вероятности: функция потерь `BCEWithLogitsLoss` численно устойчивее, а в `predict_proba` логит превращается в вероятность сигмоидой.

Пример

```
net = TimeSeriesTransformer(15, TransformerConfig())
sum(p.numel() for p in net.parameters())     # 19617
net(torch.zeros(2, 100, 15)).shape           # torch.Size([2, 100])
```

Это проверяет `test_transformer_forward_shape`.

метод TimeSeriesTransformer.__init__

```
def __init__(self, n_channels: int, config: TransformerConfig) -> None
```

```
    def __init__(self, n_channels: int, config: TransformerConfig) -> None:
        super().__init__()
        self.embed = nn.Conv1d(
            n_channels, config.d_model, kernel_size=config.conv_kernel, padding=config.conv_kernel // 2
        )
        self.position = PositionalEncoding(config.d_model, config.max_length)
        layer = nn.TransformerEncoderLayer(
            d_model=config.d_model,
            nhead=config.n_heads,
            dim_feedforward=config.dim_feedforward,
            dropout=config.dropout,
            batch_first=True,
            norm_first=True,
        )
        self.encoder = nn.TransformerEncoder(layer, num_layers=config.n_layers, enable_nested_tensor=False)
        self.head = nn.Sequential(nn.LayerNorm(config.d_model), nn.Linear(config.d_model, 1))
```

Собирает слои. Параметры: `n_channels` — число входных каналов (`FeatureBuilder.sequence_size` = 15), `config` — `TransformerConfig` (`d_model`, `n_heads`, `n_layers`, `dim_feedforward`, `dropout`, `conv_kernel`, `max_length`). Число каналов передаётся отдельно, чтобы размер входа нельзя было рассинхронизировать с признаками. `enable_nested_tensor=False` отключает оптимизацию PyTorch, которая плохо сочетается с маской паддинга.

Пример

```
TimeSeriesTransformer(15, TransformerConfig(d_model=64, n_layers=3, dim_feedforward=128))
```

Этот вариант (крупнее) на данных проекта работал хуже, чем 32 / 2 слоя.

метод TimeSeriesTransformer.forward

```
def forward(self, x: torch.Tensor, padding_mask: torch.Tensor | None = None) -> torch.Tensor
```

```
    def forward(self, x: torch.Tensor, padding_mask: torch.Tensor | None = None) -> torch.Tensor:
        hidden = self.embed(x.transpose(1, 2)).transpose(1, 2)
        hidden = self.encoder(self.position(hidden), src_key_padding_mask=padding_mask)
        return self.head(hidden).squeeze(-1)
```

Прямой проход. Параметры: `x` — `(batch, L, 15)`, `float32`; `padding_mask` — булев `(batch, L)`, `True` там, где данных нет (хвост короткого окна, добитый нулями), либо `None`. Шаги: `x.transpose(1, 2)` (Conv1d ждёт `(batch, каналы, дни)`) → свёртка → обратно `(batch, L, 32)` → позиционное кодирование → энкодер (внимание игнорирует позиции паддинга) → голова → `squeeze(-1)`. Возвращает `(batch, L)` логитов.

Пример

Скважина из 1587 дней: `net(x)` с `x.shape == (1, 1587, 15)` даёт `(1, 1587)` без маски; при обучении батч из окон разной длины идёт с `padding_mask`.

класс TransformerSegmenter

`class TransformerSegmenter(EventSegmenter)`

```
class TransformerSegmenter(EventSegmenter):
    """Обучение и применение :class:`TimeSeriesTransformer`."""

    name = "transformer"
    title = "Transformer (своя модель, PyTorch)"
...

...

...

...

...

...

...
```

Обёртка сети в интерфейс `EventSegmenter`: готовит признаки и окна, обучает, предсказывает, сохраняет. Атрибуты: `name = "transformer"`, `title = "Transformer (своя модель, PyTorch)"`, `config` (архитектура), `training` (параметры обучения), `network` — сеть или `None` до `fit`.

Пример

```
model = TransformerSegmenter()          # network is None, n_parameters == 0
model.fit(train_wells)                  # network обучена, переведена в eval()
```

Быстрая проверка из командной строки: `python -m gtm_detector.ml_freq compare --models transformer --epochs 20`.

метод TransformerSegmenter.__init__

`def __init__( self, config: TransformerConfig | None = None, training: TrainingConfig | None = None, features: FeatureBuilder | None = None, ) -> None`

```
    def __init__(
        self,
```

```
        config: TransformerConfig | None = None,
        training: TrainingConfig | None = None,
        features: FeatureBuilder | None = None,
    ) -> None:
        super().__init__(features)
        self.config = config or TransformerConfig()
        self.training = training or TrainingConfig()
        self.network: TimeSeriesTransformer | None = None
```

Параметры: `config` — `TransformerConfig` (архитектура), `training` — `TrainingConfig` (эпохи, батч, зерно), `features` — `FeatureBuilder`. Все по умолчанию берутся из `config.py`. Контекст обучения `context_days` = 45 приходит из `EventSegmenter`.

Пример

```
TransformerSegmenter(training=TrainingConfig(epochs=20, seed=1))
```

Зерно `seed` — единственный способ менять случайность: оно идёт и в `torch.manual_seed`, и в `np.random.default_rng` для окон.

свойство TransformerSegmenter.file_suffix

```
def file_suffix(self) -> str
```

```
    @property
    def file_suffix(self) -> str:
        return ".pt"
```

Расширение файла весов — `.pt` (формат `torch.save`). Нужен, чтобы `load_model` и команда `train` выбирали верный загрузчик.

Пример

```
TransformerSegmenter().file_suffix   # '.pt'
```

свойство TransformerSegmenter.n_parameters

```
def n_parameters(self) -> int
```

```
    @property
    def n_parameters(self) -> int:
        return sum(p.numel() for p in self.network.parameters()) if self.network else 0
```

Число обучаемых параметров сети (`numel` по всем тензорам), 0 до обучения. Используется в отчётах `compare` и для контроля размера модели относительно данных.

Пример

```
TransformerSegmenter().n_parameters      # 0
# после fit при настройках по умолчанию: 19617
```

метод TransformerSegmenter.fit

```
def fit(self, wells: Sequence[WellSeries]) -> TransformerSegmenter
```

```
    def fit(self, wells: Sequence[WellSeries]) -> TransformerSegmenter:
        cfg = self.training
        torch.manual_seed(cfg.seed)
        rng = np.random.default_rng(cfg.seed)
        sequences = [self.features.sequence(well) for well in wells]
```

```
        targets = [well.target() for well in wells]
        masks = [well.training_mask(self.context_days) for well in wells]

        positives = float(sum(t[m].sum() for t, m in zip(targets, masks, strict=True)))
        negatives = float(sum(m.sum() for m in masks)) - positives
        loss_fn = nn.BCEWithLogitsLoss(
            pos_weight=torch.tensor(negatives / max(positives, 1.0)), reduction="none"
        )

        self.network = TimeSeriesTransformer(self.features.sequence_size, self.config)
        optimizer = torch.optim.AdamW(
            self.network.parameters(), lr=cfg.learning_rate, weight_decay=cfg.weight_decay
        )
        scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=cfg.epochs)

        self.network.train()
        for epoch in range(cfg.epochs):
            crops = self._sample_crops(sequences, targets, masks, rng)
            order = rng.permutation(len(crops))
            total = 0.0
            for begin in range(0, len(order), cfg.batch_size):
                x, y, padding, weight = self._collate(
                    [crops[i] for i in order[begin : begin + cfg.batch_size]]
                )
                logits = self.network(x, padding_mask=padding)
                loss = (loss_fn(logits, y) * weight).sum() / weight.sum().clamp(min=1.0)
                optimizer.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(self.network.parameters(), 1.0)
                optimizer.step()
                total += loss.item()
            scheduler.step()
            if (epoch + 1) % 20 == 0:
                logger.debug("эпоха %d/%d: loss=%.4f", epoch + 1, cfg.epochs, total)
        self.network.eval()
        return self
```

Обучает сеть на размеченных скважинах. Последовательность:

1. Зерно: `torch.manual_seed(seed)` и `np.random.default_rng(seed)` — воспроизводимость.
2. Для каждой скважины: `sequence()` (L × 15), `target()` (0/1) и `training_mask(45)`.
3. Вес положительного класса `pos_weight = отрицательные / положительные` по дням внутри масок. На всех размеченных скважинах: 14 247 дней в маске, 5 213 положительных, значит `pos_weight ≈ 1,73`. Почему так: эффект занимает меньшую часть окна, без веса сеть предпочла бы всегда отвечать «нет».
4. Сеть, AdamW и `CosineAnnealingLR` (скорость обучения плавно падает по косинусу за 80 эпох).
5. Каждую эпоху `_sample_crops` заново нарезает окна, они перемешиваются, батчи по 16 собираются `_collate`. Потеря = сумма `BCE × weight` / сумма весов, то есть **считается только по дням маски обучения**; дни после `censored`-мероприятий и дни вне окрестности размеченных событий в потери не входят, хотя сеть их «видит» как контекст.
6. Обрезка градиента (норма ≤ 1) стабилизирует обучение на маленьких батчах.
7. В конце `network.eval()` — отключается dropout.

Возвращает `self`. Каждые 20 эпох пишет в журнал `logger.debug` суммарную потерю.

Пример

Фолд из ~100 обучающих скважин с разметкой: 4 окна на скважину → ~400 окон за эпоху → 25 батчей по 16, 80 эпох ≈ 2000 шагов оптимизатора. На 5 фолдах — 1522 с на процессоре. Тест `test_transformer_fit_predict_save_load` проверяет цикл на 8 синтетических скважинах из `conftest.py`.

метод TransformerSegmenter.predict_proba

```
def predict_proba(self, well: WellSeries) -> np.ndarray
```

```
143     def predict_proba(self, well: WellSeries) -> np.ndarray:
144         if self.network is None:
145             raise RuntimeError("Transformer не обучен")
146         x = torch.from_numpy(self.features.sequence(well)).unsqueeze(0)
147         with torch.no_grad():
148             return torch.sigmoid(self.network(x)).squeeze(0).numpy()
```

Вероятность «день внутри эффекта» для каждого дня одной скважины. Весь ряд (L × 15) подаётся в сеть **целиком** одним батчем (`unsqueeze(0)` → `(1, L, 15)`), без нарезки на окна и без паддинга, потом `sigmoid` и `squeeze` дают массив `(L,)`. Целиком, потому что на обучении окна были лишь аугментацией, а в применении нужен контекст всего ряда. Работает под `torch.no_grad()`. Не обучена — `RuntimeError("Transformer не обучен")`.

Пример

```
p = model.predict_proba(well)      # скважина 8822: p.shape == (1587,)
```

Дальше `SegmentDecoder` сглаживает `p` окном в 5 дней и режет по порогу 0,5.

метод TransformerSegmenter.save

```
def save(self, path: Path) -> None
```

```
150     def save(self, path: Path) -> None:
151         path.parent.mkdir(parents=True, exist_ok=True)
152         torch.save(
153             {
154                 "config": asdict(self.config),
155                 "training": asdict(self.training),
156                 "channels": list(self.features.SEQUENCE_CHANNELS),
157                 "state_dict": self.network.state_dict(),
158             },
159             path,
160         )
```

Сохраняет через `torch.save` словарь: `config` и `training` (как словари dataclass), список `channels` (`SEQUENCE_CHANNELS` на момент обучения) и `state_dict` сети. Папки создаются. Список каналов нужен, чтобы `load` обнаружил изменение признаков.

Пример

```
model.save(Path("gtm_detector/ml_freq/weights/transformer.pt"))
```

метод класса TransformerSegmenter.load

```
def load(cls, path: Path) -> TransformerSegmenter
```

```
162     @classmethod
163     def load(cls, path: Path) -> TransformerSegmenter:
164         state = torch.load(path, map_location="cpu", weights_only=True)
165         model = cls(TransformerConfig(**state["config"]), TrainingConfig(**state["training"]))
166         if list(model.features.SEQUENCE_CHANNELS) != state["channels"]:
167             raise ValueError("Набор признаков изменился — переобучите модель")
168         model.network = TimeSeriesTransformer(model.features.sequence_size, model.config)
169         model.network.load_state_dict(state["state_dict"])
170         model.network.eval()
171         return model
```

Классовый метод: читает `.pt` (`weights_only=True` — безопасная загрузка без исполнения кода, `map_location="cpu"`), создаёт модель по сохранённым настройкам, сверяет список каналов и загружает веса. Если `SEQUENCE_CHANNELS` с тех пор изменились, выбрасывает `ValueError("Набор признаков изменился — переобучите модель")`: без проверки сеть получила бы данные «не в тех столбцах» и ошибалась бы молча. После загрузки — `eval()`.

Пример

```
model = TransformerSegmenter.load(Path("gtm_detector/ml_freq/weights/transformer.pt"))
model.n_parameters      # 19617
```

метод TransformerSegmenter._sample_crops

```
def _sample_crops( self, sequences: list[np.ndarray], targets: list[np.ndarray], masks: list[np.ndarray], rng: np.random.Generator, ) -> list[tuple[np.ndarray, np.ndarray, np.ndarray]]
```

```
173     def _sample_crops(
174         self,
175         sequences: list[np.ndarray],
176         targets: list[np.ndarray],
177         masks: list[np.ndarray],
178         rng: np.random.Generator,
179     ) -> list[tuple[np.ndarray, np.ndarray, np.ndarray]]:
180         """Случайные окна длиной crop_length вокруг размеченных мероприятий.
181 
182         Центр окна — случайный день из обучающей маски: окна не тратятся на дни,
183         которые всё равно не входят в функцию потерь.
184         """
185         length = self.training.crop_length
186         crops = []
187         for x, y, m in zip(sequences, targets, masks, strict=True):
188             allowed = np.flatnonzero(m)
189             if not len(allowed):
190                 continue
191             for _ in range(self.training.crops_per_well):
192                 center = int(rng.choice(allowed))
193                 begin = int(np.clip(center - length // 2, 0, max(0, len(x) - length)))
194                 window = slice(begin, begin + length)
195                 crops.append((x[window], y[window], m[window]))
196         return crops
```

Нарезает случайные окна для одной эпохи. Для каждой скважины с непустой маской обучения делает `crops_per_well` = 4 окон длиной `crop_length` = 128 дней. Центр окна — случайный день **из маски**, поэтому окна не тратятся на скважины и участки без размеченных событий. Начало окна обрезается в границы ряда (`clip(0, L − 128)`); если ряд короче 128 дней, окно короче.

Возвращает список троек `(признаки (≤128, 15), цель, маска)`. Окна каждый раз новые — это аугментация: одно и то же мероприятие оказывается в разных местах окна. Скважины без разметки пропускаются целиком.

Пример

Для 8 синтетических скважин из `tests/conftest.py` (по 120 дней, одно событие) получается 8 × 4 = 32 окна формы `(120, 15)`: ряд короче `crop_length`, окно покрывает его целиком. Скважина без разметки (`make_well(step=None)`) даёт 0 окон. На скважине 8822 (1587 дней) окно длиной 128 примерно совпадает по размеру с маской (128 дней).

статический метод TransformerSegmenter._collate

```
def _collate( batch: list[tuple[np.ndarray, np.ndarray, np.ndarray]], ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]
```

```
198     @staticmethod
199     def _collate(
200         batch: list[tuple[np.ndarray, np.ndarray, np.ndarray]],
201     ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]:
202         """Батч: признаки, цели, маска паддинга (True — нет данных) и веса дней в потерях."""
203         length = max(len(x) for x, _, _ in batch)
204         channels = batch[0][0].shape[1]
205         x = torch.zeros(len(batch), length, channels)
206         y = torch.zeros(len(batch), length)
207         weight = torch.zeros(len(batch), length)
208         padding = torch.ones(len(batch), length, dtype=torch.bool)
209         for i, (xi, yi, mi) in enumerate(batch):
210             x[i, : len(xi)] = torch.from_numpy(xi)
211             y[i, : len(yi)] = torch.from_numpy(yi)
212             weight[i, : len(mi)] = torch.from_numpy(mi.astype(np.float32))
213             padding[i, : len(xi)] = False
214         return x, y, padding, weight
```

Склеивает окна в батч, дополняя короткие нулями до самого длинного. Возвращает четыре тензора:

Тензор	Форма	Смысл
`x`	`(batch, L, 15)`	признаки, нули в паддинге
`y`	`(batch, L)`	цель 0/1
`padding`	`(batch, L)`, bool	`True` — позиция добавлена паддингом, внимание её игнорирует
`weight`	`(batch, L)`	1 — день входит в маску обучения (и в потери), 0 — нет, в том числе в паддинге

Две разные маски нужны потому, что «нет данных» (паддинг) и «данные есть, но учиться на них нельзя» (вне окрестности событий) — разные вещи: первое убирается из внимания, второе — из функции потерь.

Пример

```
x, y, padding, weight = TransformerSegmenter._collate(crops[:4])
x.shape                # torch.Size([4, 120, 15])
weight.sum(dim=1)      # tensor([102., 102., 102., 102.]): 102 из 120 дней в маске
```

Окно из 50 дней рядом с окном из 120: `padding.sum(1) == [70, 0]` — у короткого 70 позиций паддинга.

evaluation/__init__.py

Пакет `evaluation` — всё, что происходит **после** того, как модель выдала вероятность «в этот день эффект есть» для каждого дня скважины. Он собирает четыре шага в одну цепочку:

1. `SegmentDecoder` (`decoding.py`) превращает ряд вероятностей в список интервалов «начало — окончание».
2. `EventMatcher` и `Score` (`metrics.py`) сопоставляют найденные интервалы с разметкой технологов и считают метрики.
3. `CrossValidator` и `ModelResult` (`experiment.py`) повторяют это для каждой модели на одних и тех же фолдах, разбитых по скважинам.
4. `ComparisonReport` (`report.py`) пишет итог в `comparison.xlsx` и `comparison.png`.

Файл `__init__.py` ничего не вычисляет: он только собирает шесть публичных имён (`ComparisonReport`, `CrossValidator`, `EventMatcher`, `ModelResult`, `Score`, `SegmentDecoder`), чтобы в других местах писать `from .evaluation import CrossValidator`. Так делает `cli.py` в командах `compare` и `predict`.

Цепочка в проекте: «Excel → скважины (`data`) → признаки (`features`) → модель (`models`) → **интервалы и метрики (`evaluation`)**».

Пример

```
from gtm_detector.ml_freq.evaluation import ComparisonReport, CrossValidator, SegmentDecoder
```

Для `compare --models gboost extra_trees` `cli.run_compare` создаёт `CrossValidator`, получает `list[ModelResult]` и отдаёт его в `ComparisonReport`. Для `predict` используется только `SegmentDecoder`: разметки нет, метрики считать не с чем.

Код модуля вне классов и функций

```
3  from .decoding import SegmentDecoder
4  from .experiment import CrossValidator, ModelResult
5  from .metrics import EventMatcher, Score
6  from .report import ComparisonReport
7
8  __all__ = ["ComparisonReport", "CrossValidator", "EventMatcher", "ModelResult", "Score",
   "SegmentDecoder"]
```

evaluation/decoding.py

Модуль содержит единственный класс `SegmentDecoder` — последний шаг между моделью и технологом. Любая модель (Transformer, `gboost`, `logreg` …) выдаёт одно и то же: массив вероятностей по дням. Декодер превращает его в интервалы `Interval(start, end)`.

Декодер **общий для всех моделей** и настраивается только через `DecoderConfig` в `config.py`. Если бы у каждой модели были свои пороги, разница в метриках отражала бы разницу в порогах, а не в качестве моделей. Поэтому `CrossValidator` (сравнение) и `run_predict` в `cli.py` (боевой запуск) используют один и тот же класс с теми же настройками.

Код модуля вне классов и функций

```
 3 from __future__ import annotations
 4
 5 from collections.abc import Sequence
 6 from datetime import date
 7
 8 import numpy as np
 9 import pandas as pd
10
11 from ..config import DecoderConfig
12 from ..data import Interval
```

класс SegmentDecoder

`class SegmentDecoder`

```
15 class SegmentDecoder:
16     """Общий для всех моделей шаг: одинаковые правила — честное сравнение.
17
18     1. Вероятности сглаживаются скользящим средним (``smooth_days``).
19     2. Дни выше ``threshold`` — «в эффекте».
20     3. Разрывы не длиннее ``max_gap`` дней склеиваются.
21     4. Периоды короче ``min_length`` дней отбрасываются.
22     5. Период, дошедший до конца данных, — эффект продолжается (``end=None``).
23     """
24
   ...
27
   ...
40
   ...
46
```

Правила декодирования, по порядку:

1. Вероятности сглаживаются скользящим средним по `smooth_days` = 5 дням (окно по центру).
2. День считается «в эффекте», если сглаженная вероятность **строго больше** `threshold` = 0,5.
3. Подряд идущие такие дни образуют период; два периода, между которыми не больше `max_gap` = 3 дней, склеиваются.
4. Периоды короче `min_length` = 2 дней выбрасываются.
5. Если период дошёл до **последнего дня данных**, его окончание неизвестно: `end=None` («эффект продолжается»).

Зачем каждый шаг: сглаживание убирает единичные скачки вероятности; склейка нужна, потому что в середине настоящего эффекта частота или дебит иногда «проваливаются» на пару дней; минимальная длина отсекает шум; `end=None` соответствует строкам разметки «ПРОДОЛЖАЕТСЯ ЭФФЕКТ».

Пример

Скважина с рядом на 30 дней с 01.08.2023. Модель уверена (0,9) в днях 10–19, но в день 14 дала 0, и ещё есть короткий всплеск в днях 25–26:

```
from datetime import date, timedelta
import numpy as np
from gtm_detector.ml_freq.evaluation import SegmentDecoder

dates = [date(2023, 8, 1) + timedelta(i) for i in range(30)]
proba = np.zeros(30)
proba[10:20] = 0.9
proba[14] = 0.0
proba[25:27] = 0.9
SegmentDecoder().decode(proba, dates)
# [Interval(start=2023-08-11, end=2023-08-20)]
```

Провал в день 14 «залечило» сглаживание, а всплеск из двух дней после сглаживания даёт не больше 0,36 < 0,5 и до порога не доходит. Если же вероятность высока от 23.08 до конца данных (`proba[22:] = 0.9`), результат — `Interval(start=2023-08-23, end=None)`.

метод SegmentDecoder.__init__

```
def __init__(self, config: DecoderConfig | None = None) -> None
```

```
25      def __init__(self, config: DecoderConfig | None = None) -> None:
26          self.config = config or DecoderConfig()
```

Принимает `DecoderConfig` или `None`; при `None` берутся значения по умолчанию (`threshold=0.5`, `smooth_days=5`, `max_gap=3`, `min_length=2`). Конфиг сохраняется в `self.config`.

`CrossValidator` передаёт сюда `ExperimentConfig.decoder`, поэтому при сравнении и при `predict` значения совпадают.

```
SegmentDecoder().config.smooth_days   # 5
SegmentDecoder(DecoderConfig(threshold=0.7)).config.threshold   # 0.7
```

метод SegmentDecoder.decode

```
def decode(self, proba: np.ndarray, dates: Sequence[date]) -> list[Interval]
```

```
28      def decode(self, proba: np.ndarray, dates: Sequence[date]) -> list[Interval]:
29          cfg = self.config
30          smoothed = pd.Series(proba).rolling(cfg.smooth_days, center=True,
    min_periods=1).mean().to_numpy()
31          active = smoothed > cfg.threshold
32          runs = self._runs(active)
33          runs = self._merge(runs, cfg.max_gap)
34          last = len(proba) - 1
35          return [
36              Interval(start=dates[begin], end=None if end == last else dates[end])
37              for begin, end in runs
38              if end - begin + 1 >= cfg.min_length
39          ]
```

Главный метод. Принимает `proba` — массив `float` длины N (вероятность эффекта по каждому дню, 0…1) и `dates` — N дат тех же дней. Возвращает список `Interval` в хронологическом порядке; `end=None`, если период доходит до последнего дня.

Окно сглаживания центрированное и с `min_periods=1`, поэтому на краях ряда оно просто короче и данные не теряются. Условие «до конца данных» проверяется по индексу последнего дня (`end == len(proba) - 1`), а не по дате. Фильтр `min_length` применяется **после** склейки, чтобы две короткие части одного эффекта не были отброшены порознь.

Пример из `run_predict`: на скважине с моделью `gboost` декодер выдаёт строки для `events_gboost.xlsx` — столбцы «начало эффекта», «окончание эффекта», «статус». Для интервала с `end=None` в «статус» пишется «эффект продолжается».

```
SegmentDecoder().decode(np.zeros(30), dates)   # [] — эффектов нет
```

статический метод SegmentDecoder._runs

```
def _runs(active: np.ndarray) -> list[tuple[int, int]]
```

```
41      @staticmethod
42      def _runs(active: np.ndarray) -> list[tuple[int, int]]:
43          edges = np.diff(np.r_[0, active.astype(int), 0])
44          starts, stops = np.flatnonzero(edges == 1), np.flatnonzero(edges == -1) - 1
45          return list(zip(starts.tolist(), stops.tolist(), strict=True))
```

Служебный статический метод: из булева массива `active` находит непрерывные отрезки `True` и возвращает пары `(первый_индекс, последний_индекс)` включительно. Приём: к массиву с обеих сторон добавляются нули, берётся разность `np.diff`; +1 — начало отрезка, −1 — позиция сразу после его конца.

```
SegmentDecoder._runs(np.array([0, 1, 1, 0, 1, 1, 1, 0, 1], bool))
# [(1, 2), (4, 6), (8, 8)]
```

статический метод SegmentDecoder._merge

```
def _merge(runs: list[tuple[int, int]], max_gap: int) -> list[tuple[int, int]]
```

```
47      @staticmethod
48      def _merge(runs: list[tuple[int, int]], max_gap: int) -> list[tuple[int, int]]:
49          merged: list[tuple[int, int]] = []
50          for begin, end in runs:
51              if merged and begin - merged[-1][1] - 1 <= max_gap:
52                  merged[-1] = (merged[-1][0], end)
53              else:
54                  merged.append((begin, end))
55          return merged
```

Склеивает соседние отрезки из `_runs`, если дней между ними (между концом предыдущего и началом следующего) не больше `max_gap`. Принимает список пар индексов и `max_gap` (дни), возвращает новый список.

Расстояние считается как `begin − prev_end − 1`: количество дней в разрыве. Для `max_gap=3` разрыв из трёх дней склеивается, из четырёх — нет.

```
SegmentDecoder._merge([(2, 3), (5, 8), (20, 22)], 3)
# [(2, 8), (20, 22)]   — разрыв в 1 день склеен, разрыв в 11 дней нет
```

evaluation/metrics.py

Как понять, что модель «нашла мероприятие». Модуль сопоставляет интервалы модели (`predicted`) с разметкой технологов (`labels`) и считает метрики уровня **мероприятий**, а не дней.

Ключевые решения:

- **Сопоставление один к одному по дате начала с допуском ±7 дней** (`EventMatcher`). Одно найденное мероприятие не может «подтвердить» две строки разметки, а одна строка разметки — два найденных.
- **Ошибка окончания не считается для `censored`**: у мероприятий «ПРОДОЛЖАЕТСЯ ЭФФЕКТ» дата окончания в разметке — просто конец наблюдений, а не конец эффекта.
- **Precision занижена намеренно**: разметка неполная, и модель может найти настоящий подъём частоты, которого технолог не записал; он считается «ложным». Поэтому главные метрики — recall и точность дат.

Метрики по скважинам складываются оператором `+` (`Score.__add__`), так что итог по модели — сумма по всем тестовым скважинам всех фолдов.

Код модуля вне классов и функций

```
3  from __future__ import annotations
4
5  from collections.abc import Sequence
6  from dataclasses import dataclass, field
7
8  import numpy as np
9
10 from ..data import Interval
```

функция _share

```
def _share(errors: Sequence[int], days: int) -> float
```

```
13 def _share(errors: Sequence[int], days: int) -> float:
14     return float(np.mean(np.abs(errors) <= days)) if len(errors) else float("nan")
```

Доля ошибок, не превышающих `days` по модулю. Принимает список целых ошибок в днях (`найдено − разметка`), возвращает число 0…1, а для пустого списка — `nan` (а не 0: «нечего измерять» не то же самое, что «всё плохо»).

```
_share([-1, 1, 5, 0], 1)   # 0.75: три из четырёх ошибок в пределах ±1 день
_share([], 3)              # nan
```

Из неё получаются столбцы «начало: точно» (`days=0`), «начало: ±1 дн», «начало: ±3 дн», «конец: ±3 дн», «конец: ±7 дн».

функция _mae

```
def _mae(errors: Sequence[int]) -> float
```

```
17 def _mae(errors: Sequence[int]) -> float:
18     return float(np.mean(np.abs(errors))) if len(errors) else float("nan")
```

Средняя абсолютная ошибка в днях; для пустого списка — `nan`.

```
_mae([-1, 1, 5, 0])   # 1.75
```

В `comparison.xlsx` для `gboost`: «начало: MAE, дн» = 1,86, «конец: MAE, дн» = 20,5. Окончание ошибается сильнее: конец эффекта размыт, а редкие большие промахи (десятки дней) тянут среднее вверх.

класс-данные (dataclass) Score

`class Score`

```
21 @dataclass
22 class Score:
23     """Метрики уровня мероприятий; складываются оператором ``+``."""
24
25     true_positives: int = 0
26     false_positives: int = 0
27     false_negatives: int = 0
28     start_errors: list[int] = field(default_factory=list)  # найдено – разметка, дней
29     end_errors: list[int] = field(default_factory=list)
30     open_ends: int = 0  # в разметке конец есть, модель считает эффект продолжающимся
31
   …
36
   …
41
   …
46
   …
56
```

Счётчики и списки ошибок, из которых получаются все метрики. Поля:

Поле	Смысл
`true_positives` (TP)	размеченное мероприятие нашлось
`false_negatives` (FN)	размеченное не нашлось
`false_positives` (FP)	найденное не соответствует разметке
`start_errors`	`начало модели − начало разметки`, дни, по каждой паре TP
`end_errors`	то же для окончания, только для пар с известным концом
`open_ends`	в разметке конец есть, а модель считает эффект продолжающимся — ошибка окончания не определена

Класс — обычный изменяемый `dataclass`, у него нет допущений о скважинах: его можно складывать.

Пример

Реальные цифры `gboost` из `results/ml_freq/comparison/…(20-26 год, полугодия).xlsx`: TP = 312, FN = 112, FP = 3857, «не закрыт конец» = 0. Всего размеченных мероприятий 312 + 112 = 424 — столько же, сколько строк на листе `gboost`. Огромное FP объясняется неполной разметкой: большинство реальных подъёмов частоты в 6,8 года истории скважин технологи не размечали.

свойство Score.recall

`def recall(self) -> float`

```
32     @property
33     def recall(self) -> float:
```

```
34          total = self.true_positives + self.false_negatives
35          return self.true_positives / total if total else 0.0
```

`TP / (TP + FN)` — доля размеченных мероприятий, которые модель нашла. Если размеченных нет — 0.

```
Score(true_positives=84, false_negatives=51).recall   # 0.6222
```

Это столбец «найдено размеченных (recall)». Для сравнения: детектор на правилах из ветки `feature/increase-f-gtm` находит 89 %.

свойство Score.precision

```
def precision(self) -> float
```

```
37      @property
38      def precision(self) -> float:
39          found = self.true_positives + self.false_positives
40          return self.true_positives / found if found else 0.0
```

`TP / (TP + FP)` — какая часть найденного совпала с разметкой. Нет найденного — 0.

```
Score(true_positives=84, false_positives=143).precision   # 0.3700
```

Низкое значение не означает, что 63 % найденного — ошибки: часть FP — настоящие подъёмы частоты, не внесённые в разметку.

свойство Score.f1

```
def f1(self) -> float
```

```
42      @property
43      def f1(self) -> float:
44          p, r = self.precision, self.recall
45          return 2 * p * r / (p + r) if p + r else 0.0
```

Гармоническое среднее precision и recall; 0, если оба нуля. По F1 сортируется таблица сравнения.

```
Score(true_positives=84, false_positives=143, false_negatives=51).f1   # 0.4641 (в отчёте 0.464088)
```

метод Score.__add__

```
def __add__(self, other: Score) -> Score
```

```
47      def __add__(self, other: Score) -> Score:
48          return Score(
49              self.true_positives + other.true_positives,
50              self.false_positives + other.false_positives,
51              self.false_negatives + other.false_negatives,
52              self.start_errors + other.start_errors,
53              self.end_errors + other.end_errors,
54              self.open_ends + other.open_ends,
55          )
```

Складывает два `Score`: счётчики суммируются, списки ошибок объединяются. Возвращает новый объект. Нужно, чтобы накопить результат по всем скважинам: `result.score += matcher.score(predicted, well.labels)` в `CrossValidator._evaluate`.

Важно: складываются **ошибки**, а не доли, поэтому итоговая точность даты — это доля по всем мероприятиям, а не среднее долей по скважинам.

```
a = Score(1, 2, 3, [1], [2], 0)
a + a
# Score(true_positives=2, false_positives=4, false_negatives=6, start_errors=[1, 1], end_errors=[2, 2],
open_ends=0)
```

метод Score.as_dict

```
def as_dict(self) -> dict[str, float]
```

```
57      def as_dict(self) -> dict[str, float]:
58          return {
59              "найдено размеченных (recall)": self.recall,
60              "точность (precision)": self.precision,
61              "F1": self.f1,
62              "начало: точно": _share(self.start_errors, 0),
63              "начало: ±1 дн": _share(self.start_errors, 1),
64              "начало: ±3 дн": _share(self.start_errors, 3),
65              "начало: MAE, дн": _mae(self.start_errors),
66              "конец: точно": _share(self.end_errors, 0),
67              "конец: ±3 дн": _share(self.end_errors, 3),
68              "конец: ±7 дн": _share(self.end_errors, 7),
69              "конец: MAE, дн": _mae(self.end_errors),
70              "TP": self.true_positives,
71              "FN": self.false_negatives,
72              "FP": self.false_positives,
73              "не закрыт конец": self.open_ends,
74          }
```

Все метрики одним словарём с русскими названиями — это заголовки столбцов листа «Сравнение» в `comparison.xlsx`: recall, precision, F1, доли начала «точно / ±1 / ±3», MAE начала, доли конца «точно / ±3 / ±7», MAE конца, TP, FN, FP, «не закрыт конец». Метрики даты считаются только по найденным парам.

```
# Score после примера в EventMatcher.score
{'найдено размеченных (recall)': 1.0, 'точность (precision)': 0.667, 'F1': 0.8,
 'начало: точно': 0.0, 'начало: ±1 дн': 0.5, 'начало: ±3 дн': 1.0, 'начало: MAE, дн': 1.5,
 'конец: точно': 0.0, 'конец: ±3 дн': 1.0, 'конец: ±7 дн': 1.0, 'конец: MAE, дн': 2.0,
 'TP': 2, 'FN': 0, 'FP': 1, 'не закрыт конец': 0}
```

класс EventMatcher

```
class EventMatcher
```

```
77  class EventMatcher:
78      """Один-к-одному по дате начала (|Δ| ≤ ``start_tolerance``), сначала ближайшие пары."""
79
    ...
82
    ...
99
```

Сопоставляет найденные интервалы с размеченными. Пара допустима, только если даты начала различаются не более чем на `start_tolerance` дней (по умолчанию 7, `ExperimentConfig.start_tolerance`). Из всех допустимых пар сначала берутся самые близкие; каждый найденный и каждый размеченный интервал используется не более одного раза.

Почему «один к одному»: иначе одно длинное найденное мероприятие засчитывалось бы сразу за несколько размеченных, а recall оказался бы завышен. Окончание в сопоставлении не участвует: сначала

находим мероприятие по началу, затем отдельно смотрим, насколько точно угадан конец.

метод EventMatcher.__init__

```
def __init__(self, start_tolerance: int = 7) -> None
```

```
80      def __init__(self, start_tolerance: int = 7) -> None:
81          self.start_tolerance = start_tolerance
```

Принимает `start_tolerance` — допуск в днях. Значение 7 одинаково для всех моделей (его передаёт `CrossValidator` из конфига), поэтому сравнение честное.

```
EventMatcher().start_tolerance   # 7
```

метод EventMatcher.match

```
def match( self, predicted: Sequence[Interval], labels: Sequence[Interval] ) -> list[tuple[Interval,
Interval | None]]
```

```
83      def match(
84          self, predicted: Sequence[Interval], labels: Sequence[Interval]
85      ) -> list[tuple[Interval, Interval | None]]:
86          candidates = sorted(
87              (abs((p.start - t.start).days), i, j)
88              for i, p in enumerate(predicted)
89              for j, t in enumerate(labels)
90              if abs((p.start - t.start).days) <= self.start_tolerance
91          )
92          used_predicted: set[int] = set()
93          found: dict[int, int] = {}
94          for _, i, j in candidates:
95              if i not in used_predicted and j not in found:
96                  used_predicted.add(i)
97                  found[j] = i
98          return [(label, predicted[found[j]] if j in found else None) for j, label in
    enumerate(labels)]
```

Принимает найденные интервалы `predicted` и размеченные `labels`. Возвращает список пар `(метка, найденный | None)` **по одной на каждую метку**, в порядке меток; `None` — не нашли.

Алгоритм жадный: все допустимые пары сортируются по `(|Δ начала|, номер найденного, номер метки)`, затем проходом по списку берутся пары, где ни найденное, ни метка ещё не заняты. При равном расстоянии выигрывает найденный интервал с меньшим номером, то есть более ранний.

Ненайденные интервалы (лишние) в результат не попадают — их считает `score` как FP. По этой функции `CrossValidator._rows` строит листы с предсказаниями по каждой модели.

Пример

Скважина с двумя метками: подъём 14.08.2023 (до 20.09) и мероприятие 01.12.2023 с пометкой «продолжается» (`censored=True`). Модель нашла три интервала:

```
labels = [Interval(date(2023, 8, 14), date(2023, 9, 20)),
          Interval(date(2023, 12, 1), date(2023, 12, 31), censored=True)]
predicted = [Interval(date(2023, 8, 15), date(2023, 9, 22)),
             Interval(date(2023, 12, 3), None),
             Interval(date(2024, 3, 1), date(2024, 3, 9))]
EventMatcher().match(predicted, labels)
# [(метка 14.08, найдено 15.08–22.09), (метка 01.12, найдено 03.12–…)]
```

Третий интервал (01.03.2024) пары не нашёл — он лишний и будет учтён как FP.

метод EventMatcher.score

```
def score(self, predicted: Sequence[Interval], labels: Sequence[Interval]) -> Score
```

```python
    def score(self, predicted: Sequence[Interval], labels: Sequence[Interval]) -> Score:
        pairs = self.match(predicted, labels)
        score = Score()
        for label, hit in pairs:
            if hit is None:
                score.false_negatives += 1
                continue
            score.true_positives += 1
            score.start_errors.append((hit.start - label.start).days)
            if label.end is not None and not label.censored:  # у censored конец эффекта неизвестен
                if hit.end is None:
                    score.open_ends += 1
                else:
                    score.end_errors.append((hit.end - label.end).days)
        score.false_positives = len(predicted) - score.true_positives
        return score
```

Строит `Score` для одной скважины: сопоставляет через `match` и считает:

- нет пары с меткой → `false_negatives += 1`;
- есть пара → `true_positives += 1`, в `start_errors` добавляется `найденное_начало − начало_метки` в днях (отрицательное — модель раньше технолога);
- ошибка окончания записывается, только если у метки есть конец **и** она не `censored`; если при этом модель считает эффект продолжающимся (`end=None`), увеличивается `open_ends`, а не `end_errors`;
- `false_positives = len(predicted) − TP`.

Для примера из `match`: TP = 2, FN = 0, FP = 1; ошибки начала `[1, 2]` (MAE 1,5), ошибка окончания одна: `22.09 − 20.09 = 2` дня. Вторая метка `censored`, поэтому её конец в метрику не идёт. Это правило объясняет, почему у `gboost` «конец» оценён по ~60 мероприятиям из 84 найденных.

```python
EventMatcher().score(predicted, labels).as_dict()['конец: ±3 дн']   # 1.0
```

evaluation/experiment.py

Перекрёстная проверка: честный ответ на вопрос «какая модель лучше». Модуль берёт набор размеченных скважин, делит его на фолды **по скважинам**, для каждой модели по очереди учит на четырёх фолдах и проверяет на пятом, а потом собирает результаты в `ModelResult`.

Почему по скважинам, а не по дням: соседние дни одной скважины почти одинаковы. Если дни одной скважины попадут и в обучение, и в проверку, модель «узнает» скважину, и метрики будут завышены (утечка между скважинами — первое правило ML-части проекта).

Все модели получают **одинаковые фолды** (их считает `folds` один раз из `ExperimentConfig.seed`), один `SegmentDecoder` и один `EventMatcher`. Результат передаётся в `ComparisonReport`.

Код модуля вне классов и функций

```python
from __future__ import annotations

import logging
import time
from collections.abc import Callable, Mapping, Sequence
from dataclasses import dataclass, field

import numpy as np
from sklearn.metrics import roc_auc_score

from ..config import ExperimentConfig
from ..data import WellSeries
from ..models import EventSegmenter
from .decoding import SegmentDecoder
from .metrics import EventMatcher, Score

logger = logging.getLogger(__name__)
```

класс-данные (dataclass) ModelResult

`class ModelResult`

```python
@dataclass
class ModelResult:
    """Итог перекрёстной проверки одной модели."""

    name: str
    title: str
    score: Score = field(default_factory=Score)
    day_auc: float = float("nan")  # ROC-AUC по дням: качество вероятностей до декодирования
    train_seconds: float = 0.0
    predictions: list[dict] = field(default_factory=list)  # строки «метка — найдено»

```

Итог проверки одной модели (изменяемый `dataclass`).

Поле	Смысл
`name`	ключ в `model_registry()`: `transformer`, `gboost`… Используется как имя листа в Excel
`title`	читаемое название: «Градиентный бустинг (HistGB)»
`score`	суммарный `Score` по всем тестовым скважинам всех фолдов
`day_auc`	ROC-AUC по дням: качество вероятностей **до** декодирования
`train_seconds`	время `fit` по всем фолдам, секунды

`predictions`	строки «метка — найдено» для листа с предсказаниями

Пример

Для `gboost` в `results/ml_freq/comparison/сравнение_качества_моделей_(20-26 год, полугодия).xlsx`: `score` — TP 312, FN 112, FP 3857; `day_auc` = 0,871; `train_seconds` = 214,4; `predictions` — 424 строки (по одной на размеченное мероприятие).

метод ModelResult.as_row

```
def as_row(self) -> dict
```

```
33      def as_row(self) -> dict:
34          return {
35              "модель": self.title,
36              **self.score.as_dict(),
37              "ROC-AUC по дням": self.day_auc,
38              "обучение, с (все фолды)": round(self.train_seconds, 1),
39          }
```

Превращает результат в одну строку таблицы сравнения: «модель» (это `title`), все метрики из `Score.as_dict()`, «ROC-AUC по дням» и «обучение, с (все фолды)» (секунды, округлено до 0,1). Из таких строк `ComparisonReport.table` собирает лист «Сравнение».

```
result.as_row()['ROC-AUC по дням']   # 0.917137 для gboost
result.as_row()['обучение, с (все фолды)']   # 112.2
```

класс CrossValidator

```
class CrossValidator
```

```
42  class CrossValidator:
43      """k-fold по скважинам: модель не видит скважины, на которых её проверяют."""
44
    ...
49
    ...
53
    ...
59
    ...
93
```

Считает метрики моделей методом k-fold по скважинам. Отдельные шаги: `folds` — разбиение, `run` — цикл по моделям, `_evaluate` — проверка одной модели, `_rows` — строки для листа предсказаний.

ROC-AUC считается **по всем дням всех тестовых скважин сразу** (вероятности и целевые значения склеиваются в два длинных массива), а не по скважинам с последующим усреднением: в отдельной скважине может не быть ни одного дня эффекта, и для неё AUC не определён.

Команда: `python -m gtm_detector.ml_freq compare --models gboost extra_trees` (около 2 минут), все модели — около 15 минут.

метод CrossValidator.__init__

```
def __init__(self, config: ExperimentConfig | None = None) -> None
```

```
45      def __init__(self, config: ExperimentConfig | None = None) -> None:
46          self.config = config or ExperimentConfig()
```

```
47          self.decoder = SegmentDecoder(self.config.decoder)
48          self.matcher = EventMatcher(self.config.start_tolerance)
```

Принимает `ExperimentConfig` или `None` (по умолчанию `folds=5`, `seed=0`, `start_tolerance=7`, `DecoderConfig()`). Создаёт `SegmentDecoder` и `EventMatcher` из этого же конфига — чтобы декодирование и допуск были одинаковы у всех моделей.

В `cli.run_compare`: `CrossValidator(ExperimentConfig(folds=args.folds, seed=args.seed))`. Ключи `--folds` и `--seed` попадают сюда.

```
cv = CrossValidator()
cv.matcher.start_tolerance   # 7
cv.decoder.config.threshold  # 0.5
```

метод CrossValidator.folds

```
def folds(self, wells: Sequence[WellSeries]) -> list[list[int]]
```

```
50      def folds(self, wells: Sequence[WellSeries]) -> list[list[int]]:
51          order = np.random.default_rng(self.config.seed).permutation(len(wells))
52          return [part.tolist() for part in np.array_split(order, self.config.folds)]
```

Возвращает список из `config.folds` списков индексов скважин. Индексы перемешиваются генератором `np.random.default_rng(seed)` (воспроизводимо, без глобального `np.random`), затем делятся `np.array_split` на почти равные части.

Получается не «кусок подряд из Excel», а случайная выборка скважин: иначе соседние скважины одного куста (похожие по условиям) попадали бы в один фолд. Фолды одни и те же для всех моделей при том же `seed`.

```
CrossValidator().folds([None] * 12)
# [[9, 2, 7], [4, 5, 11], [0, 3], [6, 10], [8, 1]]
```

На 369 размеченных скважинах набора 2020–2026 фолды получаются размером 74, 74, 74, 74, 73.

метод CrossValidator.run

```
def run( self, wells: Sequence[WellSeries], models: Mapping[str, Callable[[], EventSegmenter]] ) ->
list[ModelResult]
```

```
54      def run(
55          self, wells: Sequence[WellSeries], models: Mapping[str, Callable[[], EventSegmenter]]
56      ) -> list[ModelResult]:
57          folds = self.folds(wells)
58          return [self._evaluate(name, factory, wells, folds) for name, factory in models.items()]
```

Принимает скважины `wells` (только с разметкой) и словарь `{имя: фабрика_модели}` — обычно часть `model_registry()`. Фолды считаются один раз, затем для каждой модели вызывается `_evaluate`. Возвращает `list[ModelResult]` в порядке словаря.

Модель передаётся **фабрикой** (вызываемым без аргументов), а не готовым объектом: на каждом фолде нужна свежая, необученная копия, иначе знания о предыдущем фолде утекли бы в следующий.

```
results = CrossValidator().run(wells, {"gboost": registry["gboost"]})
results[0].title   # 'Градиентный бустинг (HistGB)'
```

метод CrossValidator._evaluate

```
def _evaluate( self, name: str, factory: Callable[[], EventSegmenter], wells: Sequence[WellSeries], folds: list[list[int]], ) -> ModelResult
```

```
60      def _evaluate(
61          self,
62          name: str,
63          factory: Callable[[], EventSegmenter],
64          wells: Sequence[WellSeries],
65          folds: list[list[int]],
66      ) -> ModelResult:
67          result: ModelResult | None = None
68          targets, probas = [], []
69          for k, test_idx in enumerate(folds):
70              test_set = set(test_idx)
71              train = [well for i, well in enumerate(wells) if i not in test_set]
72              model = factory()
73              if result is None:
74                  result = ModelResult(name=name, title=model.title)
75              started = time.perf_counter()
76              model.fit(train)
77              result.train_seconds += time.perf_counter() - started
78
79              for i in test_idx:
80                  well = wells[i]
81                  proba = model.predict_proba(well)
82                  targets.append(well.target())
83                  probas.append(proba)
84                  predicted = self.decoder.decode(proba, well.dates)
85                  result.score += self.matcher.score(predicted, well.labels)
86                  result.predictions.extend(self._rows(well, predicted))
87              logger.info("%s: фолд %d/%d готов", name, k + 1, len(folds))
88
89          y, p = np.concatenate(targets), np.concatenate(probas)
90          result.day_auc = float(roc_auc_score(y, p)) if 0 < y.sum() < len(y) else float("nan")
91          logger.info("%s: recall=%.3f F1=%.3f", name, result.score.recall, result.score.f1)
92          return result
```

Проверка одной модели. Для каждого из k фолдов:

1. `train` — все скважины, **кроме** тестовых; `factory()` создаёт новую модель; `model.fit(train)` замеряется по времени (секунды суммируются в `train_seconds`).
2. Для каждой тестовой скважины: `predict_proba(well)` → вероятности по дням; `well.target()` — истинные дни эффекта; `decoder.decode` → интервалы; `matcher.score` → `Score`, который прибавляется к `result.score`; `_rows` добавляет строки в `result.predictions`.

После всех фолдов дневные массивы склеиваются и считается `roc_auc_score`. Если в данных нет ни одного дня эффекта или все дни — эффект, AUC не определён, записывается `nan`. Фиксируется только `title` первой модели фолда: `ModelResult` создаётся лениво, потому что название известно лишь у объекта модели.

Обучение идёт по `training_mask` (±45 дней вокруг размеченных мероприятий — `CONTEXT_DAYS`), это делает сама модель в `fit`, а проверка идёт по **всему** ряду тестовой скважины: именно так модель будет работать в `predict`.

В лог (`-v`) пишется «gboost: фолд 3/5 готов» и итог «recall=0.622 F1=0.464».

метод CrossValidator._rows

```
def _rows(self, well: WellSeries, predicted: Sequence) -> list[dict]
```

```
94      def _rows(self, well: WellSeries, predicted: Sequence) -> list[dict]:
95          rows = []
```

```
        for label, hit in self.matcher.match(predicted, well.labels):
            rows.append(
                {
                    "скважина": well.number,
                    "куст": well.cluster,
                    "начало (разметка)": label.start,
                    "окончание (разметка)": label.end,
                    "начало (модель)": hit.start if hit else None,
                    "окончание (модель)": hit.end if hit else None,
                    "Δ начала, дн": (hit.start - label.start).days if hit else None,
                    "Δ окончания, дн": (
                        (hit.end - label.end).days
                        if hit and hit.end and label.end and not label.censored
                        else None
                    ),
                    "эффект продолжался": label.censored,
                }
            )
        return rows
```

Строит строки листа предсказаний для одной скважины: по одной на каждую размеченную метку (из `matcher.match`). Столбцы: «скважина», «куст», «начало (разметка)», «окончание (разметка)», «начало (модель)», «окончание (модель)», «Δ начала, дн», «Δ окончания, дн», «эффект продолжался».

Для ненайденной метки модельные поля пусты. «Δ окончания» заполняется только когда у обеих сторон есть конец и метка не `censored` — те же правила, что в `EventMatcher.score`.

Строки листа `gboost` из `comparison.xlsx`:

скважина	куст	начало (разметка)	начало (модель)	Δ начала, дн	Δ окончания, дн	эффект продолжался
8822	615A	30.07.2022	29.07.2022	−1	−1	False
25571	2161	30.08.2022	31.08.2022	1	—	True
32416	1910	25.11.2024	— (не найдено)	—	—	True

Лист нужен, чтобы увидеть, на каких скважинах модель промахнулась, и показать это технологам.

evaluation/report.py

Последний шаг команды `compare`: результаты моделей (`list[ModelResult]`) превращаются в файлы, которые можно открыть без Python, — Excel-таблицу `comparison.xlsx` и картинку `comparison.png`. Расчётов здесь нет, только оформление.

В папке `results/ml_freq/comparison/` лежат файлы, переименованные вручную («сравнение_качества_моделей_(20-26 год, полугодия).xlsx» и `.png`, а также (на 1м старом датасете)); команда создаёт их под именами `comparison.xlsx` и `comparison.png`.

Код модуля вне классов и функций

```
3  from __future__ import annotations
4
5  import logging
6  from collections.abc import Sequence
7  from pathlib import Path
8
9  import pandas as pd
10
11 from .experiment import ModelResult
12
13 logger = logging.getLogger(__name__)
14
15 BLUE = "#2a78d6"
16 TEXT_SECONDARY = "#6b6a65"
17 GRID = "#e4e3de"
```

класс ComparisonReport

`class ComparisonReport`

```
20 class ComparisonReport:
21     """Пишет ``comparison.xlsx`` (метрики + предсказания по моделям) и ``comparison.png``."""
22
23     KEY_METRICS = (
24         "найдено размеченных (recall)",
25         "начало: точно",
26         "начало: ±3 дн",
27         "конец: точно",
28         "конец: ±7 дн",
29         "ROC-AUC по дням",
30     )
31
   ...
34
   ...
38
   ...
42
   ...
51
```

Отчёт по списку `ModelResult`. Создаётся в `cli.run_compare`:

```
report = ComparisonReport(results)
report.save(out_dir)          # comparison.xlsx + comparison.png
print(report.to_text())       # короткая таблица в консоль
```

Атрибут `KEY_METRICS` — шесть главных метрик, они попадают на график и в консольную таблицу: recall, «начало: точно», «начало: ±3 дн», «конец: точно», «конец: ±7 дн», «ROC-AUC по дням». Эти метрики

выбраны потому, что precision занижена из-за неполной разметки.

Цвета графика заданы константами `BLUE`, `TEXT_SECONDARY`, `GRID` в модуле.

метод ComparisonReport.__init__

```
def __init__(self, results: Sequence[ModelResult]) -> None
```

```
32      def __init__(self, results: Sequence[ModelResult]) -> None:
33          self.results = list(results)
```

Принимает любую последовательность `ModelResult` и сохраняет её копией списка в `self.results`. Порядок не важен: таблица сортируется сама.

```
ComparisonReport(results).results[0].name   # 'transformer' и т. д.
```

метод ComparisonReport.table

```
def table(self) -> pd.DataFrame
```

```
35      def table(self) -> pd.DataFrame:
36          table = pd.DataFrame([result.as_row() for result in self.results]).set_index("модель")
37          return table.sort_values("F1", ascending=False)
```

Возвращает `DataFrame`: одна строка на модель, индекс — «модель» (читаемое название), столбцы — все метрики `Score.as_dict()` плюс «ROC-AUC по дням» и время обучения. Строки отсортированы по F1 по убыванию — лучшая модель сверху.

Пример из `results/ml_freq/comparison/…(20-26 год, полугодия)` (по F1): Случайный лес 0,141; Градиентный бустинг 0,136; Extra Trees 0,131; Transformer 0,081; Логистическая регрессия 0,069. По ROC-AUC по дням: Extra Trees 0,884, Градиентный бустинг 0,871, Случайный лес 0,862, логистическая регрессия 0,818, Transformer 0,751. ROC-AUC оценивает вероятности до декодирования, F1 — уже готовые интервалы, поэтому по ним удобно отличать «плохая модель» от «плохо подобранный порог декодера».

метод ComparisonReport.to_text

```
def to_text(self) -> str
```

```
39      def to_text(self) -> str:
40          table = self.table()[list(self.KEY_METRICS) + ["F1"]]
41          return table.to_string(float_format=lambda value: f"{value:.2f}")
```

Строка для консоли: таблица только по `KEY_METRICS` и F1, числа с двумя знаками. Так выглядит вывод после `compare` (числа из `comparison.xlsx` для merged-набора):

```
                          найдено размеченных (recall)  начало: точно  начало: ±3 дн  конец: точно  конец:
±7 дн  ROC-AUC по дням    F1
Случайный лес                                     0.61           0.14           0.78           0.12
0.68             0.92  0.48
```

метод ComparisonReport.save

```
def save(self, out_dir: Path) -> None
```

```
43      def save(self, out_dir: Path) -> None:
44          out_dir.mkdir(parents=True, exist_ok=True)
45          with pd.ExcelWriter(out_dir / "comparison.xlsx") as writer:
```

```
            self.table().to_excel(writer, sheet_name="Сравнение")
            for result in self.results:
                pd.DataFrame(result.predictions).to_excel(writer, sheet_name=result.name[:31],
    index=False)
        self._plot(out_dir / "comparison.png")
        logger.info("Отчёт: %s", out_dir)
```

Принимает папку `out_dir` (создаётся при необходимости) и пишет:

- `comparison.xlsx`: лист «Сравнение» — `table()`; затем по листу на каждую модель (имя листа — `result.name`, не длиннее 31 символа — ограничение Excel) со строками предсказаний «метка — найдено»;
- `comparison.png` — график `_plot`.

Если `--out-dir` не задан, `cli.run_compare` создаёт папку `results/ml_freq/<дд-мм-гггг_чч-мм-сс>`. В `.vscode/launch.json` отладочный запуск пишет в `results/ml_freq/debug/compare`.

```
.venv/Scripts/python -m gtm_detector.ml_freq compare --models gboost --out-dir results/ml_freq/debug/compare
```

В файле, как в `comparison/…(20-26 год, полугодия)`: листы `Сравнение`, `transformer`, `logreg`, `random_forest`, `extra_trees`, `gboost`; на каждом из пяти листов моделей по 424 строки (число размеченных мероприятий).

метод ComparisonReport._plot

```
def _plot(self, path: Path) -> None
```

```
    def _plot(self, path: Path) -> None:
        import matplotlib

        matplotlib.use("Agg")
        import matplotlib.pyplot as plt

        table = self.table()
        figure, axes = plt.subplots(2, 3, figsize=(15, 7.5), sharey=True)
        for axis, metric in zip(axes.flat, self.KEY_METRICS, strict=True):
            values = table[metric].fillna(0)
            bars = axis.barh(table.index, values, color=BLUE, height=0.6)
            for bar, value in zip(bars, values, strict=True):
                axis.text(
                    value + 0.01,
                    bar.get_y() + bar.get_height() / 2,
                    f"{value:.0%}",
                    va="center",
                    fontsize=9,
                    color=TEXT_SECONDARY,
                )
            axis.set_xlim(0, 1.12)
            axis.set_title(metric, loc="left", fontsize=10)
            axis.invert_yaxis()
            for side in ("top", "right"):
                axis.spines[side].set_visible(False)
            axis.grid(axis="x", color=GRID)
            axis.set_axisbelow(True)
        figure.suptitle("Сравнение моделей (перекрёстная проверка по скважинам)", x=0.01, ha="left")
        figure.tight_layout()
        figure.savefig(path, dpi=110)
        plt.close(figure)
```

Рисует сетку 2×3: по одной панели на каждую из шести `KEY_METRICS`, на панели — горизонтальные столбцы по моделям, подписи в процентах. Пропуски (`nan`, например нет ошибок окончания) рисуются как 0. Ось X общая, 0…1,12, с запасом под подпись. Бэкенд `Agg` включается внутри функции, чтобы график строился без

окна — в консоли и в тестах. После сохранения (110 dpi) фигура закрывается, иначе память утекала бы при многократных запусках.

Порядок моделей на графике такой же, как в `table()`, то есть по F1.

audit.py

Команда `audit` отвечает на вопрос «каким данным можно верить». В прежней склеенной статистике часть блоков была сдвинута на 180 дней (в исходных полугодовых файлах 2020–2026 сдвига нет), а в разметке технологов есть мероприятия, которых замеры не подтверждают. Модуль берёт результат `MergedWellRepository.build()` (скважины + `DataAudit` с решениями) и делает из него три вещи:

1. Текстовый отчёт в консоль (`DataAuditReport.to_text`).
2. Файлы в `results/ml_freq/data_audit/` (актуальный запуск лежит в `data_audit_halfyear/`): `data_audit.xlsx`, `data_audit.png`, `rejected_labels.csv` (`DataAuditReport`).
3. Файл `training_data.xlsx` — ровно те данные, на которых учатся модели (`TrainingDataReport`).

Сам алгоритм исправления сдвигов находится в `data/repair.py` (`BlockShiftRepair`), отбор мероприятий — в `data/dataset.py`. Здесь только отчётность и командная строка (`build_parser`, `run`, `main`). Длинная документация в начале файла объясняет, откуда берётся сдвиг и как принимаются решения «без сдвига», «сдвиг −180 дн», «исключён».

Запуск:

```
.venv/Scripts/python -m gtm_detector.ml_freq audit
.venv/Scripts/python -m gtm_detector.ml_freq audit --out-dir results/ml_freq/data_audit_halfyear
```

Первое чтение 14 файлов `.xls` занимает около трёх минут, дальше данные берутся из кэша.

Цифры последнего запуска (полугодовые файлы, `data_audit_halfyear/`)

- Из 786 строк разметки принято 424 мероприятия на 369 скважинах; отброшено 313 «частота в замерах не поднялась», 40 дубликатов, 5 «нет телеметрии частоты», 3 «частота не увеличена», 1 «не выполнено». Принято по годам: 2021 — 48, 2022 — 49, 2023 — 82, 2024 — 59, 2025 — 109, 2026 — 77.
- Сегментов нет: сдвигов в исходных файлах нет, раздел «Сегменты» в отчёте содержит пометку «нет: статистика из исходных полугодовых файлов, исправлять нечего».
- Скважины: 369 с разметкой, 242 без разметки, 5 без телеметрии частоты (не используются).
- Для прежнего склеенного файла: принято 135 на 122 скважинах, 17 % ячеек исключено (папка `data_audit/`).

Код модуля вне классов и функций

```
68  from __future__ import annotations
69
70  import argparse
71  import sys
72  from collections.abc import Sequence
73  from pathlib import Path
74
75  import matplotlib
76
77  matplotlib.use("Agg")
78  import matplotlib.pyplot as plt  # noqa: E402
79  import pandas as pd  # noqa: E402
80
81  from .config import CONTEXT_DAYS, DataConfig  # noqa: E402
82  from .data import DataAudit, MergedWellRepository, WellSeries  # noqa: E402
83  from .data.dataset import ACCEPTED  # noqa: E402
84  from .data.statistics import MergedStatisticsReader  # noqa: E402
85
86  # Главные причины отказа -> пояснение для технологов (файл rejected_labels.csv).
87  REJECTED = {
88      "скважины нет в статистике": "скважины с таким номером/кустом нет в файле статистики",
```

```
    "нет замеров около даты начала": "у скважины нет замеров за ±14 дней от даты начала",
    "частота в замерах не поднялась около даты начала": (
        "в ±3 дня от даты начала частота не выросла: ни до «Нового значения», "
        "ни на половину запланированного прироста"
    ),
}

# Листы training_data.xlsx — с именами и в порядке исходного файла статистики.
STATISTICS_SHEETS = {
    "debit_liquid": "Qж (из OIS)",
    "watercut": "Обв (из OIS)",
    "debit_oil": "Qн (из OIS)",
    "pressure_line": "Рлин (из OIS)",
    "frequency": "Fэцн ТМ(сут)",
    "pressure_zab": "Рзаб",
}

# Столбцы разметки -> заголовки итогового файла технологов.
LABEL_HEADERS = {
    "куст": "Куст",
    "скважина": "Скважина",
    "тип ГТМ": "Тип ГТМ",
    "начало": "дата начала мониторинга",
    "окончание": "дата окончания мониторинга",
    "частота до, Гц": "Текущее значение",
    "частота после, Гц": "Новое значение",
    "причина окончания": "Причина прекращения мониторинга",
    "год": "Год",
}

# Цвета: линии — нейтральный и первый категориальный; заливка решений — светлые ступени.
COLORS = {"before": "#9a9993", "after": "#2a78d6", "shifted": "#cde2fb", "dropped": "#fbd9cb"}

...
if __name__ == "__main__":
    sys.exit(main())
```

класс DataAuditReport

`class DataAuditReport`

```
class DataAuditReport:
    """Текст, Excel и график по :class:`DataAudit`."""

...

...

...

...

...

...
```

Представление `DataAudit` (результат репозитория) в виде таблиц, текста и графика. Сам ничего не пересчитывает: берёт готовые поля `segments`, `labels`, `wells`, `non_numeric`, `wells_per_day_before/after`.

Листы Excel и CSV нужны технологам: по ним видно, какие мероприятия не вошли в обучение и по какой причине.

метод DataAuditReport.__init__

```
def __init__(self, audit: DataAudit) -> None
```

```
126      def __init__(self, audit: DataAudit) -> None:
127          self.audit = audit
```

Принимает `DataAudit` и хранит его в `self.audit`.

```
wells, audit = MergedWellRepository(statistics, labels, reference).build()
report = DataAuditReport(audit)
```

метод DataAuditReport.segments_table

```
def segments_table(self) -> pd.DataFrame
```

```
129      def segments_table(self) -> pd.DataFrame:
130          rows = []
131          for s in self.audit.segments:
132              rows.append(
133                  {
134                      "начало": s.start,
135                      "конец": s.end,
136                      "дней": s.days,
137                      "скважин": s.wells,
138                      "ячеек с данными": s.cells,
139                      "разметка, сдвиг 0": _evidence(s.labels.get(0)),
140                      "разметка, сдвиг 180": _evidence(s.labels.get(180)),
141                      "выгрузка, сдвиг 0": _evidence(s.reference.get(0)),
142                      "выгрузка, сдвиг 180": _evidence(s.reference.get(180)),
143                      "решение": s.decision,
144                      "обоснование": s.verdict,
145                      "наложений (оставлены прежние)": s.collisions,
146                  }
147              )
148          return pd.DataFrame(rows)
```

Таблица сегментов: по строке на отрезок календаря, где набор скважин с данными не менялся (след одной выгрузки). Столбцы: «начало», «конец», «дней», «скважин», «ячеек с данными», четыре столбца «доказательств» (по разметке и по исходной выгрузке, для сдвигов 0 и 180), «решение», «обоснование», «наложений».

«Доказательства» имеют вид `a/b`: подтверждено a из b проверенных; `–`, если проверять нечем (форматирует `_evidence`). Это лист «сегменты» в `data_audit.xlsx` (39 строк).

Пример — последний сегмент 2026 года:

начало	конец	скважин	выгрузка, сдвиг 180	решение
2026-09-04	2026-12-27	343	343/343	сдвиг −180 дн

Календарь, где ноябрь–декабрь 2026 ещё не наступили, — явный признак сдвинутого блока; сверка с исходной выгрузкой совпала у всех 343 скважин.

метод DataAuditReport.non_numeric_by_month

```
def non_numeric_by_month(self) -> pd.Series
```

```
150      def non_numeric_by_month(self) -> pd.Series:
151          series = self.audit.non_numeric
152          monthly = series.groupby(series.index.to_period("M")).sum()
```

```
153         return monthly[monthly > 0]
```

Сколько заполненных ячеек статистики содержат не число (текст, дата), по месяцам. Месяцы без таких ячеек отбрасываются. Такие ячейки считаются пропусками. На текущих данных результат пустой, и в отчёте печатается «нет: все заполненные ячейки — числа»; лист «не числа» в Excel тогда пустой.

Возвращает `pd.Series` с `PeriodIndex` по месяцам.

метод DataAuditReport.to_text

```
def to_text(self) -> str
```

```
155     def to_text(self) -> str:
156         a = self.audit
157         segments = self.segments_table()
158         labels, wells = a.labels, a.wells
159         accepted = labels[labels["решение"] == ACCEPTED]
160         cells = {}
161         if not segments.empty:
162             cells = {
163                 d: segments.loc[segments["решение"] == d, "ячеек с данными"].sum()
164                 for d in segments["решение"]
165             }
166 
167         lines = [
168             "=" * 100,
169             "1. Заполненные ячейки не с числом (текст, дата) — считаются пропусками",
170             "=" * 100,
171         ]
172         monthly = self.non_numeric_by_month()
173         lines += [f"   {period}: {count:>8,}".replace(",", " ") for period, count in monthly.items()]
174         if monthly.empty:
175             lines.append(
176                 "   нет: все заполненные ячейки — числа (пустые столбцы 2020 и конца 2026 г. — 
    пропуски)"
177             )
178 
179         lines += ["", "=" * 100, "2. Сегменты (след отдельных выгрузок) и решения", "=" * 100]
180         if segments.empty:
181             lines.append("   нет: статистика из исходных полугодовых файлов, исправлять нечего")
182         else:
183             lines.append(
184                 "   разметка «a/b» — подтверждено a мероприятий из b проверенных; "
185                 "выгрузка «a/b» — совпало a скважин из b"
186             )
187             lines.append(_indent(segments.drop(columns=["ячеек с данными"]).to_string(index=False)))
188             lines.append("")
189         total = sum(cells.values())
190         for decision, count in sorted(cells.items(), key=lambda item: -item[1]):
191             lines.append(f"   {decision:<16} {count:>9,} ячеек ({count / total:.0%})".replace(",", " 
    "))
192         if a.reference is not None:
193             lines.append(f"   исходная выгрузка для сверки: {a.reference}")
194 
195         lines += ["", "=" * 100, "3. Мероприятия итогового файла («Лист1»)", "=" * 100]
196         funnel = labels["решение"].value_counts()
197         for reason, count in funnel.items():
198             lines.append(f"   {count:>5}  {reason}")
199         lines.append(f"   {len(labels):>5}  всего строк")
200         lines.append("")
201         by_year = pd.crosstab(labels["год"], labels["решение"] == ACCEPTED).rename(
202             columns={True: "принято", False: "не принято"}
203         )
204         lines.append(_indent(by_year.to_string()))
205         censored = int(accepted["эффект продолжался"].sum())
206         lines.append(f"\n   из принятых «эффект продолжался» (конец не известен): {censored}")
```

```
207
208         lines += ["", "=" * 100, "4. Скважины статистики", "=" * 100]
209         for status, count in wells["статус"].value_counts().items():
210             lines.append(f"   {count:>5}  {status}")
211
212         lines += ["", "=" * 100, "ИТОГ ДЛЯ ОБУЧЕНИЯ", "=" * 100]
213         used = wells[wells["статус"] == "используется, с разметкой"]
214         lines.append(f"   скважин с разметкой: {len(used)}, мероприятий: {len(accepted)}")
215         lines.append(
216             f"   скважин без разметки (только для поиска мероприятий): "
217             f"{(wells['статус'] == 'используется, без разметки').sum()}"
218         )
219         return "\n".join(lines)
```

Собирает консольный отчёт из пяти разделов:

1. Ячейки не с числом.
2. Сегменты и решения; в конце — доли ячеек по решениям («сдвиг −180 дн 74 050 ячеек (44%)» для склеенного файла). Если сегментов нет (полугодовые файлы), печатается пометка «нет: статистика из исходных полугодовых файлов, исправлять нечего», и пустая таблица не строится.
3. Воронка мероприятий итогового файла: сколько строк по каждой причине и таблица «год × принято / не принято».
4. Статусы скважин.
5. «Итог для обучения»: скважин с разметкой — 369, мероприятий — 424; скважин без разметки — 242 (по ним `predict` ищет мероприятия, но учиться на них нельзя).

Отдельно печатается, сколько из принятых мероприятий — «эффект продолжался» (censored; 81 из 424): у них неизвестен конец эффекта.

Принято по годам: 2021 — 48, 2022 — 49, 2023 — 82, 2024 — 59, 2025 — 109, 2026 — 77 (не принято: 74, 59, 42, 35, 105, 47). Раньше, на склеенном файле, за 2025 год не было принято ни одного мероприятия (скважин 2025 г. не было в статистике); теперь это самый большой год.

метод DataAuditReport.rejected_table

```
def rejected_table(self) -> pd.DataFrame
```

```
221     def rejected_table(self) -> pd.DataFrame:
222         """Мероприятия, отброшенные по трём главным причинам, — для уточнения у технологов."""
223         table = self.audit.labels[self.audit.labels["решение"].isin(REJECTED)].copy()
224         table["пояснение"] = table["решение"].map(REJECTED)
225         table = table.rename(columns={"решение": "причина"})
226         return table.sort_values(["причина", "год", "скважина"])
```

Мероприятия, отброшенные по трём главным причинам из словаря `REJECTED` (скважины нет в статистике; нет замеров около даты начала; частота в замерах не поднялась). Добавляется столбец «пояснение» понятным языком; столбец «решение» переименован в «причина». Сортировка: причина, год, скважина. Возвращает `DataFrame` (на полугодовых данных 313 строк — все с причиной «частота в замерах не поднялась»; на склеенном файле было 575 = 333 + 153 + 89).

Остальные причины (дубликат, не выполнено, частота не увеличена) технологам уточнять не нужно, поэтому в этот список они не входят.

метод DataAuditReport.save

```
def save(self, out_dir: Path) -> tuple[Path, Path, Path]
```

```
228     def save(self, out_dir: Path) -> tuple[Path, Path, Path]:
229         out_dir.mkdir(parents=True, exist_ok=True)
```

```
        csv = out_dir / "rejected_labels.csv"
        # «;» и BOM — чтобы Excel с русской локалью открыл файл без настройки импорта.
        self.rejected_table().to_csv(csv, sep=";", index=False, encoding="utf-8-sig")
        xlsx = out_dir / "data_audit.xlsx"
        with pd.ExcelWriter(xlsx) as writer:
            self.segments_table().to_excel(writer, sheet_name="сегменты", index=False)
            self.audit.labels.to_excel(writer, sheet_name="мероприятия", index=False)
            self.audit.wells.to_excel(writer, sheet_name="скважины", index=False)
            self.non_numeric_by_month().rename("ячеек «не число»").to_frame().to_excel(
                writer, sheet_name="не числа"
            )
        png = out_dir / "data_audit.png"
        self.plot(png)
        return xlsx, png, csv
```

Создаёт папку `out_dir` и записывает:

- `rejected_labels.csv` — таблица из `rejected_table`, разделитель `;`, кодировка `utf-8-sig` (BOM), чтобы Excel с русской локалью открыл файл сразу;
- `data_audit.xlsx` — листы «сегменты», «мероприятия» (все 786 строк с колонкой «решение»), «скважины» (616 строк со статусом), «не числа»;
- `data_audit.png` — график `plot`.

Возвращает кортеж путей `(xlsx, png, csv)`.

```
xlsx, png, csv = report.save(Path("results/ml_freq/debug/audit"))
```

метод DataAuditReport.plot

```
def plot(self, path: Path) -> None
```

```
    def plot(self, path: Path) -> None:
        a = self.audit
        figure, axes = plt.subplots(2, 1, figsize=(14, 7), sharex=True, sharey=True)
        panels = (
            (axes[0], a.wells_per_day_before, COLORS["before"], "До исправления (как в файле)"),
            (axes[1], a.wells_per_day_after, COLORS["after"], "После исправления"),
        )
        for axis, series, color, title in panels:
            for s in a.segments:
                if s.shift is None or s.shift:
                    fill = COLORS["dropped"] if s.shift is None else COLORS["shifted"]
                    if axis is axes[0]:
                        axis.axvspan(pd.Timestamp(s.start), pd.Timestamp(s.end), color=fill, lw=0)
            axis.plot(series.index, series.to_numpy(), color=color, lw=1.5)
            axis.set_title(title, loc="left", fontsize=11)
            axis.set_ylabel("скважин с замерами")
            axis.grid(axis="y", color="#e6e5e0", lw=0.8)
            for side in ("top", "right"):
                axis.spines[side].set_visible(False)
        handles = [
            plt.Rectangle((0, 0), 1, 1, color=COLORS["shifted"], label="сегмент сдвинут на −180
дней"),
            plt.Rectangle((0, 0), 1, 1, color=COLORS["dropped"], label="сегмент исключён"),
        ]
        axes[0].legend(handles=handles, loc="upper left", frameon=False)
        figure.suptitle("Статистика 2020–2026: скважин с замерами по дням", x=0.01, ha="left",
fontsize=13)
        figure.tight_layout()
        figure.savefig(path, dpi=130)
        plt.close(figure)
```

Рисует два графика один под другим с общими осями: число скважин с замерами по дням календаря «до исправления (как в файле)» и «после исправления». На верхнем графике сегменты, которые будут сдвинуты на −180 дней, закрашены голубым, исключённые — розовым. Сравнивать удобно: видно, как «прыгающий» календарь после исправления становится ровнее. Бэкенд `Agg` включён на уровне модуля — окно не открывается.

класс TrainingDataReport

`class TrainingDataReport`

```
275 class TrainingDataReport:
276     """Данные, на которых учатся модели: принятая разметка и исправленные замеры.
277 
278     Замеры — после исправления сдвига на 180 дней и исключения сегментов, только по
279     скважинам с принятой разметкой, в раскладке исходного файла статистики:
280     лист на параметр, строка — скважина, столбец — дата.
281     """
282 
    ...
292 
    ...
298 
    ...
311 
    ...
319 
```

Выгрузка «то, что видит модель». После исправления сдвигов и исключения сегментов берутся только скважины с принятой разметкой (369) и записываются в раскладке исходного файла статистики: лист на параметр, строка — скважина, столбец — дата. Так технолог может открыть файл и сравнить с исходным Excel.

Даты столбцов — непрерывный календарь от первого до последнего дня данных этих скважин (`pd.date_range`): с 01.01.2020 по 02.11.2026, всего 2 498 дней плюс два служебных столбца «№ скв.» и «№ куста».

Листы файла `training_data.xlsx`:

Лист	Содержимое
`мероприятия`	424 принятые строки разметки (заголовки как у технологов)
`Qж (из OIS)`, `Обв (из OIS)`, `Qн (из OIS)`, `Рлин (из OIS)`, `Fэцн ТМ(сут)`, `Рзаб`	замеры: 369 скважин × даты
`разметка по дням`	1 — день эффекта, 0 — эффекта нет, пусто — день не участвует в обучении

метод TrainingDataReport.__init__

`def __init__( self, wells: Sequence[WellSeries], audit: DataAudit, context_days: int = CONTEXT_DAYS ) -> None`

```
283     def __init__(
284         self, wells: Sequence[WellSeries], audit: DataAudit, context_days: int = CONTEXT_DAYS
285     ) -> None:
286         self.wells = [well for well in wells if well.labels]
287         self.audit = audit
288         self.context_days = context_days
289         start = min(well.frame.index[0] for well in self.wells)
290         end = max(well.frame.index[-1] for well in self.wells)
```

```
291         self.dates = pd.date_range(start, end)
```

Принимает список скважин `wells` (из репозитория), `audit` и `context_days` (по умолчанию `CONTEXT_DAYS` = 45). Оставляет скважины с непустой разметкой, вычисляет общую ось дат — от самого раннего первого дня до самого позднего последнего.

```
TrainingDataReport(wells, audit)   # 369 скважин, ось дат с 2020-01-01
```

метод TrainingDataReport.labels_table

```
def labels_table(self) -> pd.DataFrame
```

```
293     def labels_table(self) -> pd.DataFrame:
294         table = self.audit.labels[self.audit.labels["решение"] == ACCEPTED]
295         columns = ["строка Excel", *LABEL_HEADERS, "эффект продолжался"]
296         table = table[columns].rename(columns=LABEL_HEADERS)
297         return table.sort_values(["Скважина", "дата начала мониторинга"])
```

Принятые мероприятия (решение == `ACCEPTED`) из `audit.labels`; столбцы переименованы по `LABEL_HEADERS` в заголовки итогового файла технологов («Куст», «Скважина», «дата начала мониторинга», «Текущее значение», «Новое значение»...) и добавлен «эффект продолжался». Сортировка по скважине и дате начала. Столбец «строка Excel» позволяет найти исходную строку в файле технологов.

Пример: скважина 10335, куст 988, начало 14.08.2023, окончание 20.09.2023, частота 49,5 → 49,96 Гц, причина «прекращение эффекта по воде», строка Excel 307.

метод TrainingDataReport.wide

```
def wide(self, rows: Sequence[pd.Series]) -> pd.DataFrame
```

```
299     def wide(self, rows: Sequence[pd.Series]) -> pd.DataFrame:
300         """Ряды скважин -> таблица «скважина × дата» с заголовками как в исходном файле."""
301         values = [row.reindex(self.dates).to_numpy() for row in rows]
302         table = pd.DataFrame(values, columns=self.dates)
303         table.columns = [day.strftime("%d.%m.%Y") for day in self.dates]
304         ids = pd.DataFrame(
305             {
306                 MergedStatisticsReader.NUMBER: [well.number for well in self.wells],
307                 MergedStatisticsReader.CLUSTER: [well.cluster for well in self.wells],
308             }
309         )
310         return pd.concat([ids, table], axis=1)
```

Превращает ряды скважин (по одному `pd.Series` с датами в индексе на скважину, в порядке `self.wells`) в таблицу «скважина × дата». Каждый ряд приводится к общей оси дат (`reindex`, недостающие дни — пусто), заголовки столбцов дат — в формате `дд.мм.гггг`, перед ними столбцы «№ скв.» и «№ куста» (имена берутся из `MergedStatisticsReader`).

```
table = report.wide([well.frame["frequency"] for well in report.wells])
# лист «Фэцн ТМ(сут)»: 369 строк, 2 500 столбцов (2 498 дат + «№ скв.» и «№ куста»)
```

метод TrainingDataReport.target_table

```
def target_table(self) -> pd.DataFrame
```

```
312     def target_table(self) -> pd.DataFrame:
313         """1 — день эффекта по разметке, 0 — эффекта нет; пусто — день не участвует в обучении."""
314         rows = []
315         for well in self.wells:
```

```
            target = pd.Series(well.target(), index=well.frame.index)
            rows.append(target.where(well.training_mask(self.context_days)))
        return self.wide(rows)
```

Таблица целевой переменной: для каждой скважины `well.target()` (1 — день внутри размеченного мероприятия) оставляется только там, где `training_mask(context_days)` истинна, остальное — пусто. Это в точности те дни, которые попадают в функцию потерь: ±45 дней вокруг размеченного мероприятия, без дней после censored-мероприятий.

На текущих данных в листе `разметка по дням` 32 739 нулей и 16 301 единица; остальные ячейки пусты. Лист позволяет проверить, не обучается ли модель на днях, которые технолог не размечал.

метод TrainingDataReport.save

```
def save(self, path: Path) -> Path
```

```
    def save(self, path: Path) -> Path:
        path.parent.mkdir(parents=True, exist_ok=True)
        freeze = (1, 2)  # заголовок и номера скважины/куста видны при прокрутке
        with pd.ExcelWriter(path) as writer:
            self.labels_table().to_excel(writer, sheet_name="мероприятия", index=False)
            for column, sheet in STATISTICS_SHEETS.items():
                table = self.wide([well.frame[column] for well in self.wells])
                table.to_excel(writer, sheet_name=sheet, index=False, freeze_panes=freeze)
            self.target_table().to_excel(
                writer, sheet_name="разметка по дням", index=False, freeze_panes=freeze
            )
        return path
```

Принимает путь к `.xlsx`, создаёт папку и пишет все листы: «мероприятия», шесть листов замеров (имена листов — словарь `STATISTICS_SHEETS`, как в исходном файле статистики) и «разметка по дням». На листах с таблицами закреплены заголовок и два первых столбца (`freeze_panes=(1, 2)`), чтобы номер скважины был виден при прокрутке. Возвращает путь.

```
TrainingDataReport(wells, audit).save(out_dir / "training_data.xlsx")
```

функция _evidence

```
def _evidence(e) -> str
```

```
def _evidence(e) -> str:
    return f"{e.confirmed}/{e.tested}" if e is not None and e.tested else "—"
```

Форматирует «доказательство» сегмента: `"подтверждено/проверено"`, например `"6/8"`, или `"—"`, если проверки не было (`None` или ноль проверенных).

```
_evidence(None)   # '—'
```

функция _indent

```
def _indent(text: str) -> str
```

```
def _indent(text: str) -> str:
    return "\n".join("   " + line for line in text.splitlines())
```

Добавляет три пробела перед каждой строкой текста. Нужна, чтобы таблицы `pandas` в консольном отчёте были сдвинуты под заголовком раздела.

```
_indent("a\nb")   # '   a\n   b'
```

функция build_parser

```
def build_parser(prog: str | None = None) -> argparse.ArgumentParser
```

```
342 def build_parser(prog: str | None = None) -> argparse.ArgumentParser:
343     defaults = DataConfig()
344     parser = argparse.ArgumentParser(
345         prog=prog or "python -m gtm_detector.ml_freq.audit",
346         description="Отчёт о качестве объединённых данных 2021–2026.",
347     )
348     add_arguments(parser, defaults)
349     return parser
```

Создаёт `argparse`-парсер для самостоятельного запуска `python -m gtm_detector.ml_freq.audit`. Аргументы добавляет `add_arguments`; значения по умолчанию берутся из `DataConfig`. Параметр `prog` задаёт имя программы в справке.

```
.venv/Scripts/python -m gtm_detector.ml_freq.audit --help
```

функция add_arguments

```
def add_arguments(parser: argparse.ArgumentParser, defaults: DataConfig) -> None
```

```
352 def add_arguments(parser: argparse.ArgumentParser, defaults: DataConfig) -> None:
353     parser.add_argument("--statistics", type=Path, default=defaults.statistics, help="склеенная
    статистика")
354     parser.add_argument("--labels", type=Path, default=defaults.labels, help="итоговый файл
    технологов")
355     parser.add_argument(
356         "--reference", type=Path, default=defaults.reference, help="исходная выгрузка для сверки
    сдвигов"
357     )
358     parser.add_argument("--no-reference", action="store_true", help="не сверять с исходной выгрузкой")
359     parser.add_argument(
360         "--out-dir",
361         type=Path,
362         default=defaults.output_dir / "data_audit",
363         help="куда сохранить Excel и график",
364     )
```

Добавляет в парсер аргументы команды:

Аргумент	Смысл	По умолчанию
`--statistics`	папка полугодовых файлов `.xls` или один склеенный файл	`resources/statistics`
`--labels`	разметка технологов	`resources/labeled_data.xlsx`
`--reference`	эталонная выгрузка для сверки сдвигов склеенного файла	`None` (для полугодовых файлов не нужна)
`--no-reference`	не сверять с исходной выгрузкой	выключено
`--out-dir`	куда сохранить результаты	`results/ml_freq/data_audit` (актуальный запуск сохранён с `--out-dir results/ml_freq/data_audit_halfyear`)

Функция вынесена отдельно, чтобы `cli.py` мог добавить те же аргументы в подкоманду `audit`, не копируя код.

функция run

```
def run(args: argparse.Namespace) -> int
```

```
367 def run(args: argparse.Namespace) -> int:
368     reference = None if args.no_reference else args.reference
369     wells, audit = MergedWellRepository(args.statistics, args.labels, reference).build()
370     report = DataAuditReport(audit)
371     print(report.to_text())
372     xlsx, png, csv = report.save(args.out_dir)
373     training = TrainingDataReport(wells, audit).save(args.out_dir / "training_data.xlsx")
374     print(f"\nТаблицы: {xlsx}\nГрафик:  {png}\nОтброшенные мероприятия: {csv}")
375     print(f"Данные для обучения: {training}")
376     return 0
```

Выполняет аудит: строит репозиторий (`MergedWellRepository(...).build()` — в отличие от `load()` возвращает ещё и `DataAudit`), печатает `to_text()`, сохраняет Excel, график и CSV, а затем `training_data.xlsx`. Печатает пути к файлам. Возвращает код выхода 0.

При `--no-reference` сверка с исходной выгрузкой не делается, решения принимаются только по разметке: быстрее, но у сегментов без мероприятий проверять нечем, и они остаются на месте.

```
Таблицы: …\data_audit\data_audit.xlsx
График:  …\data_audit\data_audit.png
Отброшенные мероприятия: …\data_audit\rejected_labels.csv
Данные для обучения: …\data_audit\training_data.xlsx
```

функция main

```
def main(argv: Sequence[str] | None = None) -> int
```

```
379 def main(argv: Sequence[str] | None = None) -> int:
380     return run(build_parser().parse_args(argv))
```

Точка входа: разбирает аргументы (`argv` или `sys.argv`) и вызывает `run`. Возвращает код выхода для `sys.exit`.

```
main(["--out-dir", "results/ml_freq/debug/audit"])   # 0
```

cli.py

Командная строка пакета: `python -m gtm_detector.ml_freq <команда>`. Модуль только связывает готовые части — загрузку данных (`data`), модели (`models`), проверку и отчёт (`evaluation`), аудит (`audit`) — и не содержит собственных вычислений.

Четыре команды:

Команда	Что делает	Что создаёт
`compare`	перекрёстная проверка по скважинам, сравнение моделей	`comparison.xlsx`, `comparison.png` в `results/ml_freq/<дата_время>/`
`train`	обучение одной модели на всей разметке	веса в `gtm_detector/ml_freq/weights/` (`.pt` или `.joblib`)
`predict`	поиск мероприятий обученной моделью (разметка не нужна)	`events_<модель>.xlsx`
`audit`	отчёт о качестве данных	`data_audit.xlsx`, `training_data.xlsx` и др.

Примеры запуска из корня проекта:

```
.venv/Scripts/python -m gtm_detector.ml_freq compare --models gboost extra_trees      # около 2 минут
.venv/Scripts/python -m gtm_detector.ml_freq compare                                  # все модели, около 15
минут
.venv/Scripts/python -m gtm_detector.ml_freq compare --models transformer --epochs 20 # быстрая проверка
.venv/Scripts/python -m gtm_detector.ml_freq train --model gboost
.venv/Scripts/python -m gtm_detector.ml_freq predict --model gboost
.venv/Scripts/python -m gtm_detector.ml_freq audit
```

Те же команды заведены в `.vscode/launch.json` (вкладка Run and Debug, F5): «compare — gboost (быстро)», «compare — transformer, 2 эпохи», «train — выбранная модель (перезапишет веса)», «predict — выбранная модель». Отладочные запуски пишут в `results/ml_freq/debug/`.

`print` разрешён только здесь: это пользовательский вывод, а не лог библиотеки.

Код модуля вне классов и функций

```
15  from __future__ import annotations
16
17  import argparse
18  import logging
19  import sys
20  from collections.abc import Sequence
21  from datetime import datetime
22  from pathlib import Path
23
24  import pandas as pd
25
26  from .audit import add_arguments as audit_arguments
27  from .audit import run as run_audit
28  from .config import DataConfig, ExperimentConfig, TrainingConfig
29  from .data import MergedWellRepository, WellRepository
30  from .evaluation import ComparisonReport, CrossValidator, SegmentDecoder
31  from .models import TransformerSegmenter, load_model, model_registry
32
33  logger = logging.getLogger(__package__)
34
    ...
198 if __name__ == "__main__":
199     sys.exit(main())
```

функция _run_dir

```
def _run_dir(base: Path) -> Path
```

```
36 def _run_dir(base: Path) -> Path:
37     return base / datetime.now().strftime("%d-%m-%Y_%H-%M-%S")
```

Возвращает папку для результатов запуска: `base / "дд-мм-гггг_чч-мм-сс"` по текущему времени. Принимает базовую папку (обычно `results/ml_freq`), саму папку не создаёт. Уникальное имя не даёт новому запуску затереть старый отчёт.

```
_run_dir(Path("results/ml_freq"))   # results/ml_freq/29-10-2026_14-05-31
```

функция _registry

```
def _registry(args: argparse.Namespace) -> dict
```

```
40 def _registry(args: argparse.Namespace) -> dict:
41     registry = model_registry(seed=args.seed)
42     if args.epochs:
43         registry["transformer"] = lambda: TransformerSegmenter(
44             training=TrainingConfig(epochs=args.epochs, seed=args.seed)
45         )
46     return registry
```

Формирует словарь `{имя: фабрика_модели}` для запуска. Берёт `model_registry(seed=args.seed)` и, если указан `--epochs`, заменяет фабрику `transformer` так, чтобы модель училась заданное число эпох (по умолчанию 80, `TrainingConfig().epochs`). Остальные модели не затрагиваются. Зерно `--seed` передаётся и в модели, и в `TrainingConfig` — запуск воспроизводим.

```
args.epochs = 2
_registry(args)["transformer"]()   # TransformerSegmenter, 2 эпохи
```

функция load_wells

```
def load_wells(args: argparse.Namespace, labeled: bool) -> list
```

```
49 def load_wells(args: argparse.Namespace, labeled: bool) -> list:
50     """Скважины выбранного набора данных; ``labeled`` — только с разметкой."""
51     if args.dataset == "merged":
52         reference = None if args.no_reference else args.reference
53         wells = MergedWellRepository(args.statistics, args.labels, reference).load()
54     else:
55         wells = WellRepository(args.measurements, args.results if labeled else None).load()
56     return [well for well in wells if well.labels] if labeled else wells
```

Загружает скважины по выбранному набору данных. Принимает `args` и `labeled`:

- `--dataset merged` (по умолчанию): `MergedWellRepository(statistics, labels, reference).load()` — статистика 2020–2026: папка полугодовых `.xls` (по умолчанию) или один склеенный файл (тогда исправляются сдвиги на 180 дней); `--no-reference` отключает сверку склеенного файла с эталонной выгрузкой;
- `--dataset test`: `WellRepository(measurements, results)` — первые тестовые файлы (`resources/test_datas`, в проекте этой папки больше нет); разметка читается, только если `labeled=True`.

Если `labeled=True` (команды `compare`, `train`), остаются только скважины с разметкой — 369 на merged-наборе. При `labeled=False` (команда `predict`) возвращаются все 611 скважин с данными, в том числе 242

без разметки: именно на них ищутся новые мероприятия.

функция run_compare

```
def run_compare(args: argparse.Namespace) -> int
```

```python
def run_compare(args: argparse.Namespace) -> int:
    data = DataConfig()
    wells = load_wells(args, labeled=True)
    registry = _registry(args)
    unknown = set(args.models) - set(registry)
    if unknown:
        logger.error("Неизвестные модели: %s. Доступны: %s", sorted(unknown), sorted(registry))
        return 1
    print(
        f"Размеченных скважин: {len(wells)}, мероприятий: {sum(len(w.labels) for w in wells)}; "
        f"модели: {', '.join(args.models)}; фолдов: {args.folds}"
    )

    validator = CrossValidator(ExperimentConfig(folds=args.folds, seed=args.seed))
    results = validator.run(wells, {name: registry[name] for name in args.models})
    report = ComparisonReport(results)
    out_dir = args.out_dir or _run_dir(data.output_dir)
    report.save(out_dir)
    print("\n" + report.to_text())
    print(f"\nОтчёт: {out_dir / 'comparison.xlsx'}, график: {out_dir / 'comparison.png'}")
    return 0
```

Команда `compare`. Порядок:

1. Загружает размеченные скважины и реестр моделей.
2. Проверяет `--models`; неизвестное имя — сообщение в лог и код 1.
3. Печатает «Размеченных скважин: 369, мероприятий: 424; модели: …; фолдов: 5».
4. `CrossValidator(ExperimentConfig(folds, seed)).run(...)` — перекрёстная проверка по скважинам.
5. `ComparisonReport(results).save(out_dir)` и вывод `to_text()`.

Параметры: `--models` (по умолчанию все: `transformer`, `logreg`, `random_forest`, `extra_trees`, `gboost`), `--folds` (5), `--seed`, `--epochs`, `--out-dir`. Возвращает 0.

Результаты на 424 мероприятиях (`results/ml_freq/comparison/сравнение_качества_моделей_(20-26 год, полугодия).xlsx`, зерно 0): Градиентный бустинг — найдено 74 %, начало точно 36 %, ±3 дня 89 %, F1 0,136; Случайный лес — 68 %, F1 0,141; Extra Trees — 63 %, F1 0,131; Transformer — 46 %, F1 0,081; Логистическая регрессия — 61 %, F1 0,069. Precision и F1 с прежним прогоном (135 мероприятий, F1 0,46–0,48) сравнивать нельзя: теперь проверяется вся история скважин, а реальные подъёмы вне разметки считаются «лишними». Детектор на правилах (89 %) считали на других данных.

```
.venv/Scripts/python -m gtm_detector.ml_freq compare --models gboost --out-dir results/ml_freq/debug/compare
```

функция run_train

```
def run_train(args: argparse.Namespace) -> int
```

```python
def run_train(args: argparse.Namespace) -> int:
    data = DataConfig()
    wells = load_wells(args, labeled=True)
    model = _registry(args)[args.model]()
    model.fit(wells)
    path = data.weights_dir / f"{model.name}{model.file_suffix}"
    model.save(path)
    print(f"{model.title}: обучена на {len(wells)} скважинах -> {path}")
```

```
    return 0
```

Команда `train`: обучает одну модель на **всех** размеченных скважинах (без отложенной выборки — для боевого использования) и сохраняет веса в `weights_dir / "<имя><суффикс>"`: `transformer.pt` у PyTorch-модели, `gboost.joblib` и т. п. у моделей scikit-learn. Печатает «Градиентный бустинг (HistGB): обучена на 369 скважинах -> ...\weights\gboost.joblib». Параметр `--model` по умолчанию `transformer`; обучение Transformer на CPU долгое, поэтому запускайте в фоне. Размер весов: `extra_trees.joblib` 178 МБ, `random_forest.joblib` 85 МБ, остальные — килобайты и около 1 МБ.

Команда перезаписывает существующие веса.

```
.venv/Scripts/python -m gtm_detector.ml_freq train --model gboost
```

функция run_predict

```
def run_predict(args: argparse.Namespace) -> int
```

```
def run_predict(args: argparse.Namespace) -> int:
    data = DataConfig()
    path = args.weights or _find_weights(data.weights_dir, args.model)
    model = load_model(path)
    wells = load_wells(args, labeled=False)
    decoder = SegmentDecoder()

    rows = []
    for well in wells:
        for interval in decoder.decode(model.predict_proba(well), well.dates):
            rows.append(
                {
                    "скважина": well.number,
                    "куст": well.cluster,
                    "начало эффекта": interval.start,
                    "окончание эффекта": interval.end,
                    "статус": "эффект продолжается" if interval.end is None else "",
                }
            )
    out_dir = args.out_dir or _run_dir(data.output_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    out = out_dir / f"events_{model.name}.xlsx"
    pd.DataFrame(rows).to_excel(out, index=False)
    print(
        f"{model.title}: найдено {len(rows)} мероприятий на "
        f"{len({(r['скважина'], r['куст']) for r in rows})} скважинах -> {out}"
    )
    return 0
```

Команда `predict`: загружает веса (`--weights` или найденные `_find_weights`), прогоняет модель по всем скважинам, декодирует вероятности тем же `SegmentDecoder`, что и при проверке, и пишет Excel `events_<модель>.xlsx` со столбцами «скважина», «куст», «начало эффекта», «окончание эффекта», «статус». Для интервала без окончания «статус» = «эффект продолжается».

Результат `results/ml_freq/predict_gboost_merged/events_gboost.xlsx` (получен старыми весами, обученными на 122 скважинах склеенного файла): 770 мероприятий на 468 скважинах, из них 112 «эффект продолжается». Нашлось больше мероприятий, чем было размечено (135), потому что предсказания идут по всем скважинам, а не только по размеченным; технолог должен проверить найденное. С новыми весами результат будет другим.

```
.venv/Scripts/python -m gtm_detector.ml_freq predict --model gboost --out-dir results/ml_freq/debug/predict
```

функция _find_weights

```
def _find_weights(directory: Path, name: str) -> Path
```

```
123 def _find_weights(directory: Path, name: str) -> Path:
124     for suffix in (".pt", ".joblib"):
125         candidate = directory / f"{name}{suffix}"
126         if candidate.exists():
127             return candidate
128     raise FileNotFoundError(f"Нет весов модели {name} в {directory}: сначала выполните train")
```

Ищет в папке файл весов модели с именем `name`: сначала `<имя>.pt`, затем `<имя>.joblib`. Если ничего нет — `FileNotFoundError` с подсказкой «сначала выполните train». Папка `gtm_detector/ml_freq/weights` сейчас содержит `transformer.pt`, `gboost.joblib`, `logreg.joblib`, `random_forest.joblib`, `extra_trees.joblib`.

```
_find_weights(data.weights_dir, "gboost")   # …/weights/gboost.joblib
```

функция build_parser

```
def build_parser(prog: str | None = None) -> argparse.ArgumentParser
```

```
131 def build_parser(prog: str | None = None) -> argparse.ArgumentParser:
132     defaults = DataConfig()
133     names = list(model_registry())
134     parser = argparse.ArgumentParser(
135         prog=prog or f"python -m {__package__}",
136         description="ML-детекция мероприятий «увеличение частоты F»: Transformer и модели scikit-
    learn.",
137     )
138     parser.add_argument("-v", "--verbose", action="store_true", help="подробный лог")
139     commands = parser.add_subparsers(dest="command", required=True)
140 
141     def common(sub: argparse.ArgumentParser) -> None:
142         sub.add_argument(
143             "--dataset",
144             choices=("merged", "test"),
145             default=defaults.dataset,
146             help="merged — статистика 2020–2026 и разметка, test — первые тестовые
    (resources/test_datas)",
147         )
148         sub.add_argument("--statistics", type=Path, default=defaults.statistics, help="merged: папка
    полугодий или склеенный файл")
149         sub.add_argument("--labels", type=Path, default=defaults.labels, help="merged: разметка
    технологов")
150         sub.add_argument(
151             "--reference", type=Path, default=defaults.reference, help="merged: выгрузка для сверки
    склеенного файла"
152         )
153         sub.add_argument("--no-reference", action="store_true", help="merged: не сверять с выгрузкой")
154         sub.add_argument("--measurements", type=Path, default=defaults.measurements, help="test:
    замеры")
155         sub.add_argument("--results", type=Path, default=defaults.results, help="test: разметка")
156         sub.add_argument("--seed", type=int, default=0, help="зерно случайности")
157         sub.add_argument(
158             "--epochs", type=int, help=f"эпох обучения Transformer (по умолчанию
    {TrainingConfig().epochs})"
159         )
160 
161     compare = commands.add_parser("compare", help="перекрёстная проверка и сравнение моделей")
162     common(compare)
163     compare.add_argument("--models", nargs="+", default=names, help=f"какие модели: {'
    '.join(names)}")
164     compare.add_argument("--folds", type=int, default=5, help="фолдов по скважинам")
165     compare.add_argument(
166         "--out-dir", type=Path, help="папка отчёта (по умолчанию results/ml_freq/<дата_время>)"
```

```
    )
    compare.set_defaults(handler=run_compare)

    train = commands.add_parser("train", help="обучить модель на всей разметке и сохранить веса")
    common(train)
    train.add_argument("--model", choices=names, default="transformer")
    train.set_defaults(handler=run_train)

    predict = commands.add_parser("predict", help="найти мероприятия обученной моделью")
    common(predict)
    predict.add_argument("--model", choices=names, default="transformer")
    predict.add_argument("--weights", type=Path, help="файл весов (по умолчанию из ml_freq/weights)")
    predict.add_argument("--out-dir", type=Path, help="папка результата")
    predict.set_defaults(handler=run_predict)

    audit = commands.add_parser("audit", help="отчёт о качестве объединённых данных 2021–2026")
    audit_arguments(audit, defaults)
    audit.set_defaults(handler=run_audit)
    return parser
```

Строит парсер с подкомандами `compare`, `train`, `predict`, `audit`. Общий блок аргументов (`common`) для `compare`, `train`, `predict`: `--dataset {merged,test}`, `--statistics` (папка полугодовых файлов или склеенный файл), `--labels` (разметка технологов), `--reference` (эталон для склеенного файла, по умолчанию `None`), `--no-reference`, `--measurements`, `--results`, `--seed` (0), `--epochs`. Пути по умолчанию берутся из `DataConfig`: `resources/statistics` и `resources/labeled_data.xlsx`. Флаг `-v/--verbose` включает подробный лог. Для `audit` аргументы добавляет `audit.add_arguments`. Каждой подкоманде назначен `handler` (`run_compare`, `run_train`, `run_predict`, `run_audit`).

Доступные модели берутся из `model_registry()`; значит, новая модель, добавленная в реестр, сразу появляется в `--models` и `--model`.

```
.venv/Scripts/python -m gtm_detector.ml_freq compare --help
```

функция main

```
def main(argv: Sequence[str] | None = None, prog: str | None = None) -> int
```

```
def main(argv: Sequence[str] | None = None, prog: str | None = None) -> int:
    args = build_parser(prog).parse_args(argv)
    logging.basicConfig(
        level=logging.INFO if args.verbose else logging.WARNING,
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    )
    logging.getLogger(f"{__package__}.evaluation").setLevel(logging.INFO)
    return args.handler(args)
```

Точка входа. Разбирает аргументы, настраивает `logging` (по умолчанию уровень WARNING, с `-v` — INFO; логгер `evaluation` всегда на INFO, чтобы при `compare` было видно ход фолдов: «gboost: фолд 3/5 готов») и вызывает `args.handler(args)`. Возвращает код выхода команды, который передаётся в `sys.exit` в блоке `__main__`.

```
main(["audit", "--out-dir", "results/ml_freq/debug/audit"])   # 0
```

tests/conftest.py

Общие заготовки для всех тестов: **синтетические скважины** с заранее известными датами подъёма частоты и эффекта. Реальные Excel-файлы в тестах не читаются, поэтому весь набор (14 тестов) проходит за десятки секунд, а ожидаемый ответ известен до запуска.

Файл `conftest.py` pytest подхватывает сам: фикстура `wells` доступна в любом тесте по имени аргумента, а `make_well` и `day` тесты импортируют явно (`from gtm_detector.ml_freq.tests.conftest import day, make_well`).

В цепочке «Excel → скважины → признаки → модель → интервалы» синтетическая скважина заменяет первые два шага: сразу получается готовый `WellSeries` с таблицей `frame` и разметкой `labels`.

Код модуля вне классов и функций

```
 3  from __future__ import annotations
 4
 5  from datetime import date, timedelta
 6
 7  import numpy as np
 8  import pandas as pd
 9  import pytest
10
11  from gtm_detector.ml_freq.data import Interval, WellSeries
12
13  START = date(2026, 1, 1)
```

функция day

```
def day(offset: int) -> date
```

```
16  def day(offset: int) -> date:
17      return START + timedelta(days=offset)
```

Дата по номеру дня от начала календаря: `day(0)` — 1 января 2026 (константа `START`). Тесты записывают даты как числа, а в проверках переводят их в `date`.

Пример

```
day(0)    # date(2026, 1, 1)
day(40)   # date(2026, 2, 10)  - день подъёма частоты в make_well() по умолчанию
day(42)   # date(2026, 2, 12)  - начало эффекта в разметке
```

функция make_well

```
def make_well(n_days: int = 120, step: int | None = 40, effect: int = 30, seed: int = 0) -> WellSeries
```

```
20  def make_well(n_days: int = 120, step: int | None = 40, effect: int = 30, seed: int = 0) ->
    WellSeries:
21      """Частота 48 -> 50 Гц в день ``step``, Qж/Qн выше базы ``effect`` дней, затем возврат."""
22      rng = np.random.default_rng(seed)
23      index = pd.date_range(START, periods=n_days, freq="D")
24      frequency = np.full(n_days, 48.0)
25      liquid = np.full(n_days, 100.0)
26      labels = []
27      if step is not None:
28          frequency[step:] = 50.0
29          liquid[step + 2 : step + 2 + effect] = 110.0
30          labels.append(Interval(day(step + 2), day(step + 1 + effect), day(step)))
31      frame = pd.DataFrame(
```

```
        {
            "frequency": frequency,
            "debit_liquid": liquid + rng.normal(0, 0.3, n_days),
            "debit_oil": liquid * 0.1,
            "watercut": np.full(n_days, 90.0),
            "pressure_zab": np.full(n_days, 50.0),
            "pressure_line": np.full(n_days, 12.0),
            "state": np.ones(n_days),
        },
        index=index,
    )
    return WellSeries(number=f"W{seed}", cluster="1", frame=frame, labels=labels)
```

Собирает одну синтетическую скважину `WellSeries` с одним подъёмом частоты и одним периодом эффекта.

Параметры:

Параметр	По умолчанию	Смысл
`n_days`	120	длина ряда, дней
`step`	40	номер дня, когда частота поднимается; `None` — подъёма нет, разметки нет
`effect`	30	сколько дней дебит жидкости остаётся повышенным
`seed`	0	зерно шума; номер скважины получается `W<seed>`

Что внутри (для значений по умолчанию):

- `frequency` = 48 Гц, с дня `step` (40-й, 10.02.2026) — 50 Гц;
- `debit_liquid` = 100 м³/сут, а в дни `step+2 … step+1+effect` (42–71-й) равен 110, плюс шум N(0; 0,3) из `np.random.default_rng(seed)`. Эффект запаздывает на 2 дня после подъёма частоты, как и в реальных данных;
- `debit_oil` = 0,1 от дебита жидкости **без шума** (10, в эффекте 11), `watercut` = 90 %, `pressure_zab` = 50, `pressure_line` = 12, `state` = 1 (насос работает);
- разметка — один `Interval(start=day(step+2), end=day(step+1+effect), date_done=day(step))`: то есть 12.02.2026 — 13.03.2026, «дата выполнения» 10.02.2026.

Зерно случайности передаётся в `default_rng`, а не в глобальный `np.random` — правило проекта о воспроизводимости.

Пример

```
well = make_well()
well.frame.iloc[[0, 39, 40, 42, 71, 72]][["frequency", "debit_liquid", "debit_oil"]].round(2)
#             frequency  debit_liquid  debit_oil
# 2026-01-01       48.0        100.04       10.0
# 2026-02-09       48.0        100.45       10.0
# 2026-02-10       50.0         99.62       10.0   <- подъём частоты, дебит ещё прежний
# 2026-02-12       50.0        110.40       11.0   <- начало эффекта (разметка)
# 2026-03-13       50.0        110.21       11.0   <- конец эффекта (разметка)
# 2026-03-14       50.0        100.30       10.0   <- возврат к базе
len(well), well.number      # (120, 'W0')
well.labels                 # [Interval(start=2026-02-12, end=2026-03-13, date_done=2026-02-10)]
```

`make_well(step=None)` даёт скважину без события: частота 48 весь период, `labels == []`.

функция wells

```
def wells() -> list[WellSeries]
```

```python
@pytest.fixture
def wells() -> list[WellSeries]:
    return [make_well(step=30 + 3 * i, effect=25 + 2 * i, seed=i) for i in range(8)]
```

Фикстура pytest: **восемь скважин** `W0 … W7` для тестов моделей. Скважина `i` создаётся как `make_well(step=30+3*i, effect=25+2*i, seed=i)`: подъёмы и длины эффекта у всех разные, чтобы модели не могли выучить одну дату, а шум отличался.

Скважина	Подъём частоты (день)	Эффект с — по
W0	30	02.02.2026 — 26.02.2026
W1	33	05.02.2026 — 03.03.2026
W3	39	11.02.2026 — 13.03.2026
W7	51	23.02.2026 — 02.04.2026

Восьми хватает для `ExperimentConfig(folds=2)`: в каждом фолде по четыре скважины, у каждой ровно одно размеченное мероприятие, поэтому тест перекрёстной проверки может сверить `TP + FN == 8`.

Пример

```python
def test_something(wells):          # pytest подставит список из 8 скважин
    assert len(wells) == 8 and wells[3].number == "W3"
```

tests/test_data_and_features.py

Тесты слоя данных и признаков: разметка скважины (`WellSeries.target`, `training_mask`), `FeatureBuilder` и разбор «грязных» значений из Excel (`normalize_id`, `parse_date`). Все данные синтетические — скважины из `make_well` (см. `conftest.py`). Запуск файла целиком:

```
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests/test_data_and_features.py -q
```

Код модуля вне классов и функций

```
1  from datetime import date
2
3  import numpy as np
4
5  from gtm_detector.ml_freq.data import normalize_id, parse_date
6  from gtm_detector.ml_freq.features import FeatureBuilder
7  from gtm_detector.ml_freq.tests.conftest import make_well
```

функция test_target_and_training_mask

```
def test_target_and_training_mask()
```

```
10 def test_target_and_training_mask():
11     well = make_well(step=40, effect=30)
12     target = well.target()
13     assert target.sum() == 30 and target[42] == 1 and target[41] == 0
14     mask = well.training_mask(context_days=10)
15     assert mask[32] and not mask[31]  # 10 дней до начала (42)
16     assert mask[81] and not mask[82]  # 10 дней после окончания (71)
17     assert well.training_mask(None).all()
```

Что проверяет. Как размеченный интервал превращается в целевой вектор по дням и в маску дней, на которых разрешено учиться.

Данные. `make_well(step=40, effect=30)`: ряд 120 дней, частота поднята в день 40, эффект — дни 42–71 (12.02–13.03.2026).

Проверки:

- `target.sum() == 30`, `target[42] == 1`, `target[41] == 0` — единицы ровно на дни эффекта, включая оба конца;
- `training_mask(context_days=10)`: `mask[32]` истинно, `mask[31]` ложно — 10 дней контекста до начала (42 − 10 = 32); `mask[81]` истинно, `mask[82]` ложно — 10 дней после окончания (71 + 10 = 81). Всего в маске 50 дней: 10 + 30 + 10;
- `training_mask(None).all()` — без ограничения обучаются все дни.

Какую ошибку поймал бы. Сдвиг границ на день (`<` вместо `<=`), неверный отсчёт контекста: модель начала бы учиться на днях, где разметке верить нельзя (в разметке есть не все реальные подъёмы), или потеряла бы день начала. Ошибка в маске — нарушение правила «обучаться только на `training_mask`».

```
.venv/Scripts/python -m pytest
gtm_detector/ml_freq/tests/test_data_and_features.py::test_target_and_training_mask -q
```

функция test_features_shapes_and_values

```
def test_features_shapes_and_values()
```

```python
def test_features_shapes_and_values():
    builder = FeatureBuilder()
    well = make_well()
    sequence, tabular = builder.sequence(well), builder.tabular(well)
    assert sequence.shape == (len(well), builder.sequence_size)
    assert tabular.shape == (len(well), len(builder.tabular_names))
    assert not np.isnan(tabular).any() and np.abs(tabular).max() <= 3.0
    channel = builder.SEQUENCE_CHANNELS.index
    assert sequence[40, channel("f_diff")] > 0  # скачок частоты виден
    assert sequence[45, channel("liq_vs_step")] > 0.05  # Qж выше уровня до скачка
```

Что проверяет. Форму и разумность признаков `FeatureBuilder` и то, что в них видно событие.

Данные. `make_well()`: частота 48 → 50 Гц в день 40, дебит жидкости 100 → 110 в дни 42–71.

Проверки:

- `sequence` имеет форму `(120, 15)` (15 каналов `SEQUENCE_CHANNELS`), `tabular` — `(120, 106)` (по числу `tabular_names`);
- в табличных признаках нет `NaN`, а модуль значений не больше 3 (константа `CLIP`);
- канал `f_diff` в день 40 положителен (в тесте 0,4: скачок на 2 Гц при опорной частоте около 50 Гц, умноженный на 10);
- канал `liq_vs_step` в день 45 больше 0,05 (в тесте 0,094): дебит жидкости выше уровня до скачка примерно на 10 %.

Какую ошибку поймал бы. Добавленный канал без правки `SEQUENCE_CHANNELS` (форма не совпадёт), `NaN` из-за деления на ноль или пропусков, потерю обрезки значений, перепутанный знак разности, сдвиг каналов «от скачка» на день. Если канал добавлен намеренно, тест надо обновить вместе с README и помнить, что старые веса `.pt` перестанут загружаться.

```
.venv/Scripts/python -m pytest
gtm_detector/ml_freq/tests/test_data_and_features.py::test_features_shapes_and_values -q
```

функция test_parsing_helpers

`def test_parsing_helpers()`

```python
def test_parsing_helpers():
    assert normalize_id(" 561E ") == "561E" and normalize_id(7782.0) == "7782"
    assert parse_date("21.04.2026г") == date(2026, 4, 21)
    assert parse_date("08.02.206") == date(2026, 2, 8)
    assert parse_date("нет") is None
```

Что проверяет. Функции чтения «грязных» данных из `data/readers.py`, которые применяются и к разметке, и к статистике.

Данные. Реальные виды опечаток из Excel:

Вход	Результат	Зачем
`" 561E "`	`"561E"`	пробелы убраны, латинская `E` заменена кириллической `E`
`7782.0`	`"7782"`	номер скважины, прочитанный как float, без `.0`
`"21.04.2026г"`	`date(2026, 4, 21)`	лишняя буква после года
`"08.02.206"`	`date(2026, 2, 8)`	пропущена цифра в году (206 → 2026)
`"нет"`	`None`	не дата

Какую ошибку поймал бы. Номера скважин из статистики и разметки перестали бы совпадать (латиница и кириллица в «561Е» выглядят одинаково), и мероприятия молча «потерялись» бы; даты с опечатками превратились бы в `None` или в дату из 206 года.

```
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests/test_data_and_features.py::test_parsing_helpers -q
```

tests/test_models_and_evaluation.py

Тесты второй половины цепочки: «вероятности по дням → интервалы → сравнение с разметкой» и сами модели. Проверяются `SegmentDecoder`, `EventMatcher`, Transformer, scikit-learn-модель и `CrossValidator`. Обучение идёт на восьми синтетических скважинах `wells` (см. `conftest.py`); Transformer берётся уменьшенный (`SMALL`: `d_model=16`, 2 головы, 1 слой, `dim_feedforward=32`) и учится 3 эпохи по окнам в 64 дня (`QUICK`) — так тест укладывается в секунды, но проверяет все шаги: обучение, предсказание, сохранение, загрузку.

Тесты проверяют **механику**, а не качество: настоящие цифры даёт команда `compare`. Запуск файла целиком:

```
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests/test_models_and_evaluation.py -q
```

Код модуля вне классов и функций

```python
import numpy as np
import torch

from gtm_detector.ml_freq.config import DecoderConfig, ExperimentConfig, TrainingConfig,
TransformerConfig
from gtm_detector.ml_freq.data import Interval
from gtm_detector.ml_freq.evaluation import CrossValidator, EventMatcher, SegmentDecoder
from gtm_detector.ml_freq.models import (
    SklearnSegmenter,
    TimeSeriesTransformer,
    TransformerSegmenter,
    classic_models,
    load_model,
)
from gtm_detector.ml_freq.tests.conftest import day, make_well

SMALL = TransformerConfig(d_model=16, n_heads=2, n_layers=1, dim_feedforward=32)
QUICK = TrainingConfig(epochs=3, crop_length=64)
```

функция test_decoder_merges_gaps_and_marks_open_end

```
def test_decoder_merges_gaps_and_marks_open_end()
```

```python
def test_decoder_merges_gaps_and_marks_open_end():
    proba = np.zeros(40)
    proba[5:15] = 0.9
    proba[16:20] = 0.9  # разрыв в 1 день — склеится
    proba[35:] = 0.9  # доходит до конца — эффект продолжается
    decoder = SegmentDecoder(DecoderConfig(smooth_days=1, max_gap=2, min_length=2))
    events = decoder.decode(proba, [day(i) for i in range(40)])
    assert events == [Interval(day(5), day(19)), Interval(day(35), None)]
```

Что проверяет. Правила `SegmentDecoder`: склейку коротких разрывов и признак «эффект продолжается».

Данные. Вектор вероятностей на 40 дней: 0,9 на днях 5–14, провал в день 15 (1 день), 0,9 на днях 16–19, снова 0,9 с дня 35 до конца. Декодер настроен без сглаживания: `DecoderConfig(smooth_days=1, max_gap=2, min_length=2)`.

Ожидание. Ровно два интервала: `Interval(day(5), day(19))` (разрыв в 1 день ≤ `max_gap` склеен) и `Interval(day(35), None)` (период дошёл до последнего дня данных, поэтому `end=None`).

Какую ошибку поймал бы. Рассыпание одного мероприятия на два из-за шума модели (вырос бы счёт ложных срабатываний), потерю признака открытого конца (тогда у мероприятия «ПРОДОЛЖАЕТСЯ

ЭФФЕКТ» появилась бы выдуманная дата окончания), сдвиг границ на день.

```
.venv/Scripts/python -m pytest
gtm_detector/ml_freq/tests/test_models_and_evaluation.py::test_decoder_merges_gaps_and_marks_open_end -q
```

функция test_matcher_one_to_one_by_start

`def test_matcher_one_to_one_by_start()`

```
30  def test_matcher_one_to_one_by_start():
31      labels = [Interval(day(10), day(20)), Interval(day(60), day(70))]
32      predicted = [Interval(day(11), day(20)), Interval(day(12), day(25)), Interval(day(100), None)]
33      score = EventMatcher(start_tolerance=7).score(predicted, labels)
34      assert (score.true_positives, score.false_negatives, score.false_positives) == (1, 1, 2)
35      assert score.start_errors == [1] and score.end_errors == [0]
```

Что проверяет. Что `EventMatcher` сопоставляет мероприятия **один к одному** по дате начала с допуском ±7 дней и считает метрики `Score`.

Данные. Разметка: 10–20-й день и 60–70-й. Найдено три интервала: 11–20-й (рядом с первым), 12–25-й (тоже рядом с первым, но дальше) и 100-й до конца (далеко от всех).

Ожидание. Первую метку берёт ближайший интервал (разница 1 день, а не 2): `true_positives=1`, `false_negatives=1` (метка 60–70 не найдена), `false_positives=2` (второй кандидат к первой метке и интервал 100-го дня). Ошибка начала `[1]`, ошибка окончания `[0]`.

Какую ошибку поймал бы. Один найденный интервал, засчитанный сразу двум меткам (завышенная полнота), или два интервала на одну метку (занижение числа ложных срабатываний), выбор не ближайшей пары.

```
.venv/Scripts/python -m pytest
gtm_detector/ml_freq/tests/test_models_and_evaluation.py::test_matcher_one_to_one_by_start -q
```

функция test_transformer_forward_shape

`def test_transformer_forward_shape()`

```
38  def test_transformer_forward_shape():
39      network = TimeSeriesTransformer(n_channels=5, config=SMALL)
40      logits = network(torch.randn(2, 30, 5))
41      assert logits.shape == (2, 30)
```

Что проверяет. Форму выхода сети `TimeSeriesTransformer`: на вход батч `(2, 30, 5)` — 2 скважины, 30 дней, 5 каналов, на выход — `(2, 30)`: один логит на каждый день.

Какую ошибку поймал бы. Любой сбой в размерностях (позиционное кодирование, свёртка перед энкодером, итоговый слой): сеть, выдающая одно число на скважину, не годится для разметки по дням. Тест не требует данных и выполняется мгновенно.

```
.venv/Scripts/python -m pytest
gtm_detector/ml_freq/tests/test_models_and_evaluation.py::test_transformer_forward_shape -q
```

функция test_transformer_fit_predict_save_load

`def test_transformer_fit_predict_save_load(tmp_path, wells)`

```
44  def test_transformer_fit_predict_save_load(tmp_path, wells):
```

```
    model = TransformerSegmenter(SMALL, QUICK).fit(wells)
    proba = model.predict_proba(wells[0])
    assert proba.shape == (len(wells[0]),) and ((proba >= 0) & (proba <= 1)).all()
    path = tmp_path / "transformer.pt"
    model.save(path)
    assert np.allclose(load_model(path).predict_proba(wells[0]), proba, atol=1e-6)
```

Что проверяет. Полный жизненный цикл `TransformerSegmenter`: `fit` → `predict_proba` → `save` → `load_model`.

Данные. Фикстура `wells` (8 скважин). Модель `SMALL`, 3 эпохи.

Проверки: вероятности имеют длину ряда скважины и лежат в [0; 1]; после записи в `transformer.pt` и чтения через `load_model` (тип выбирается по расширению `.pt`) предсказания совпадают с исходными (`atol=1e-6`).

Какую ошибку поймал бы. Потерю части параметров или нормировок при сохранении (в весах хранится и порядок каналов `SEQUENCE_CHANNELS`), разные предсказания до и после загрузки, выход за [0; 1] (пропущена сигмоида).

```
.venv/Scripts/python -m pytest
gtm_detector/ml_freq/tests/test_models_and_evaluation.py::test_transformer_fit_predict_save_load -q
```

функция test_sklearn_models_learn_the_effect

```
def test_sklearn_models_learn_the_effect(tmp_path, wells)
```

```
def test_sklearn_models_learn_the_effect(tmp_path, wells):
    model = classic_models()["gboost"]().fit(wells)
    test = make_well(step=50, effect=30, seed=99)
    proba = model.predict_proba(test)
    assert proba[60:75].mean() > 0.7 and proba[:40].mean() < 0.3
    path = tmp_path / "gboost.joblib"
    model.save(path)
    assert isinstance(load_model(path), SklearnSegmenter)
```

Что проверяет. Что scikit-learn-модель (`gboost`) на табличных признаках действительно выучивает эффект, и что её можно сохранить и загрузить.

Данные. Обучение на `wells`. Проверка на **новой** скважине `make_well(step=50, effect=30, seed=99)`: подъём частоты в день 50, эффект в днях 52–81. В обучении такой скважины не было — так же, как при перекрёстной проверке по скважинам.

Проверки: средняя вероятность на днях 60–74 (внутри эффекта) выше 0,7, на днях 0–39 (до подъёма) ниже 0,3; после `save` в `gboost.joblib` и `load_model` получается `SklearnSegmenter`.

Какую ошибку поймал бы. Перепутанные метки (модель учит «не эффект»), развал признаков (например, смещение контекста `@±N` в `tabular`), поломку сохранения `.joblib`. Это тест на здравый смысл, а не на точность: на настоящих данных `gboost` даёт гораздо более скромные цифры (см. `compare`).

```
.venv/Scripts/python -m pytest
gtm_detector/ml_freq/tests/test_models_and_evaluation.py::test_sklearn_models_learn_the_effect -q
```

функция test_cross_validation_runs_for_all_models

```
def test_cross_validation_runs_for_all_models(wells)
```

```
def test_cross_validation_runs_for_all_models(wells):
```

```
    models = {"logreg": classic_models()["logreg"], "transformer": lambda: TransformerSegmenter(SMALL, QUICK)}
    results = CrossValidator(ExperimentConfig(folds=2)).run(wells, models)
    assert [r.name for r in results] == ["logreg", "transformer"]
    for result in results:
        assert result.score.true_positives + result.score.false_negatives == len(wells)
        assert 0.0 <= result.day_auc <= 1.0
```

Что проверяет. Что `CrossValidator` прогоняет через общую схему обе семьи моделей — scikit-learn и Transformer — и возвращает согласованный результат.

Данные. `wells` (8 скважин), `ExperimentConfig(folds=2)`, модели `logreg` и маленький Transformer.

Проверки: результаты идут в порядке `["logreg", "transformer"]`; для каждой модели `TP + FN` равно 8 — каждое размеченное мероприятие учтено ровно один раз (каждая скважина побывала в тестовом фолде один раз); ROC-AUC по дням `day_auc` в [0; 1].

Какую ошибку поймал бы. Потерю или дублирование скважин при разбиении на фолды, разную обработку моделей разных семейств (честное сравнение требует одинаковых фолдов и общего `SegmentDecoder`), ошибки в сборке метрик.

```
.venv/Scripts/python -m pytest
gtm_detector/ml_freq/tests/test_models_and_evaluation.py::test_cross_validation_runs_for_all_models -q
```

tests/test_repair.py

Тесты `data/repair.py`: проверка разметки по частоте (`FrequencyCheck`) и исправление сдвинутых блоков (`BlockShiftRepair`). Это самое хрупкое место данных: прежняя склеенная статистика 2021–2026 была склеена макросом, и часть выгрузок записана на 180 дней позже. В тестах сдвиг короче — `SHIFT = 30` дней, чтобы всё помещалось в ряд из 120 дней; логика та же, поэтому `BlockShiftRepair` создаётся как `BlockShiftRepair(shifts=(0, SHIFT))`.

Общая синтетическая картина (`_shifted_block`): шесть скважин, блок данных на 60 дней записан на 30 дней позже, чем надо. Запуск файла:

```
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests/test_repair.py -q
```

Код модуля вне классов и функций

```python
from __future__ import annotations

import numpy as np
import pandas as pd

from gtm_detector.ml_freq.data import BlockShiftRepair, FrequencyCheck, Interval
from gtm_detector.ml_freq.data.repair import LabelPoint
from gtm_detector.ml_freq.evaluation import EventMatcher

from .conftest import day, make_well

SHIFT = 30  # в тестах сдвиг короче настоящих 180 дней
```

функция test_frequency_check

```
def test_frequency_check()
```

```python
def test_frequency_check():
    check = FrequencyCheck()
    series = np.r_[np.full(20, 48.0), np.full(20, 50.0)]
    assert check.confirms(series, 20, 48.0, 50.0)
    assert check.confirms(series, 18, 48.0, 50.0)  # дата технолога на 2 дня раньше скачка
    assert check.confirms(series + 3, 20, 48.0, 50.0)  # телеметрия смещена, прирост тот же
    assert not check.confirms(series, 20, 48.0, 56.0)  # прирост 2 из запланированных 8 Гц
    assert not check.confirms(np.full(40, 48.0), 20, 48.0, 50.0)
    assert check.confirms(np.full(40, np.nan), 20, 48.0, 50.0) is None
```

Что проверяет. Решение `FrequencyCheck.confirms`: поднялась ли частота у даты начала так, как записано в разметке.

Данные. Ряд из 40 дней: 20 дней по 48 Гц, потом 20 дней по 50 Гц; технолог указал «48 → 50», скачок на 20-м дне.

Вызов	Ответ	Почему
`confirms(series, 20, 48, 50)`	`True`	подъём ровно в указанный день
`confirms(series, 18, 48, 50)`	`True`	дата технолога на 2 дня раньше скачка, допуск ±3 дня
`confirms(series + 3, 20, 48, 50)`	`True`	телеметрия смещена на 3 Гц, но прирост (2 Гц, т.е. не меньше половины запланированных) тот же

`confirms(series, 20, 48, 56)`	`False`	запланировано +8 Гц, фактически +2 — меньше половины
`confirms(const 48, 20, 48, 50)`	`False`	подъёма нет
`confirms(NaN, 20, 48, 50)`	`None`	замеров нет — проверить нельзя, это не опровержение

Какую ошибку поймал бы. Ужесточение или ослабление проверки: от неё зависит, какие мероприятия разметки допускаются к обучению и какой сдвиг блока считается подтверждённым. Отдельно важно различие `False` и `None`: «нет данных» не должно голосовать против сдвига.

```
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests/test_repair.py::test_frequency_check -q
```

функция _shifted_block

`def _shifted_block() -> tuple[np.ndarray, np.ndarray, list[LabelPoint], pd.DatetimeIndex]`

```python
def _shifted_block() -> tuple[np.ndarray, np.ndarray, list[LabelPoint], pd.DatetimeIndex]:
    """Скважины 0–5: подъёмы на дни 10+k, но блок 0–59 записан на SHIFT дней позже."""
    n_days = 120
    frequency = np.full((6, n_days), np.nan)
    labels = []
    for k in range(6):
        frequency[k, SHIFT : SHIFT + 60] = 48.0
        frequency[k, SHIFT + 10 + k : SHIFT + 60] = 50.0
        labels.append(LabelPoint(row=k, day=10 + k, current=48.0, new=50.0))
    return frequency, ~np.isnan(frequency), labels, pd.date_range("2026-01-01", periods=n_days)
```

Вспомогательная функция (не тест): строит данные для остальных тестов файла. Возвращает матрицу частот `frequency` (6 скважин × 120 дней, пропуски — `NaN`), маску присутствия данных `presence`, список `LabelPoint` и календарь, начинающийся 01.01.2026.

Устройство: у скважины `k` (0–5) разметка говорит «48 → 50 Гц в день `10 + k`». Но в файле блок записан на 30 дней позже: дни 30–89 заполнены (48 Гц, с дня `40 + k` — 50 Гц), остальное `NaN`. Правильный сдвиг, значит, 30; при «сдвиге 0» подъёма в нужных днях нет вообще.

Пример

```
frequency, presence, labels, dates = _shifted_block()
frequency.shape      # (6, 120)
labels[:2]           # [LabelPoint(row=0, day=10, current=48.0, new=50.0),
                     #  LabelPoint(row=1, day=11, current=48.0, new=50.0)]
dates[30]            # Timestamp('2026-01-31') - первый день блока
```

функция test_repair_finds_and_undoes_shift

`def test_repair_finds_and_undoes_shift()`

```python
def test_repair_finds_and_undoes_shift():
    frequency, presence, labels, dates = _shifted_block()
    repair = BlockShiftRepair(shifts=(0, SHIFT))
    segments = repair.segments(presence, dates)
    assert [(s.first, s.last) for s in segments] == [(SHIFT, SHIFT + 59)]

    repair.assess(segments, presence, frequency, labels)
    assert segments[0].shift == SHIFT
    assert segments[0].labels[SHIFT].confirmed == 6 and segments[0].labels[0].confirmed == 0

    table = pd.DataFrame(frequency, columns=dates)
```

```
51      repaired = repair.apply({"frequency": table, "debit_liquid": table}, segments)
    ["frequency"].to_numpy()
52      assert np.isnan(repaired[:, 60:]).all()
53      assert repaired[0, 9] == 48.0 and repaired[0, 10] == 50.0
```

Что проверяет. Весь путь исправления: найти сегмент → оценить сдвиг → применить.

Проверки:

1. `segments(presence, dates)` находит один сегмент — дни 30–89 (`SHIFT … SHIFT+59`): там 6 скважин появились и исчезли разом, а пустые края сегментами не считаются;
2. после `assess` у сегмента `shift == 30`; доказательства по разметке: для сдвига 30 подтверждено 6 мероприятий, для сдвига 0 — ни одного (в днях 10–15 данных нет, проверить нечем);
3. после `apply` блок вернулся на место: в днях 60 и позже только `NaN`, а у скважины 0 частота 48 Гц в день 9 и 50 Гц в день 10 — как в разметке.

Какую ошибку поймал бы. Сдвиг в обратную сторону, ошибку на единицу при переносе столбцов, потерю данных блока. Если сломается этот тест, обучение пойдёт на рядах, где подъём частоты и разметка не совпадают на полгода.

```
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests/test_repair.py::test_repair_finds_and_undoes_shift -
q
```

функция test_repair_drops_segment_contradicting_labels

```
def test_repair_drops_segment_contradicting_labels()
```

```
56  def test_repair_drops_segment_contradicting_labels():
57      frequency, presence, labels, dates = _shifted_block()
58      wrong = [LabelPoint(p.row, p.day + 7, p.current, p.new) for p in labels]  # ни 0, ни SHIFT
59      repair = BlockShiftRepair(shifts=(0, SHIFT))
60      segments = repair.segments(presence, dates)
61      repair.assess(segments, presence, frequency, wrong)
62      assert segments[0].shift is None
```

Что проверяет. Что сегмент, который не подтверждается **ни одним** из вариантов сдвига, исключается (`shift is None`), а не оставляется «на всякий случай».

Данные. Та же сдвинутая картина, но даты начала в разметке нарочно перенесены на 7 дней: подъёмы не лежат ни при сдвиге 0, ни при сдвиге 30.

Какую ошибку поймал бы. Слабую защиту от плохих данных: в обучение попал бы блок с неизвестным сдвигом, а модель училась бы на несовпадающих разметке и замерах (в прежнем склеенном файле исключалось около 17 % ячеек как раз по такому признаку).

```
.venv/Scripts/python -m pytest
gtm_detector/ml_freq/tests/test_repair.py::test_repair_drops_segment_contradicting_labels -q
```

функция test_seam_takes_neighbour_decision

```
def test_seam_takes_neighbour_decision()
```

```
65  def test_seam_takes_neighbour_decision():
66      frequency, presence, labels, dates = _shifted_block()
67      # Стык: за день до блока данные есть у тех же скважин и ещё у пяти других.
68      frequency = np.vstack([frequency, np.full((5, frequency.shape[1]), np.nan)])
69      presence = np.vstack([presence, np.zeros((5, presence.shape[1]), dtype=bool)])
70      presence[:, SHIFT - 1] = True
71      repair = BlockShiftRepair(shifts=(0, SHIFT))
```

```
    segments = repair.segments(presence, dates)
    repair.assess(segments, presence, frequency, labels)
    assert segments[0].days == 1 and segments[0].shift == segments[1].shift == SHIFT
```

Что проверяет. Обработку **стыка** выгрузок: сегмента в 1 день, который проверить нечем, потому что мероприятий там нет.

Данные. К шести скважинам добавлены ещё пять, у которых данные есть только в день 29, прямо перед блоком. Появляется сегмент из одного дня (29) и основной сегмент 30–89.

Ожидание. `segments[0].days == 1`, и у стыка то же решение, что у соседа: `shift == 30`. Без этого один день на границе остался бы на месте со сдвигом 0 и «торчал» бы из блока.

Какую ошибку поймал бы. Дырки и дубли на границах склейки выгрузок; решение для коротких сегментов, принимаемое по пустому множеству доказательств.

```
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests/test_repair.py::test_seam_takes_neighbour_decision -q
```

функция test_censored_label

```
def test_censored_label()
```

```
def test_censored_label():
    well = make_well(n_days=200, step=40, effect=30)
    end = well.labels[0].end
    well.labels = [Interval(well.labels[0].start, end, censored=True)]
    mask = well.training_mask(45)
    after_end = np.asarray(well.frame.index > pd.Timestamp(end))
    assert not mask[after_end].any() and mask[~after_end].any()

    found = [Interval(well.labels[0].start, day(150))]
    score = EventMatcher().score(found, well.labels)
    assert score.true_positives == 1 and score.end_errors == [] and score.open_ends == 0
```

Что проверяет. Обработку мероприятий со статусом «ПРОДОЛЖАЕТСЯ ЭФФЕКТ» (`censored=True`): наблюдение остановлено, но эффект продолжался. Тест проверяет `WellSeries.training_mask` и `EventMatcher`.

Данные. `make_well(n_days=200, step=40, effect=30)`; разметку заменили на тот же интервал с `censored=True` (начало 12.02.2026, «конец» 13.03.2026 — только последний день наблюдения).

Проверки:

- в `training_mask(45)` нет ни одного дня после 13.03.2026 (маска 72 дня — с самого начала ряда по день 71), хотя без `censored` в неё вошли бы ещё 45 дней: после последнего наблюдения про эффект ничего не известно, и штрафовать модель за «эффект» там нельзя;
- модель нашла мероприятие 12.02.2026 — 31.05.2026 (день 150): `true_positives == 1`, `end_errors == []`, `open_ends == 0`. Окончание у цензурированной метки не сравнивается: конец эффекта неизвестен.

Какую ошибку поймал бы. Штраф за правильно найденный длинный эффект (метрики окончания занизились бы), обучение на «пустых» днях после обрыва мониторинга.

```
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests/test_repair.py::test_censored_label -q
```

tests/test_statistics.py

Тесты чтения полугодовых файлов статистики (`data/statistics.py`): достать поток книги из `.xls`, разобрать лист в таблицу, склеить периоды и убедиться, что пустая папка не проходит молча. Настоящие файлы `resources/statistics/*/*.xls` тяжёлые (около трёх минут на чтение), поэтому тесты строят минимальные: файл OLE2 в несколько секторов и «лист» с интерфейсом `xlrd`. Запуск:

```
.venv/Scripts/python -m pytest gtm_detector/ml_freq/tests/test_statistics.py -q
```

Код модуля вне классов и функций

```
3  from __future__ import annotations
4
5  import struct
6
7  import numpy as np
8  import pandas as pd
9  import pytest
10 import xlrd
11
12 from gtm_detector.ml_freq.data.statistics import (
13     HalfYearStatisticsReader,
14     _join_periods,
15     _sheet_table,
16     _workbook_stream,
17 )
18
19 SECTOR = 512
20 END_OF_CHAIN, FREE, FAT_SECTOR = -2, -1, -3
```

функция _ole_file

```
def _ole_file(stream: bytes, declared: int) -> bytes
```

```
23 def _ole_file(stream: bytes, declared: int) -> bytes:
24     """Минимальный файл OLE2: сектор таблицы, сектор каталога, затем поток ``Workbook``."""
25     sectors = max(1, -(-len(stream) // SECTOR))
26     links = [FAT_SECTOR, END_OF_CHAIN, *range(3, sectors + 2), END_OF_CHAIN]
27     fat = struct.pack(f"<{SECTOR // 4}i", *links, *[FREE] * (SECTOR // 4 - len(links)))
28
29     name = "Workbook".encode("utf-16-le") + b"\0\0"
30     entry = bytearray(128)
31     entry[: len(name)] = name
32     struct.pack_into("<H", entry, 64, len(name))
33     struct.pack_into("<iI", entry, 116, 2, declared)
34     directory = bytes(entry) + bytes(SECTOR - 128)
35
36     header = bytearray(SECTOR)
37     header[:8] = b"\xd0\xcf\x11\xe0\xa1\xb1\x1a\xe1"
38     struct.pack_into("<H", header, 30, 9)  # размер сектора 2**9
39     struct.pack_into("<ii", header, 44, 1, 1)  # один сектор таблицы; каталог в секторе 1
40     struct.pack_into("<ii", header, 68, END_OF_CHAIN, 0)  # продолжения таблицы нет
41     struct.pack_into("<109i", header, 76, 0, *[FREE] * 108)
42     return bytes(header) + fat + directory + stream.ljust(sectors * SECTOR, b"\0")
```

Вспомогательная функция, не тест. Собирает в памяти минимальный файл формата OLE2 (старый `.xls`): заголовок 512 байт, сектор таблицы размещения (FAT), сектор каталога с одной записью `Workbook` и сам поток. Параметр `declared` — размер потока, **записанный в каталоге**; поток при этом может быть длиннее. Именно такое расхождение есть в настоящих выгрузках, из-за чего `xlrd` отказывается их открывать.

Пример

`_ole_file(bytes(range(256)) * 6, declared=513)` — поток из 1536 байт (3 сектора), в каталоге записано 513 байт (чуть больше одного сектора).

функция test_workbook_stream_follows_chain_longer_than_declared

`def test_workbook_stream_follows_chain_longer_than_declared(tmp_path): # Как в настоящих выгрузках: в каталоге размер меньше, чем секторов в цепочке.`

```
45  def test_workbook_stream_follows_chain_longer_than_declared(tmp_path):
46      # Как в настоящих выгрузках: в каталоге размер меньше, чем секторов в цепочке.
47      stream = bytes(range(256)) * 6  # 3 сектора
48      path = tmp_path / "book.xls"
49      path.write_bytes(_ole_file(stream, declared=SECTOR + 1))
50      assert _workbook_stream(path) == stream
```

Что проверяет. `_workbook_stream` идёт по цепочке секторов до конца, а не обрезает поток по размеру из каталога.

Данные. Поток 3 сектора, в каталоге указан размер `SECTOR + 1` = 513 байт.

Результат. Функция возвращает ровно исходные 1536 байт.

Какую ошибку поймал бы. Если бы читатель доверял размеру из каталога (как делает `xlrd`), книга обрезалась бы, и вместо данных за полугодие читались бы обрывки, либо снова появилось бы «size exceeds expected».

класс _Sheet

`class _Sheet`

```
53  class _Sheet:
54      """Лист с интерфейсом xlrd: строки значений и типов."""
55
    ...
59
    ...
62
```

Тестовый двойник листа `xlrd`: хранит строки значений и отдаёт их так же, как настоящий лист. Нужен, чтобы проверять `_sheet_table` без файла.

Пример

`_Sheet([["Версия", ""], ["№ скв.", "01.01.2021"]]).nrows == 2`.

метод _Sheet.__init__

`def __init__(self, rows: list[list]) -> None`

```
56      def __init__(self, rows: list[list]) -> None:
57          self.rows = rows
58          self.nrows = len(rows)
```

Запоминает строки и число строк `nrows`.

Пример

Лист из шести строк: `nrows == 6`.

метод _Sheet.row_values

```
def row_values(self, r: int) -> list
```

```
60      def row_values(self, r: int) -> list:
61          return self.rows[r]
```

Значения строки по номеру: как `sheet.row_values(r)` в `xlrd`.

Пример

`sheet.row_values(1)[2] == "01.01.2021"` — заголовок с датой.

метод _Sheet.row_types

```
def row_types(self, r: int) -> list[int]
```

```
63      def row_types(self, r: int) -> list[int]:
64          kinds = {float: xlrd.XL_CELL_NUMBER, str: xlrd.XL_CELL_TEXT}
65          return [
66              xlrd.XL_CELL_EMPTY if value == "" else kinds[type(value)] for value in self.rows[r]
67          ]
```

Типы ячеек строки: `float` -> `XL_CELL_NUMBER`, `str` -> `XL_CELL_TEXT`, пустая строка -> `XL_CELL_EMPTY`. Так `_sheet_table` отличает число от текста и от пустой ячейки.

Пример

Для строки `["101", "5А", 48.0, "ремонт"]` типы: текст, текст, число, текст.

функция _sheet

```
def _sheet(first: str, second: str, wells: list[list]) -> _Sheet: # Как в выгрузке: первая строка — версия, вторая — заголовок, третья пустая.
```

```
70  def _sheet(first: str, second: str, wells: list[list]) -> _Sheet:
71      # Как в выгрузке: первая строка — версия, вторая — заголовок, третья пустая.
72      return _Sheet(
73          [
74              ["Версия", "", "", ""],
75              ["№ скв.", "№ куста", first, second],
76              ["", "", "", ""],
77              *wells,
78          ]
79      )
```

Строит тестовый лист в раскладке выгрузки: строка 0 — версия, строка 1 — заголовок («№ скв.», «№ куста» и две даты `first`, `second`), строка 2 — пустая, далее скважины `wells`. Ровно как в настоящих файлах: данные с третьей строки.

Пример

`_sheet("01.01.2021", "02.01.2021", [["101", "5А", 48.0, "ремонт"]])`.

функция test_sheet_table_keeps_only_numbers

```
def test_sheet_table_keeps_only_numbers()
```

```
82  def test_sheet_table_keeps_only_numbers():
83      sheet = _sheet(
84          "01.01.2021",
```

```
        "02.01.2021",
        [
            ["101", "5А", 48.0, "ремонт"],
            ["", "", 1.0, 2.0],  # строка без номера скважины — итоги, не скважина
            ["101", "5А", 7.0, 7.0],  # повтор скважины: верна первая строка
            [202.0, "6", "", 50.0],
        ],
    )
    table, bad = _sheet_table(sheet)
    assert list(table.index) == [("101", "5А"), ("202", "6")]
    assert table.loc[("101", "5А")].iloc[0] == 48.0 and np.isnan(table.loc[("101", "5А")].iloc[1])
    assert np.isnan(table.loc[("202", "6")].iloc[0]) and table.loc[("202", "6")].iloc[1] == 50.0
    assert bad.tolist() == [0, 1]  # текст считается, пустая ячейка — нет
    assert table.columns[0] == pd.Timestamp("2021-01-01")
```

Что проверяет. `_sheet_table` оставляет только числа, пропускает строки без номера скважины и повторы, считает нечисловые ячейки.

Данные. Две даты, 01.01.2021 и 02.01.2021, и четыре строки:

Строка	Что в ней	Ожидание
`101 / 5А: 48.0, «ремонт»`	число и текст	48.0 и `NaN`; текст считается в `bad`
без номера: `1.0, 2.0`	итоги листа	строка пропущена
`101 / 5А: 7.0, 7.0`	повтор скважины	игнорируется, верна первая строка
`202.0 / 6: пусто, 50.0`	номер числом, пустая ячейка	скважина «202»: `NaN` и 50.0

Результат. Индекс `[("101", "5А"), ("202", "6")]`, счётчик нечисловых `bad == [0, 1]` (текст «ремонт» считается, пустая ячейка — нет), первый столбец — `2021-01-01`.

Какую ошибку поймал бы. Строка итогов, попавшая в таблицу как «скважина»; число, превращённое в текст; пустая ячейка, посчитанная как мусор (аудит показывал бы тысячи ложных «не чисел»).

функция test_join_periods_makes_continuous_calendar

```
def test_join_periods_makes_continuous_calendar()
```

```
def test_join_periods_makes_continuous_calendar():
    first = pd.DataFrame(
        {pd.Timestamp("2021-06-29"): [1.0], pd.Timestamp("2021-06-30"): [2.0]},
        index=pd.MultiIndex.from_tuples([("101", "5")]),
    )
    # Новая скважина появилась во втором файле, а между файлами есть пропущенный день.
    second = pd.DataFrame(
        {pd.Timestamp("2021-07-02"): [3.0, 4.0]},
        index=pd.MultiIndex.from_tuples([("101", "5"), ("202", "6")]),
    )
    table = _join_periods([first, second])
    assert list(table.columns) == list(pd.date_range("2021-06-29", "2021-07-02"))
    assert table.loc[("101", "5")].tolist()[:2] == [1.0, 2.0]
    assert np.isnan(table.loc[("101", "5")].iloc[2])  # 01.07 нет ни в одном файле
    assert table.loc[("101", "5")].iloc[3] == 3.0
    assert np.isnan(table.loc[("202", "6")].iloc[0]) and table.loc[("202", "6")].iloc[3] == 4.0
```

Что проверяет. `_join_periods` строит непрерывный календарь и объединяет скважины.

Данные. Первый файл: скважина 101/5 за 29.06 и 30.06.2021. Второй: 101/5 и новая 202/6 за 02.07.2021. Дня 01.07 нет ни в одном файле.

Результат. Четыре столбца с 29.06 по 02.07; у 101/5 значения 1, 2, `NaN` (01.07), 3; у 202/6 — `NaN` везде, кроме 02.07 (4.0).

Какую ошибку поймал бы. Если бы пропущенный день исчез из столбцов, окна «30 дней назад» в `FeatureBuilder` считали бы 30.06 и 02.07 соседними днями; потерянная новая скважина означала бы, что часть разметки «нет в статистике».

функция test_reader_requires_files

`def test_reader_requires_files(tmp_path)`

```
119 def test_reader_requires_files(tmp_path):
120     with pytest.raises(FileNotFoundError):
121         HalfYearStatisticsReader(tmp_path).read()
```

Что проверяет. Пустая папка (`tmp_path`) вызывает `FileNotFoundError` в `HalfYearStatisticsReader(...).read()`.

Какую ошибку поймал бы. Неверный путь в `--statistics`: без этой проверки обучение прошло бы на пустых данных или упало бы далеко от причины.